GEN-Z WEB

Document d'architecture

1. Choix et justification de l'architecture de l'application

Pour l'architecture du projet, j'ai opté pour une architecture en couches, elle offre de nombreux avantages:

- ***Modularité**: Chaque couche est conçue pour être autonome, ce qui permet de développer, tester et maintenir chaque couche de manière isolée. Cela facilite la gestion du code et optimise le temps de développement.*

- ***Séparation des tâches**: Chaque couche est dédiée à une tâche spécifique, par exemple dans cette application d'e-commerce:*

    - ***Interface utilisateur**: Cette couche va être visible dans le navigateur de l'utilisateur et avec lequel il va pouvoir interagir.*

    - ***Backend**: Cette couche va être responsable de gérer les requêtes de l'interface utilisateur, traiter les opérations métier et renvoyer les réponses appropriées.*

    - ***Service métier**: Cette couche va être responsable de la logique métier, encapsulant les règles métier et traitant les demandes des contrôleurs.*

    - ***API REST**: Cette couche va être responsable des contrôleurs, cela va permettre à l'interface utilisateur de communiquer avec le backend, de traiter les requêtes HTTP entrantes et invoquent les service métier appropriés.*

- ***L'accès aux données***: *Cette couche va gérer l'accès aux données persistantes dans l'application, en interagissant avec la base de données et en fournissant des opérations CRUD pour les entités.*

Cette séparation permet une meilleure organisation du code, le rendant plus lisible et simplifiant sa maintenance.

- ***Evolutivité***: *L'architecture en couches permet d'ajouter, de modifier ou de supprimer des fonctionnalités plus facilement, car les changements peuvent être limités à une seule couche sans affecter les autres couches.*

- ***Sécurité***: *Chaque couche peut bénéficier de contrôles d'accès et de mesures de sécurité, ce qui renforce la sécurité de l'application contre de potentielles menaces.*

2. Choix et justification des librairies

2.1. Librairies de test

2.1.1. Front-end

Pour le Frontend, j'ai opté pour les librairies open source Jest et Cypress. Jest est reconnue et largement utilisée, elle compte + de 12M d'utilisateurs sur GitHub et + de 43K étoiles. Elle présente de nombreux avantages:

- ***Performances***: *Jest optimise l'exécution des tests en réorganisant les suites selon leur durée et peut lancer plusieurs suites à la fois, ce qui permet de gagner du temps lors de tests répétitifs ou volumineux.*

- ***Facilité d'apprentissage***: *La syntaxe de Jest est simple/claire ce qui en fait une librairie facile à prendre en main et à maintenir même pour les débutants.*

- ***Couverture de test***: *Jest offre de nombreuses fonctionnalités pour couvrir le code (avec notamment `--coverage`), elle propose également la simulation d'objets (avec les mock).*

- ***Documentation***: *Le site officiel propose une doc*

complète et régulièrement mise à jour (env. à chaque release). De nombreuses ressources sont disponibles en ligne, telles que + de 22K questions sur Stackoverflow, une communauté Discord active et de nombreux tutoriels sur YouTube.

J'ai également choisi Cypress car, bien que Jest soit efficace pour tester les fonctionnalités du code, elle ne permet pas de tester les interactions en temps réel depuis le navigateur. L'ajout de Cypress permet donc de tester le code dans son ensemble. Bien que Cypress soit moins populaire que Jest avec seulement 1.3M d'utilisateurs sur GitHub, il à pourtant une meilleure note avec + de 45K étoiles. Cypress présente ces avantages:

- ***Fiabilité**: Cypress exécute ses tests directement dans le navigateur en temps réel, ce qui permet de simuler le comportement des utilisateurs de de tester de manière plus fidèle les interactions utilisateurs.*

- ***Facilité d'apprentissage**: Grâce à son interface intuitive, Cypress simplifie le processus de débogage, permettant de visualiser chaque étape.*

- ***Documentation**: Le site officiel de Cypress propose une doc approfondie, comprenant une FAQ détaillée, des exemples d'applications et une chaîne YouTube officielle. La documentation est mise à jour très régulièrement, et de nombreuses ressources sont disponibles en ligne, telles que + de 9K question sur Stackoverflow et une communauté Discord active.*

2.1.2. Back-end

Pour le Backend, j'ai opté pour la librairie open source JUnit 5. Avec + de 400K utilisateurs sur GitHub et + de 6K étoiles, JUnit 5 est une référence dans le domaine du test unitaire en Java. Elle présente de nombreux avantages:

- ***Rapidité d'exécution**: JUnit 5 s'efforce à améliorer les temps d'exécution des tests, ce qui permet de réduire le temps pour exécuter les suites de tests.*

- ***Facilité d'apprentissage**: JUnit 5 est conçu pour simplifier le processus d'écriture des tests, avec une syntaxe claire/simple et des fonctionnalités intuitives. Les tests de JUnit 5 sont également faciles à maintenir.*

- ***Documentation**: Le site officiel de JUnit 5 propose une doc complète et régulièrement mise à jour (env. chaque release). D'autres ressources sont également disponibles en ligne, telles que des tutoriels sur YouTube, il y a également + de 3K questions sur Stackoverflow.*

Pour accélérer le processus de développement des tests et les rendre plus compréhensible, je préconise d'adopter des scénarios BDD (Behavior Driven Development) pour chaque test. Cette approche permettra de réduire les erreurs lors de l'écriture des tests. Les scénarios BDD se composent généralement de 3 parties: Given (Étant donné), When (Lorsque), Then (Alors). En suivant cette structure, les tests deviennent plus faciles à comprendre et à maintenir.

2.2. Librairie de composants visuels

Pour les composants, j'ai choisi la librairie open source Material Angular. Avec + de 809K d'utilisateurs sur GitHub et + de 24.1K d'étoiles de notations. C'est une librairie largement utilisée dans le développement d'interfaces Angular. Voici ces nombreux avantages:

- ***Large gamme de composants**: Angular Material propose une large gamme de composants prêt à l'emploi.*

- ***UX**: Elle assure une expérience utilisateur familière et intuitive pour les utilisateurs.*

- ***UI**: Les composants ont une apparence professionnelle et moderne.*

- ***Normes et accessibilité**: Material Angular se conforme aux normes WCAG et RGAA, en mettant l'accent sur l'accessibilité. Ses composants conçus pour être utilisables par tous (notamment les personnes limitées physiquement ou cognitivement) grâce à des fonctionnalités telles que le*

support de lecteurs d'écran, gestion de la navigation au clavier, contraste des couleurs approprié, etc...

- ***Compatibilité navigateurs**: Material Angular est testé pour assurer une compatibilité avec les principaux navigateurs (Chrome, Firefox, Safari et Edge).*

- ***Performances**: La bibliothèque est conçue pour être légère et réactive, tout en minimisant les ralentissement et les temps de chargement de la page pour offrir une utilisation fluide et agréable même sur des appareils moins performants.*

3. Choix et justification des paradigmes de programmation

3.1. Front-end

Pour le frontend, j'ai choisi la programmation réactive. Ce paradigme est axé sur la gestion des flux de données asynchrones et des événements changeant dans l'application. Elle offre de nombreux avantages:

- ***Performances**: La programmation réactive est conçue pour optimiser les performances. Elle réduit les appels réseaux inutiles en réagissant uniquement aux changements de données pertinents, ce qui minimise les mises à jour de l'interface et améliora la réactivité de l'application.*

- ***Fiabilité**: La programmation réactive est conçue pour rester disponible même en cas de panne.*

- ***Facilité d'apprentissage**: La programmation réactive facilite la composition des fonctionnalités complexes. Avec des concepts tels que les observables et les opérateurs, le code devient plus facile à comprendre et à prédire. Ce qui facilite la compréhension et la maintenance du code.*

Avec la programmation réactive, je préconise d'utiliser la librairie open source RxJS. Elle simplifiera le développement du code, notamment grâce à l'utilisation des observables. Cela permettra d'avoir de meilleures performances, une meilleure modularité et une réduction des bugs. Cependant, il sera obligatoire d'inclure une copie complète de la licence Apache-2.0 dans un fichier LICENSE ou README du projet. Cela garantira

que le projet respecte les termes de la licence et que vous puissiez l'utiliser en toute conformité.

3.2. Back-end

Pour le backend, j'ai opté pour la programmation orientée objet, étant donnée que c'est le paradigme principal en Java. La POO offre une structure de code bien organisée, notamment avec l'utilisation de DTO, d'entités, de modèles, de répertoires, de services, etc. En plus de cette organisation, la POO présente plusieurs avantages:

- ***Facilité d'apprentissage**: La POO favorise la modélisation des objets du monde réel, rendant le code plus facile à comprendre, à développer et à maintenir.*

- ***Facilité de code**: Elle permet de réutiliser le code grâce à l'héritage et le polymorphisme, la POO permet également de diviser le code avec des classes, des objets, etc. Ce qui réduit les risques d'erreurs et facilite la maintenabilité.*

- ***Performances**: En utilisant une structure de données et un des algorithmes appropriés, la POO peut permettre des performances élevées.*

yoga-app

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.openclassrooms.starterjwt.security.services		70 %		100 %	5	29	12	32	5	26	0	3
com.openclassrooms.starterjwt.controllers		89 %		68 %	4	26	10	99	1	18	0	4
com.openclassrooms.starterjwt.services		98 %		91 %	1	23	0	41	0	17	0	3
com.openclassrooms.starterjwt.security		100 %		n/a	0	6	0	14	0	6	0	1
Total	94 of 812	88 %	6 of 34	82 %	10	84	22	186	6	67	0	11

All files

94.08% Statements `191/203` **90.9%** Branches `60/66` **95.78%** Functions `91/95` **93.75%** Lines `165/176`

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File		Statements		Branches		Functions		Lines	
src		50%	2/4	50%	1/2	0%	0/1	50%	2/4
src/app		100%	18/18	100%	6/6	100%	10/10	100%	13/13
src/app/components/me		100%	13/13	100%	2/2	100%	7/7	100%	12/12
src/app/components/not-found		50%	1/2	0%	0/2	100%	1/1	100%	1/1
src/app/features/auth		100%	6/6	100%	4/4	100%	2/2	100%	4/4
src/app/features/auth/components/login		100%	14/14	100%	2/2	100%	5/5	100%	13/13
src/app/features/auth/components/register		100%	11/11	100%	2/2	100%	5/5	100%	10/10
src/app/features/auth/services		100%	6/6	100%	2/2	100%	4/4	100%	5/5
src/app/features/sessions		100%	7/7	100%	4/4	100%	2/2	100%	5/5
src/app/features/sessions/components/detail		89.65%	26/29	100%	2/2	85.71%	12/14	88%	22/25
src/app/features/sessions/components/form		96.55%	28/29	93.75%	15/16	100%	9/9	96.42%	27/28
src/app/features/sessions/components/list		100%	6/6	100%	2/2	100%	3/3	100%	5/5
src/app/features/sessions/services		90.9%	10/11	100%	2/2	88.88%	8/9	90%	9/10
src/app/guards		75%	12/16	75%	6/8	100%	6/6	71.42%	10/14
src/app/interceptors		100%	6/6	100%	4/4	100%	3/3	100%	5/5
src/app/services		100%	24/24	100%	6/6	100%	14/14	100%	21/21
src/environments		100%	1/1	100%	0/0	100%	0/0	100%	1/1

All files

86.91% Statements `206/237` **29.41%** Branches `5/17` **72.13%** Functions `44/61` **86.53%** Lines `180/208`

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File		Statements		Branches		Functions		Lines	
app		78.57%	11/14	100%	0/0	33.33%	1/3	75%	9/12
app/components/me		70%	14/20	100%	0/0	33.33%	2/6	66.66%	12/18
app/components/not-found		100%	5/5	100%	0/0	100%	0/0	100%	3/3
app/features/auth/components/login		100%	21/21	100%	0/0	100%	4/4	100%	19/19
app/features/auth/components/register		100%	19/19	100%	0/0	100%	4/4	100%	17/17
app/features/auth/services		100%	14/14	100%	0/0	100%	3/3	100%	12/12
app/features/sessions/components/detail		76.31%	29/38	100%	0/0	53.84%	7/13	81.81%	27/33
app/features/sessions/components/form		65.78%	25/38	29.41%	5/17	37.5%	3/8	63.88%	23/36
app/features/sessions/components/list		100%	11/11	100%	0/0	100%	2/2	100%	9/9
app/features/sessions/services		100%	16/16	100%	0/0	100%	8/8	100%	14/14
app/services		100%	41/41	100%	0/0	100%	10/10	100%	35/35

ORION

Justification des choix techniques ***Projet MDD***

Auteur : BERNADOU Clara
Version 0.0.1

Aperçu / Synthèse **1**
Choix techniques **2 - 7**

Backend 2 - 3

- Framework
- Base de Données
- JPA
- Sécurité
- Mappage des Objets
- Lombok

Frontend 3 - 4

- Outil de Gestion de Projet
- Librairie de Composants
- Réactivité
- Stylisme

Autres 4 - 5

- Design Patterns

Architecture 5

Tableau de comparaison **6**

Backend 6

Frontend 6

Aperçu / Synthèse

*Le backend sera développé avec **Java** avec **Spring Boot**, qui facilite le développement de microservices et d'applications web. La gestion des données sera confiée à **MySQL**, une base de données relationnelle largement utilisée. **Lombok** simplifiera le code en générant automatiquement les méthodes courantes. **OAuth2** et **Spring Security** seront utilisés pour sécuriser l'application. **ModelMapper** aide à mapper les objets DTO et entités.*

*Le frontend sera développé en **Angular** avec **Angular CLI** sera utilisé pour gérer le projet Angular, avec **Angular Material** pour des composants UI modernes et réactifs. **Sass** et **Tailwind** offriront des outils puissants pour le stylisme CSS. **RxJS** fournira des outils pour la programmation réactive.*

*Des design patterns comme le **Service Layer Pattern**, le **Repository Pattern**, et le **Component Pattern** organiseront le code de manière modulaire et maintenable. Les **Reactive Forms** et **Dependency Injection (DI)** amélioreront la gestion des formulaires et l'injection de dépendances respectivement. Le **Observable Pattern** facilitera la gestion des flux de données.*

*Enfin, l'application suivra une **architecture client-server**, où le client (frontend) envoie des requêtes au serveur (backend), assurant une séparation claire des responsabilités.*

Choix techniques

Backend:

Choix de Framework

choix technique	lien vers le site / la documentation / une ressource	but du choix
Spring Boot	https://spring.io/projects/spring-boot	*Faciliter le développement d'applications web.*

***Spring Boot** est choisi pour sa facilité d'utilisation, sa capacité à gérer des applications complexes avec une configuration minimale et ses nombreuses fonctionnalités intégrées. Spring Boot était déjà intégré dans le code de Heidi.*

Choix de base de données

choix technique	lien vers le site / la documentation / une ressource	but du choix
MySQL	https://dev.mysql.com/downloads/connector/j/	*Permet la communication entre l'application et la base de données MySQL.*

***MySQL** est choisi pour sa robustesse, sa performance et sa popularité dans la communauté des développeurs. Il offre une gestion efficace des transactions et une intégration facile avec Spring Boot. MySQL était déjà intégré dans le code de Heidi.*

Choix de JPA

choix technique	lien vers le site / la documentation / une ressource	but du choix
Spring Data JPA	https://spring.io/projects/spring-data-jpa	*Simplification de l'accès à la base de données en utilisant le modèle de persistance Java (JPA).*

***Spring Data JPA** est choisi pour sa capacité à simplifier l'accès aux bases de données en fournissant une interface de haut niveau pour les opérations de persistance. Il est bien intégré avec Spring Boot, ce qui permet une configuration facile et une gestion fluide des entités JPA*

Choix de sécurité

choix technique	lien vers le site / la documentation / une ressource	but du choix
Spring Security OAuth2 Resource Server	https://docs.spring.io/spring-security/site/docs/current/reference/html5/#oauth2	*Sécurisation de l'application via une gestion des accès et des autorisations.*

Spring Security OAuth2 *est choisi pour sa capacité à gérer de manière sécurisée les autorisations et les authentifications. Il est bien intégré avec Spring Boot et offre une solution complète pour la sécurité de l'application.*

Choix mappage des objets

choix technique	lien vers le site / la documentation / une ressource	but du choix
ModelMapper	http://modelmapper.org/	*Mapper facilement les objets DTO et entités pour simplifier la gestion des données.*

ModelMapper *est choisi pour sa simplicité et son efficacité à transformer des objets de manière fluide et automatique. Cela améliore la maintenabilité du code et réduit la complexité des conversions.*

Choix Lombok

choix technique	lien vers le site / la documentation / une ressource	but du choix
Lombok	https://projectlombok.org/	*Réduire le code boilerplate () en générant automatiquement des getters, setters, et autres méthodes courantes.*

Lombok *est choisi pour sa capacité à simplifier le code Java en éliminant le besoin d'écrire manuellement des méthodes répétitives. Cela améliore la lisibilité et la maintenabilité du code.*

Frontend:

Choix d'outil de gestion de projet

choix technique	lien vers le site / la documentation / une ressource	but du choix
Angular CLI	https://www.npmjs.com/package/@angular/cli	*Simplifier la gestion des projets Angular, depuis la création jusqu'à la compilation.*

Angular CLI *est choisi pour sa capacité à automatiser et simplifier le workflow de développement Angular, permettant de gagner du temps et d'assurer une structure de projet cohérente. Angular CLI était déjà intégré dans le code de Heidi.*

Choix libraire de composants

choix technique	lien vers le site / la documentation / une ressource	but du choix
Angular Material	https://material.angular.io/guides	*Fournir des composants UI modernes et réactifs conformes aux directives Material Design.*

Angular Material *est choisi pour ses composants UI prêts à l'emploi, qui permettent de construire rapidement des interfaces utilisateur fonctionnelles. Angular Material était déjà intégré dans le code de Heidi.*

Choix de réactivité

choix technique	lien vers le site / la documentation / une ressource	but du choix
rxjs	https://rxjs.dev/	*Fournir des outils pour la programmation réactive et la gestion des flux de données asynchrones.*

RxJS *est choisi pour sa puissance et sa flexibilité dans la manipulation des données asynchrones, essentielle pour la réactivité et l'interactivité de l'application. RxJS était déjà intégré dans le code de Heidi.*

Choix style de l'application

choix technique	lien vers le site / la documentation / une ressource	but du choix
Tailwindcss	https://tailwindcss.com/	*Proposer une approche utilitaire pour le stylisme CSS, permettant une conception rapide et efficace.*
Sass	https://sass-lang.com/	*Faciliter le stylisme CSS avec des fonctionnalités avancées comme les variables, les mixins, et les fonctions.*

Tailwind CSS *est choisi pour sa flexibilité et sa capacité à créer des interfaces utilisateur rapidement sans avoir à écrire du CSS personnalisé.*

Sass *est choisi pour ses fonctionnalités puissantes qui augmentent la productivité et la maintenabilité du code CSS. Sass était déjà intégré dans le code de Heidi.*

Autres:

Choix des Design Patterns

choix technique	lien vers le site / la documentation / une ressource	but du choix
Service Layer Pattern	https://en.wikipedia.org/wiki/Service_layer_pattern	*Organiser la logique métier de manière modulaire et maintenable.*
Repository Pattern	https://medium.com/@pererikbergman/repository-design-pattern-e28c0f3e4a30	*Encapsuler la logique d'accès aux données et fournir une API propre pour les opérations CRUD.*
Component Pattern	https://en.wikipedia.org/wiki/Composite_pattern	*Modulariser le code frontend en composants réutilisables et indépendants.*
Reactive Forms Pattern	https://angularindepth.com/posts/1428/angular-forms-reactive-design-patterns	*Gérer les formulaires de manière réactive, facilitant la validation et la manipulation des données.*
Dependency Injection (DI) Pattern	https://en.wikipedia.org/wiki/Dependency_injection	*Faciliter l'injection de dépendances pour améliorer la testabilité et la modularité du code.*
Observable Pattern	https://fr.wikipedia.org/wiki/Observateur_(patron_de_conception)	*Gérer les flux de données asynchrones, essentiel pour la programmation réactive.*

*Le **Service Layer Pattern** est choisi pour sa capacité à structurer la logique métier de manière claire et modulaire, facilitant la maintenance et l'évolution de l'application.*

*Le **Repository Pattern** est choisi pour sa capacité à abstraire la couche d'accès aux données, rendant le code plus propre, testable et maintenable.*

*Le **Component Pattern** est choisi pour structurer le code Angular de manière modulaire, facilitant la réutilisation et la maintenance des composants UI.*

*Les **Reactive Forms** sont choisis pour leur capacité à gérer les formulaires complexes et les validations dynamiques de manière réactive, améliorant l'expérience utilisateur.*

*Le **DI pattern** est choisi pour structurer le code de manière à rendre les dépendances explicites, facilitant ainsi les tests et la maintenance.*

*L'**Observable Pattern** est choisi pour gérer les événements et les flux de données asynchrones de manière efficace, améliorant la réactivité de l'application.*

Choix de l'architecture

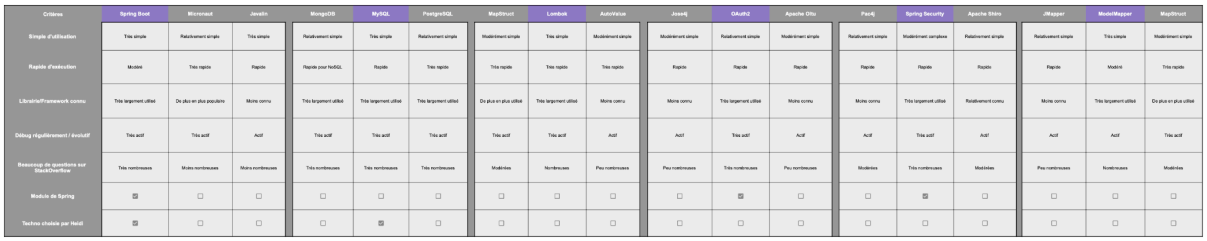
choix technique	lien vers le site / la documentation / une ressource	but du choix
Architecture Client-Server	https://en.wikipedia.org/wiki/Client%E2%80%93server_model	*Assurer une séparation claire des responsabilités*

*L'architecture **client-server** permet de séparer les préoccupations de l'interface utilisateur (client) et de la logique métier ainsi que de la gestion des données (serveur). Cela rend le code plus modulaire et plus facile à gérer.*

*Exemple dans l'appli: le client (**Angular**) est responsable de la présentation et de l'interaction utilisateur, tandis que le serveur (**Spring Boot**) gère les opérations de traitement de données et les règles métier.*

Comparaisons

Backend:



Critères	Spring Boot	Micronaut	Javalin	MongoDB	MySQL	PostgreSQL	MapStruct	Lombok	AutoValue	Jasypt	OAuth2	Apache Oltu	Pac4j	Spring Security	Apache Shiro	JMapper	ModelMapper	MapStruct
Simple d'utilisation	Très simple	Relativement simple	Très simple	Relativement simple	Très simple	Relativement simple	Modérément simple	Très simple	Modérément simple	Modérément simple	Relativement simple	Modérément simple	Relativement simple	Modérément complexe	Relativement simple	Relativement simple	Très simple	Modérément simple
Rapide d'exécution	Modéré	Très rapide	Rapide	Rapide pour NoSQL	Rapide	Très rapide	Très rapide	Très rapide	Très rapide	Rapide	Rapide	Rapide	Rapide	Rapide	Rapide	Rapide	Modéré	Très rapide
Librairie/Framework connu	Très largement utilisé	De plus en plus populaire	Moins connu	Très largement utilisé	Très largement utilisé	Très largement utilisé	De plus en plus utilisé	Très largement utilisé	Moins connu	Moins connu	Très largement utilisé	Moins connu	Moins connu	Très largement utilisé	Relativement connu	Moins connu	Très largement utilisé	De plus en plus utilisé
Débug régulièrement / évolutif	Très actif	Très actif	Actif	Très actif	Très actif	Très actif	Très actif	Très actif	Actif	Actif	Très actif	Actif	Actif	Très actif	Actif	Actif	Actif	Très actif
Beaucoup de questions sur StackOverflow	Très nombreuses	Moins nombreuses	Moins nombreuses	Très nombreuses	Très nombreuses	Très nombreuses	Modérées	Nombreuses	Peu nombreuses	Peu nombreuses	Très nombreuses	Peu nombreuses	Modérées	Très nombreuses	Modérées	Peu nombreuses	Nombreuses	Modérées
Module de Spring	☑	☐	☐	☐	☐	☐	☐	☐	☐	☐	☑	☐	☐	☑	☐	☐	☐	☐
Techno choisie par Heidi	☑	☐	☐	☐	☑	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐

https://docs.google.com/spreadsheets/d/187reVjb_WzJLQ_0pZpiWrz1olMO8vBcMQjIMb-naPBQ/edit?usp=sharing

Frontend:

Critères	Vite	Angular CLI	Nx (Narwhal Extensions)	PrimeNG	NG-ZORRO	Angular Material	Bootstrap	Sass	Tailwind	RxJS	NgRx	Redux
Simple d'utilisation	Très simple	Simple	Relativement simple	Relativement simple	Modérément simple	Relativement simple	Très simple	Modérément simple	Relativement simple	Modérément simple	Relativement complexe	Complexe
Rapide d'exécution	Très rapide	Modéré	Modéré	Rapide	Rapide	Modéré	Rapide	Rapide	Très rapide	Rapide	Rapide	Rapide
Librairie/Framework connu	De plus en plus connu	Très largement utilisé	De plus en plus connu	Bien connu	Bien connu	Très connu	Très largement utilisé	Très largement utilisé	De plus en plus connu	Très largement utilisé	De plus en plus utilisé	Très largement utilisé
Débug régulièrement / évolutif	Actif	Très actif	Actif	Actif	Actif	Très actif	Très actif	Très actif	Actif	Très actif	Actif	Très actif
Beaucoup de questions sur StackOverflow	Moins nombreuses	Très nombreuses	Moins nombreuses	Nombreuses	Nombreuses	Très nombreuses	Très nombreuses	Très nombreuses	Modérées	Très nombreuses	Nombreuses	Très nombreuses
Techno choisie par Heidi	☐	☑	☐	☐	☐	☑	☐	☑	☐	☑	☐	☐

https://docs.google.com/spreadsheets/d/10Jxovs9PgnDOHner-XcFlyN9DaDZ09U8LzBnPTz8P-8/edit?usp=sharing

Audit de l'Application Existante

Sommaire

1. **Contexte**
2. **Fonctionnalités**
3. **Expérience Utilisateur**
4. **Description Technique**
5. **Points Forts et Déficiences**
6. **Conclusion**

Contexte

LiVrai est une entreprise spécialisée dans la livraison de marchandises en grande quantité pour les professionnels. Actuellement, l'application de gestion des clients et des livraisons ne répond plus aux besoins croissants de l'entreprise et de ses clients en raison de l'expansion de son offre

---

Fonctionnalités

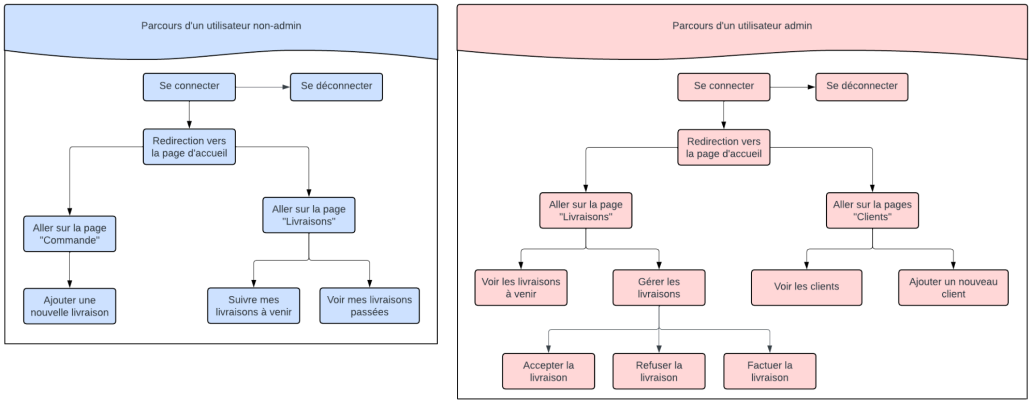
Fonctionnalités Générales :

Type d'utilisateur	Fonctionnalités
Client / Administrateur	• Se connecter • Se déconnecter

Gestion des Comptes Clients :

Type d'utilisateur	Fonctionnalités
Client	• Commander une nouvelle livraison • Afficher toutes les livraisons à venir de l'utilisateur • Afficher toutes les livraisons passées de l'utilisateur
Administrateur	• Ajouter un nouveau client • Afficher les livraisons à venir de chaque client • Gérer les livraisons : ○ Accepter la livraison ○ Rejeter la livraison ○ Facturer la livraison • Afficher les livraisons passées de chaque client

Schéma :



Expérience Utilisateur

Page(s)	Type d'utilisateur(s)	Problème	Amélioration
Toutes	Utilisateur / Administrateur	L'interface actuelle est trop simpliste et manque d'attrait visuel, ce qui peut décourager les utilisateurs	
Toutes	Utilisateur / Administrateur	Les erreurs sont présentées sous forme de rapports qui interrompent la navigation dans l'application, rendant leur compréhension difficile pour l'utilisateur.	Présenter les erreurs sous forme de messages clairs et non intrusifs, sans interrompre la navigation, afin de faciliter leur compréhension et permettre à l'utilisateur de continuer à utiliser l'application.

Accueil	Utilisateur / Administrateur	La page est quasiment vide et n'apporte aucune information utile.	Réorienter directement vers une page où il y a plus d'interactions
Clients	Administrateur	Les champs de saisie ne sont pas validés correctement, ce qui permet d'entrer des données erronées ou de laisser les champs vides.	/
Livraisons / Clients	Utilisateur / Administrateur	Lorsque un tableau est long, il faut scroller jusqu'en bas de celui-ci pour accéder à l'élément en suivant.	Ajout d'un système de pagination à un certain nombre d'éléments dans un tableau.
Livraisons / Clients	Utilisateur / Administrateur	Difficulté à trouver un élément lorsqu'il y a de nombreux éléments dans un tableau.	Ajout d'une barre de recherche et/ou de filtres pour faciliter la recherche de clients.
Livraisons	Utilisateur	Une erreur "Method Not Allowed" (405) se produit lors d'une requête POST en cas de refus de livraison.	/
Livraisons	Utilisateur / Administrateur	Les tableaux des livraisons passées présentent des problèmes de formatage : **Administrateur** : Certaines colonnes sont décalées, avec les noms apparaissant dans la colonne "volume" et d'autres informations mal positionnées.	/

Mes livraisons passées

Id	Volume	Poids	Prix	Statut
1	BERNADOU	12	12 123.0	Terminée
3	BERNADOU	78	67 19.0	Terminée
6	BERNADOU	57	99 1.0	Terminée

Utilisateur : Le poids et le prix sont affichés dans la même colonne, déplaçant le statut dans la colonne des prix, ce qui rend le tableau difficile à lire et confus.

Mes livraisons passées

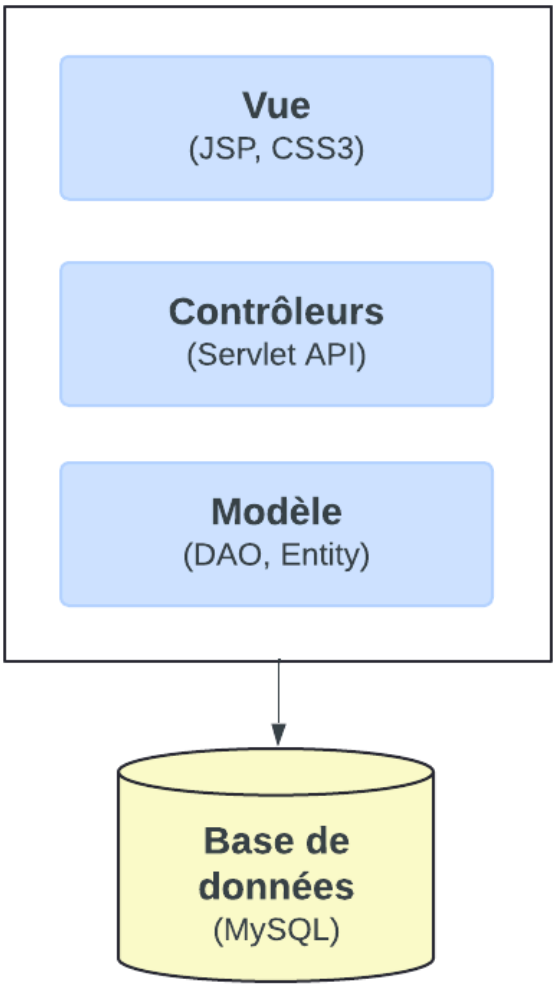
Id	Volume	Poids	Prix	Statut
1	12	12 123.0	Terminée	
3	78	67 19.0	Terminée	
6	57	99 1.0	Terminée	

Description Technique

Composant	**Version**
Architecture	Architecture en couches (MVC)
Langages	
Java	6

JSP (JavaServer Pages)	3.0
CSS	3
Frameworks	
Servlet API	3.0.1
MySQL	8.0.32
Plugins Maven	
maven-clean-plugin	3.1.0
maven-resources-plugin	3.0.2
maven-compiler-plugin	3.8.0
maven-surefire-plugin	2.22.1
maven-war-plugin	3.2.2
maven-install-plugin	2.5.2
maven-deploy-plugin	2.8.2

Schéma :



Points Forts et Déficiences

Points Forts :

- **Séparation des préoccupations :**
  L'architecture en couches (Servlets, DAO, Beans, JSPs) est bien implémentée, ce qui permet une organisation claire du code. Chaque couche remplit une fonction spécifique, ce qui facilite la maintenance et l'évolution de l'application.

- **Sécurité des requêtes SQL :**
  L’utilisation de requêtes préparées via prepareStatement permet d’éviter les attaques par injection SQL, augmentant ainsi la sécurité des transactions avec la base de données.

- **Gestion partielle de l'authentification :**
  La présence d’un filtre pour gérer une partie de l’authentification montre une prise en compte de la sécurité. Cela peut être renforcé avec des mécanismes d'authentification plus robustes.

- **Performances optimales :**
  Les temps de chargement sont très rapides, grâce à l'optimisation des requêtes et la légèreté des interactions entre le frontend et le backend.

Déficiences :

- **Manque de réactivité (Responsive) :**
  L'application n'est pas responsive, ce qui la rend difficilement utilisable sur des appareils mobiles ou des écrans de tailles différentes. Un redesign avec des techniques de responsive design est nécessaire pour améliorer l'expérience utilisateur.

- **Exposition des données sensibles :**
  Les informations sensibles comme les identifiants de la base de données, les drivers, et autres paramètres critiques sont visibles en clair, car il n'y a pas de fichier de configuration sécurisé (ex. : application.properties ou .env). Cela pose un risque majeur pour la sécurité de l'application.

- **Fermeture incorrecte des statements :**
  Certains statements ne sont pas fermés correctement après leur exécution, ce qui peut entraîner des fuites de mémoire et des performances dégradées sur le long terme.

- **Connexion à la base de données non fermée :**
  Il y a des oublis dans la fermeture des connexions à la base de données après utilisation. Cela peut saturer le pool de connexions et provoquer des problèmes de disponibilité à mesure que l’application prend de l’ampleur.

- **Absence de tests unitaires :**
  L'application actuelle ne dispose d'aucune suite de tests unitaires pour vérifier le bon fonctionnement des différentes fonctionnalités. Sans tests pour vérifier le bon fonctionnement du code, il y a un risque accru d'introduire des régressions ou des bugs lors de l'ajout de nouvelles fonctionnalités ou de corrections.

- **Architecture monolithique :**
  L'application actuelle est basée sur une architecture monolithique*, ce qui rend difficile la scalabilité et la maintenance à long terme. Toute modification ou mise à jour, même minime, nécessite de redéployer l'ensemble de l'application. Cela crée une rigidité dans les processus de développement et de déploiement, et empêche la mise en place de microservices qui pourraient offrir une plus grande flexibilité et une meilleure répartition des charges.

*Architecture monolithique : Regroupe toutes ses fonctionnalités dans un seul et même bloc de code.

Conclusion

L’audit de l’application actuelle de LiVrai met en lumière plusieurs points forts, mais également des déficiences importantes qui compromettent son évolutivité et sa sécurité. Bien que l’architecture en couches soit bien structurée et que les performances soient optimisées, plusieurs aspects nécessitent une refonte, notamment l’expérience utilisateur, la gestion des erreurs, et la sécurité des données. L’absence de réactivité et la mauvaise gestion des connexions et des statements peuvent engendrer des risques à long terme.

La modernisation de l'application, en intégrant des solutions plus robustes et adaptées à la croissance de l’entreprise, est indispensable pour maintenir la qualité du service. L'ajout de nouvelles fonctionnalités, une interface plus intuitive, et une meilleure gestion de la sécurité devraient permettre à LiVrai de répondre efficacement à ses besoins actuels et futurs.

Nouveau cahier des charges

Sommaire

1. **Contexte du projet**
    a. Besoins du client
    b. Objectifs du projet
    c. Mesure de la réussite du projet
    d. Contraintes réglementaires
    e. Éléments hors périmètre
2. **Description fonctionnelle**
    a. Liste des fonctionnalités
    b. Fonctionnement de l’application
    c. Impact Social
3. **Description technique**

Contexte du projet

Besoins du client :

LiVrai est une entreprise spécialisée dans la livraison de marchandises en grande quantité pour les professionnels. Actuellement, l'application de gestion des clients et des livraisons ne répond plus aux besoins croissants de l'entreprise et de ses clients en raison de l'expansion de son offre.

L’application actuelle centralise la gestion des comptes clients et des livraisons. Pour un client, le processus commence par l’intervention d’un administrateur, chargé de créer les comptes clients. Une fois le compte créé, les clients peuvent se connecter pour passer leurs commandes, consulter leurs livraisons à venir, ainsi que l’historique de leurs livraisons. Quant aux administrateurs, leurs comptes sont créés en interne dans la base de données. Une fois leurs comptes créés, les administrateurs peuvent se connecter pour ajouter des clients et consulter les livraisons à venir ainsi que l’historique des livraisons des clients.

Avec la croissance rapide de LiVrai, notamment l'augmentation du nombre de clients et des livraisons, l'application actuelle présente plusieurs limitations qui ne répondent plus aux exigences actuelles :

- **Dépendance au service commercial :**
  Actuellement, chaque client doit passer par le service commercial pour la création d'un compte. Cette approche ralentit le processus et limite l'autonomie des clients, alors que nous souhaitons désormais que la majorité d'entre eux puissent gérer leurs livraisons de manière indépendante.
- **Absence de certaines fonctionnalités clés :**
  La création de compte et la gestion de la facturation, par exemple, ne sont pas intégrés dans l'application actuelle.
- **Gestion limitée des utilisateurs :**
  L'application ne différencie pas assez clairement les différents types d'utilisateurs (clients, service commercial, service Livraisons), ce qui complique la gestion des permissions et des fonctionnalités spécifiques à chaque rôle.

Objectifs du projet :

Certaines fonctionnalités sont incontournables pour cette refonte :

- **Création de compte**
- **Gestion des informations clients**
- **Commande et suivi des livraisons**
- **Gestion de la facturation**
- **Historique des livraisons**

L'application devra également gérer trois types d'utilisateurs :

- **Les clients**
- **Le service commercial**
- **Le service livraisons**

Sur le plan technique, l'application devra répondre aux exigences suivantes :

- **Frontend**
  sera une application monopage (SPA) développée en **Angular**
- **Backend**
  sera développé en **Java**, dans sa version LTS (Long Term Support)
- **Déploiement en production**
  se fera dans un environnement **Docker**

Mesure de la réussite du projet :

- **Satisfaction des utilisateurs :**
  Évaluer la satisfaction des utilisateurs via des enquêtes et des tests d'utilisabilité après le déploiement.
- **Performance de l'application :**
  Mesurer les temps de réponse et la fluidité de l'application avant et après la refonte.
- **Réduction des bugs :**
  Diminuer le nombre de rapports de bugs et d'erreurs post-déploiement.
- **Taux de réussite de livraisons :**
  Mesurer l'efficacité de l'application en suivant le pourcentage de livraisons réalisées sans incidents ou retards. Un taux de réussite élevé démontrera la fiabilité du nouveau système.
- **Coûts opérationnels :**
  Une réduction significative des coûts opérationnels reflétera l'efficacité de la refonte et la durabilité à long terme de l'application.

Contraintes réglementaires :

- **Protection des données personnelles :**

Garantir que les données des utilisateurs sont stockées de manière sécurisée et traitées conformément aux exigences du Règlement Général sur la Protection des Données (RGPD).

- **Droit d'accès et de rectification :**
  Permettre aux utilisateurs de consulter, corriger, ou supprimer leurs données personnelles via une interface dédiée dans l'application.

- **Gestion des consentements :**
  Intégrer des fonctionnalités pour obtenir, gérer et enregistrer le consentement des utilisateurs concernant le traitement de leurs données personnelles.

- **Notification de violation de données :**
  Mettre en place des procédures pour détecter, signaler, et gérer toute violation de données dans les délais requis par le RGPD (72 heures). En cas de fuite ou de compromission d'informations personnelles, les utilisateurs doivent être immédiatement informés et des mesures correctives doivent être mises en œuvre.

- **Conditions générales et politiques de vente (CGV) :**
  Rédiger et intégrer les Conditions Générales de Vente dans l'application. Ces CGV doivent encadrer les relations commerciales entre LiVrai et ses clients, précisant les obligations de chaque partie.

- **Conditions générales d'Utilisation (CGU) :**
  Rédiger et afficher clairement les Conditions Générales d'Utilisation dans l'application. Celles-ci doivent préciser les droits et obligations des utilisateurs lors de l'utilisation de l'application, notamment en ce qui concerne la propriété intellectuelle, les responsabilités, et l'usage des services proposés.

- **Mentions légales :**
  Inclure les mentions légales obligatoires dans l'application, telles que l'identification de l'éditeur, l'hébergeur, les coordonnées de l'entreprise, ainsi que les informations liées à la protection des données. Ces informations doivent être facilement accessibles et conformes aux exigences légales en vigueur.

Éléments hors périmètre :

- **Intégration d'un système de paiement (exemple : Stripe)**
  Permet aux clients de régler directement leurs factures en ligne via l'application.
- **Notifications en temps réel**
  Envoi de notifications (SMS, email, push) aux clients et aux administrateurs pour les mises à jour des livraisons (ex : confirmation, retards, modifications).
- **Suivi des livraisons en temps réel via GPS**
  Permet aux clients de voir l'emplacement en temps réel de leurs marchandises en cours de livraison.
- **Gestion des retours ou des réclamations**
  Implémenter une fonctionnalité pour gérer les retours de marchandise ou soumettre des réclamations directement depuis l'application.
- **Support client intégré (exemple : chat en ligne)**
  Ajouter une option de chat en temps réel pour les clients afin de poser des questions ou obtenir de l'assistance technique.

Description fonctionnelle

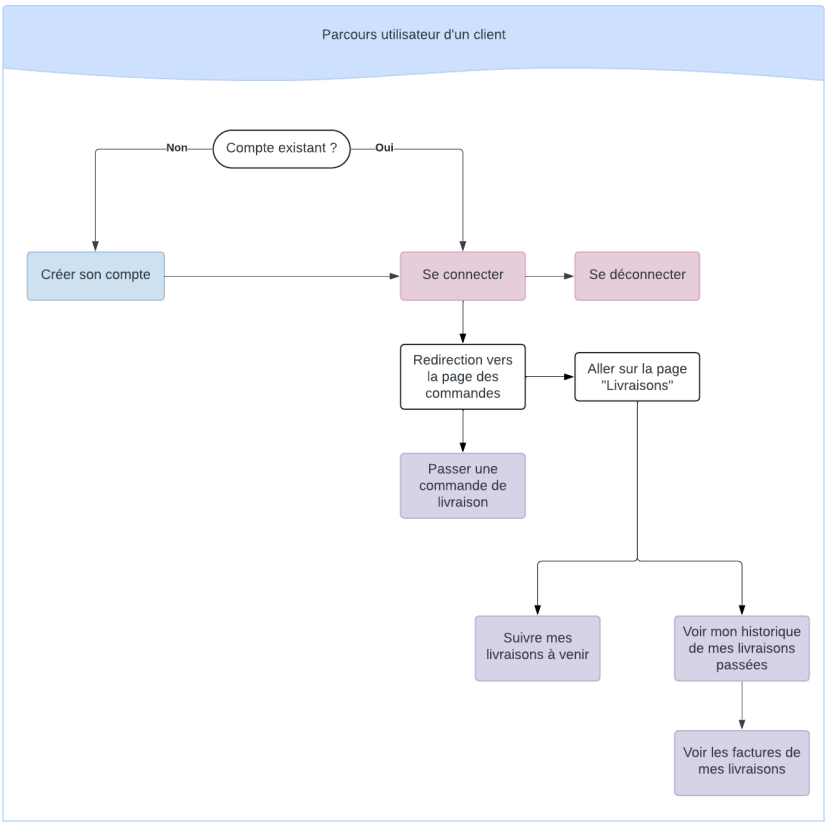
Liste des fonctionnalités :

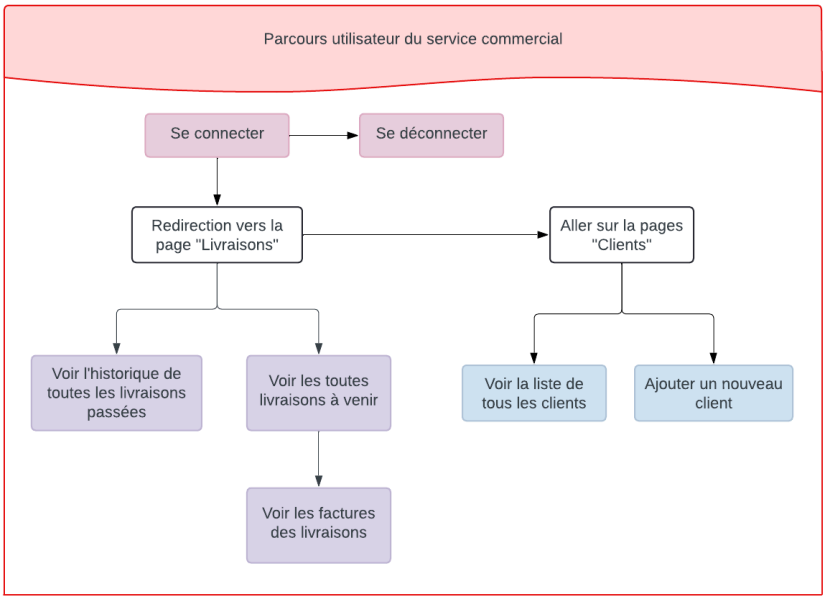
/	Type d'utilisateur	Fonctionnalités	Besoin client
Général	Tous	Se connecter	Accéder à l'application et aux fonctionnalités propres à chaque rôle
Général	Tous	Se déconnecter	Sécuriser l'accès en fermant la session

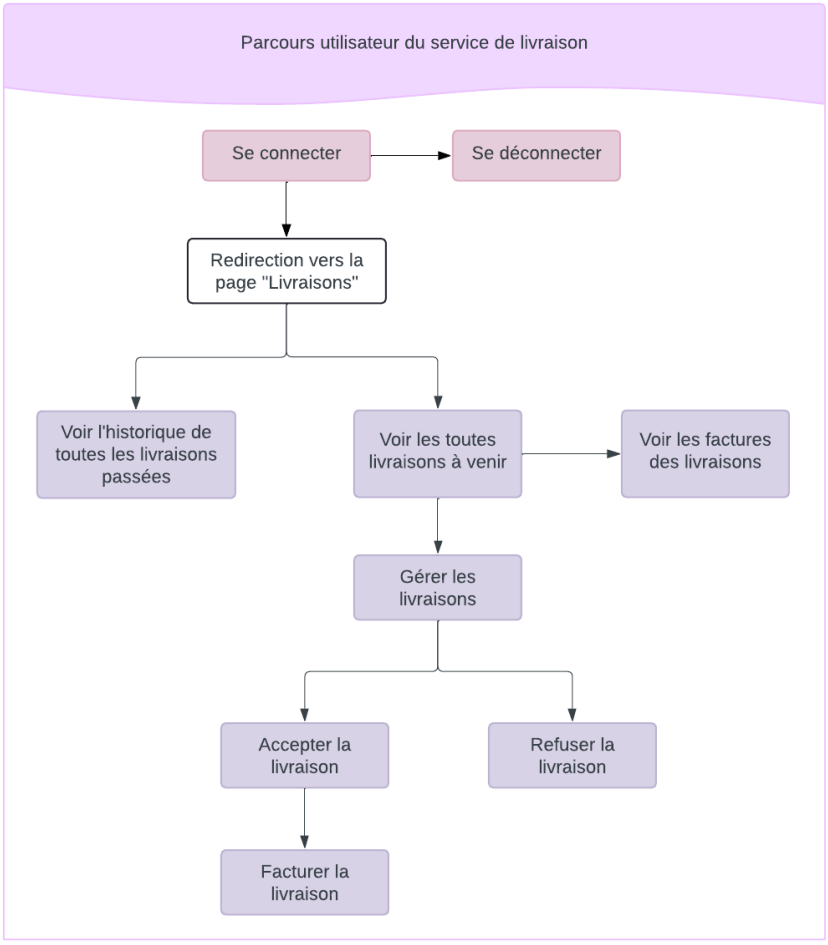
Gestion des comptes clients	Service commercial / Client	Création de compte client	Permettre l'accès au service en autonomie pour les clients ou via le service commercial
Gestion des comptes clients	Service commercial	Voir la liste de tous les clients	Permet au service commercial de gérer et de suivre l'ensemble des comptes clients dans l'application.
Gestion des comptes clients	Client	Modification du compte client	Permettre aux clients de mettre à jour leurs informations personnelles (RGPD)
Gestion des comptes clients	Client	Suppression du compte client	Offrir aux clients la possibilité de supprimer leur compte si besoin (RGPD)
Gestion des livraisons	Service commercial / Service de livraison	Suivre toutes les livraisons à venir	Facilite la gestion et le suivi des livraisons pour le service commercial et de livraison.
Gestion des livraisons	Service de livraison	Accepter ou refuser des livraisons	Donne au service de livraison le contrôle sur les commandes.
Gestion des livraisons	Service de livraison	Facturer les livraisons	Permet au service de livraison de facturer les clients après chaque livraison effectuée.
Gestion des livraisons	Service commercial	Voir tout l'historique des livraisons passées des clients	Offrir un historique pour le suivi client
Gestion des livraisons	Service commercial / Service de livraison	Voir les factures de toutes les livraisons passées	Gérer la facturation et les archives comptables

Gestion des livraisons	Client	Passer une commande de livraison	Offrir un service autonome pour commander des livraisons
Gestion des livraisons	Client	Suivre ses livraisons à venir	Offrir un suivi en temps réel des commandes en cours
Gestion des livraisons	Client	Voir son historique de ses livraisons passées	Fournir un accès à l'historique des livraisons
Gestion des livraisons	Client	Voir les factures de ses livraisons passées	Permettre aux clients de suivre leurs factures de leurs livraisons

Fonctionnement de l’application :


Parcours utilisateur d'un client
Compte existant ?
Non
Oui
Créer son compte
Se connecter
Se déconnecter
Redirection vers la page des commandes
Aller sur la page "Livraisons"
Passer une commande de livraison
Suivre mes livraisons à venir
Voir mon historique de mes livraisons passées
Voir les factures de mes livraisons


Parcours utilisateur du service commercial
Se connecter
Se déconnecter
Redirection vers la page "Livraisons"
Aller sur la pages "Clients"
Voir l'historique de toutes les livraisons passées
Voir les toutes livraisons à venir
Voir la liste de tous les clients
Ajouter un nouveau client
Voir les factures des livraisons



Impact Social :

Impact environnemental :

- **Optimisation des ressources :**

  La refonte vise à améliorer l'efficacité de l'application, réduisant ainsi les besoins en ressources serveur et, par conséquent, l'empreinte carbone associée.

Accessibilité :

- **Utilisabilité pour les personnes handicapées :**

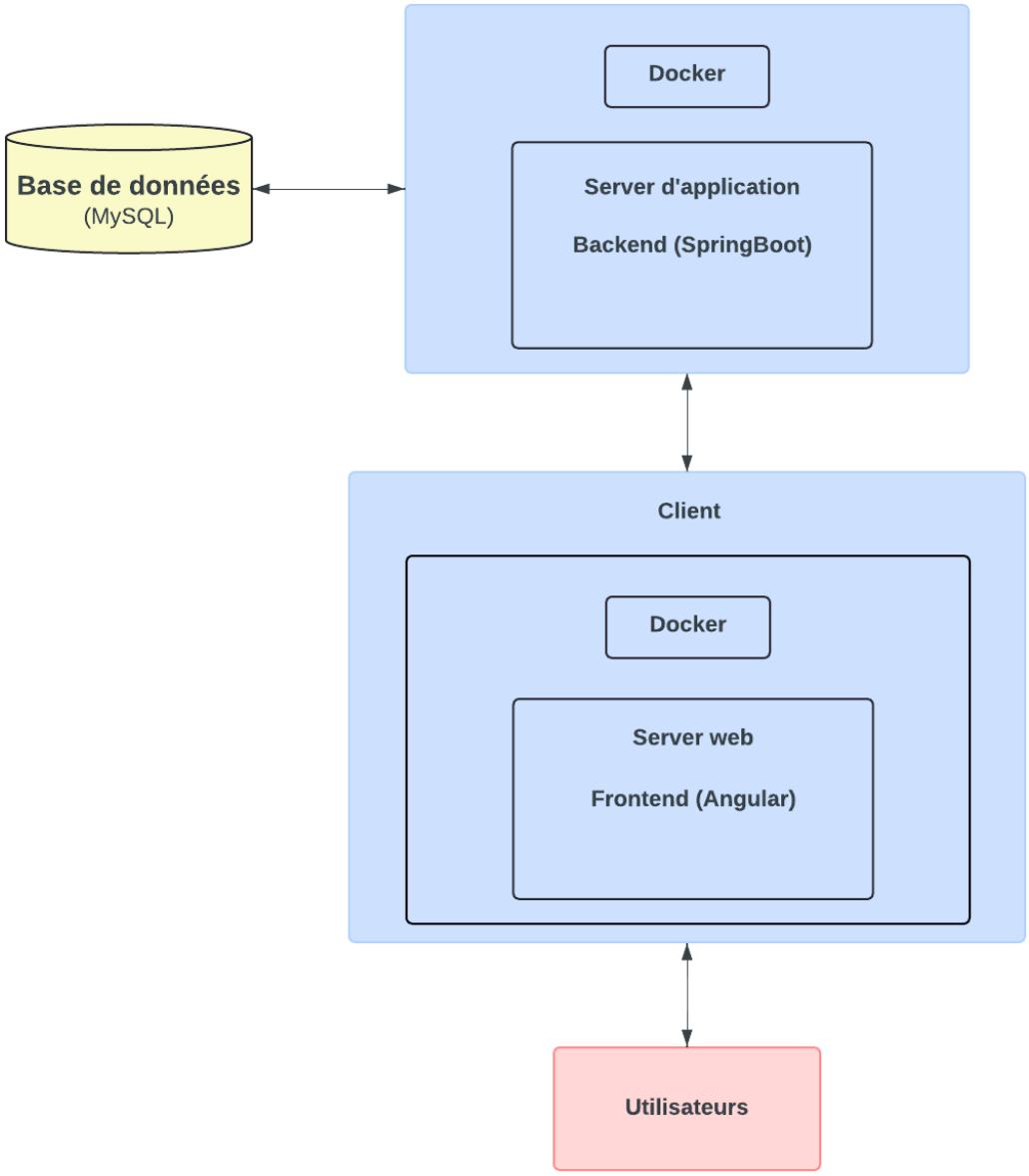
  L'application doit être conçue en tenant compte des normes d'accessibilité web (WCAG) pour garantir que les personnes en situation de handicap puissent utiliser toutes les fonctionnalités de l'application.

Description technique

Composant	**Version**
Architecture	Architecture Client-Server
Languages	
Java	17 (LTS)
Angular	16.1.0
Frameworks	
Spring Boot	3.1.2
Spring Security	6.1.0
Spring Data JPA	3.1.2
Hibernate ORM	6.2.7.Final
Lombok	1.18.30

Maven	3.8.7
MySQL	8.1.0
RxJS	7.8.0
Plugins Maven	
maven-clean-plugin	3.1.0
maven-resources-plugin	3.0.2
maven-compiler-plugin	3.11.0
maven-surefire-plugin	3.0.0-M7
maven-failsafe-plugin	3.0.0-M7
jacoco-maven-plugin (coverage)	0.8.10
Sécurité	
OAuth2 / JWT	6.1.0

Tests	
JUnit5	5.10.0
Spring Boot Test	3.1.2
Mockito	4.11.0
Cypress	13.3.0
Jest	29.6.4
Déploiement	
Docker	24.0.2


Docker
Base de données
(MySQL)
Server d'application
Backend (SpringBoot)
Client
Docker
Server web
Frontend (Angular)
Utilisateurs

LiVrai

LiVrai est une entreprise spécialisée dans la livraison de marchandises en grande quantité pour les professionnels. Leur application ne répond plus aux besoins en expansion de l'entreprise et de sa clientèle.

Un audit approfondi a été réalisé pour évaluer les forces et les faiblesses de l'application existante, ainsi que pour identifier les domaines nécessitant des améliorations significatives.

Et un nouveau cahier des charges à été également réalisé afin de définir les objectifs, les fonctionnalités et les spécifications techniques pour une refonte complète de l'application, afin de répondre aux exigences actuelles et futures de LiVrai.

Made with Gamma

Partie Audit

Made with Gamma

Contexte

LiVrai est confrontée à des défis croissants dus à l'expansion de son offre. L'application actuelle de gestion des clients et des livraisons ne répond plus aux besoins en évolution de l'entreprise. Cette situation nécessite une évaluation approfondie et une modernisation de l'application pour assurer la continuité et l'amélioration des suivis des services de livraison.

Expansion de l'offre

LiVrai élargit ses services, nécessitant une application plus robuste et flexible.

Besoins croissants

Besoin de fonctionnalités plus avancées et une meilleure expérience utilisateur.

Nécessité de modernisation

L'application actuelle atteint ses limites et requiert une refonte pour s'adapter aux nouvelles exigences du marché.

Made with Gamma

Fonctionnalités

L'application actuelle offre des fonctionnalités distinctes pour les clients et les administrateurs. Les fonctionnalités générales incluent la connexion et la déconnexion pour tous les utilisateurs. Les clients peuvent commander de nouvelles livraisons et consulter leurs livraisons à venir et passées. Les administrateurs ont des capacités supplémentaires telles que l'ajout de nouveaux clients, la gestion des livraisons (acceptation, rejet, facturation) et la visualisation des livraisons à venir et passées pour tous les clients.

Fonctionnalités Clients

- Commander une nouvelle livraison
- Afficher les livraisons à venir
- Afficher les livraisons passées

Fonctionnalités Administrateurs

- Ajouter un nouveau client
- Gérer les livraisons
- Afficher les livraisons à venir de tous les clients
- Afficher les livraisons passées de tous les clients

Fonctionnalités Générales

- Se connecter
- Se déconnecter

Made with Gamma

Expérience Utilisateur

L'expérience utilisateur de l'application actuelle présente plusieurs problèmes. L'interface manque d'attrait visuel et la présentation des erreurs est intrusive. La page d'accueil est peu informative, et la navigation dans les tableaux de données est difficile. Des problèmes de formatage affectent la lisibilité des tableaux de livraisons, en particulier pour les administrateurs.

1

Interface peu attrayante

L'interface simpliste décourage les utilisateurs et manque d'attrait visuel.

2

Gestion des erreurs

Les erreurs sont présentées de manière intrusive, interrompant la navigation.

3

Navigation difficile

Les tableaux longs nécessitent un défilement excessif et manquent de fonctionnalités de recherche.

4

Problèmes de formatage

Les tableaux de livraisons présentent des décalages et des confusions dans l'affichage des données.

Made with Gamma

Problèmes d'Affichage des Livraisons

Les tableaux des livraisons passées présentent des problèmes de formatage spécifiques pour les administrateurs et les utilisateurs. Pour les administrateurs, certaines colonnes sont décalées, avec les noms apparaissant dans la colonne "volume" et d'autres informations mal positionnées. Pour les utilisateurs, le poids et le prix sont affichés dans la même colonne, déplaçant le statut dans la colonne des prix, ce qui rend le tableau difficile à lire et confus.

Mes livraisons passées

Id	Volume	Poids	Prix	Statut
1	BERNADOU	12	12 123.0	Terminée
3	BERNADOU	78	67 19.0	Terminée
6	BERNADOU	57	99 1.0	Terminée

Vue Administrateur

Colonnes décalées, noms dans la colonne "volume"

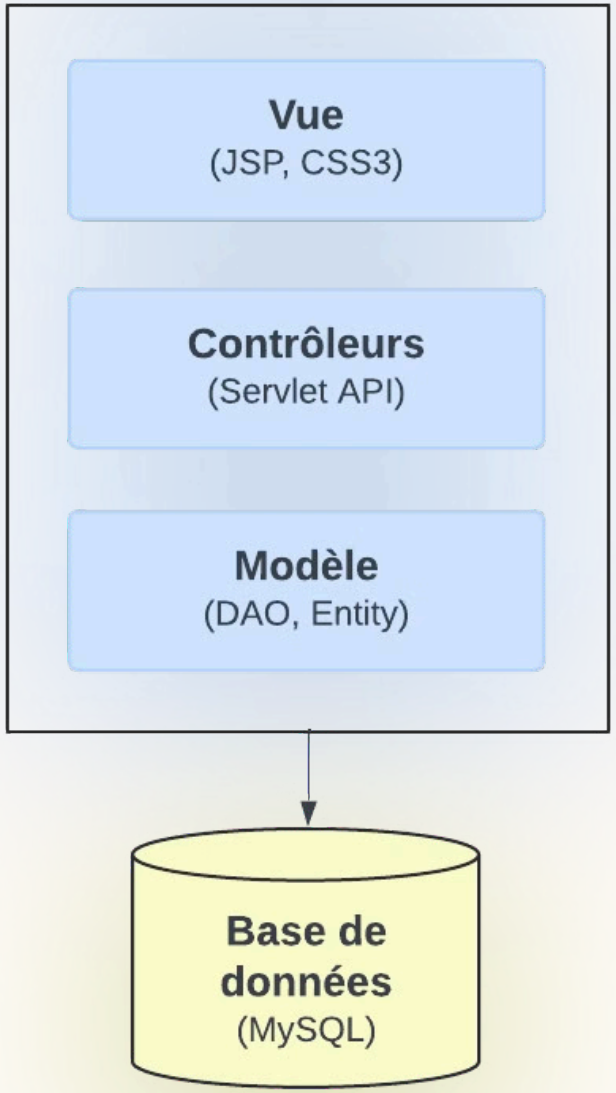
Mes livraisons passées

Id	Volume	Poids	Prix	Statut
1	12	12 123.0	Terminée	
3	78	67 19.0	Terminée	
6	57	99 1.0	Terminée	

Vue Utilisateur

Poids et prix dans la même colonne, statut déplacé

Made with Gamma



Description Technique

L'application utilise une architecture en couches (MVC) principalement avec Java 6, JSP 3.0 et CSS3. Elle s'appuie sur le framework Servlet API 3.0.1 et MySQL 8.0.32 pour la gestion de la base de données. Plusieurs plugins Maven sont utilisés pour la gestion du projet, notamment pour le nettoyage, la compilation, les tests et le déploiement.

Composant	Version
Java	6
JSP	3.0
CSS	3
Servlet API	3.0.1
MySQL	8.0.32

Made with Gamma

Points Forts et Déficiences

L'application présente des points forts tels qu'une bonne séparation des préoccupations, une sécurité des requêtes SQL, une gestion partielle de l'authentification et des performances optimales. Cependant, elle souffre de déficiences importantes, notamment un manque de réactivité, l'exposition de données sensibles, une gestion incorrecte des ressources de base de données, l'absence de tests unitaires et une architecture monolithique limitant la scalabilité.



Architecture en couches

Bonne séparation des préoccupations facilitant la maintenance.



Sécurité SQL

Utilisation de requêtes préparées pour prévenir les injections SQL.



Non responsive

L'application n'est pas adaptée aux appareils mobiles.



Absence de tests

Aucune suite de tests unitaires pour vérifier les fonctionnalités.

Made with Gamma

Conclusion

L'audit de l'application LiVrai révèle des points forts en termes d'architecture et de performances, mais souligne également des déficiences critiques en matière d'expérience utilisateur, de sécurité et d'évolutivité. La modernisation de l'application est indispensable pour répondre aux besoins croissants de l'entreprise. Cela implique une refonte de l'interface utilisateur, une amélioration de la gestion des erreurs, un renforcement de la sécurité des données et une transition vers une architecture plus flexible. Ces améliorations permettront à LiVrai de maintenir un service de qualité et de s'adapter efficacement à sa croissance future.

Made with Gamma

Partie Cahier des Charges

Made with Gamma

Contexte du projet

1 Dépendance au service commercial

Chaque client doit passer par le service commercial pour la création d'un compte, ralentissant le processus et limitant l'autonomie des clients.

2 Absence de fonctionnalités clés

La création de compte et la gestion de la facturation ne sont pas intégrées dans l'application actuelle.

3 Gestion limitée des utilisateurs

L'application ne différencie pas assez clairement les différents types d'utilisateurs, compliquant la gestion des permissions et des fonctionnalités spécifiques à chaque rôle.

Made with Gamma

Objectifs du projet

Fonctionnalités incontournables

- Création de compte
- Gestion des informations clients
- Commande et suivi des livraisons
- Gestion de la facturation
- Historique des livraisons

Types d'utilisateurs

- Les clients
- Le service commercial
- Le service livraisons

Exigences techniques

- Frontend: Angular (SPA)
- Backend: Java (version LTS)
- Déploiement: Docker

Made with Gamma

Contraintes réglementaires

1

Protection des données personnelles

Garantir le stockage sécurisé et le traitement conforme au RGPD des données utilisateurs.

2

Droit d'accès et de rectification

Permettre aux utilisateurs de consulter, corriger, ou supprimer leurs données personnelles via une interface dédiée.

3

Gestion des consentements

Intégrer des fonctionnalités pour obtenir, gérer et enregistrer le consentement des utilisateurs.

4

Notification de violation de données

Mettre en place des procédures pour détecter, signaler, et gérer toute violation de données dans les délais requis par le RGPD.

Made with Gamma

Éléments hors périmètre

Intégration d'un système de paiement

Permet aux clients de régler directement leurs factures en ligne via l'application.

Notifications en temps réel

Envoi de notifications (SMS, email, push) aux clients et aux administrateurs pour les mises à jour des livraisons.

Suivi des livraisons en temps réel via GPS

Permet aux clients de voir l'emplacement en temps réel de leurs marchandises en cours de livraison.

Gestion des retours ou des réclamations

Implémenter une fonctionnalité pour gérer les retours de marchandise ou soumettre des réclamations directement depuis l'application.

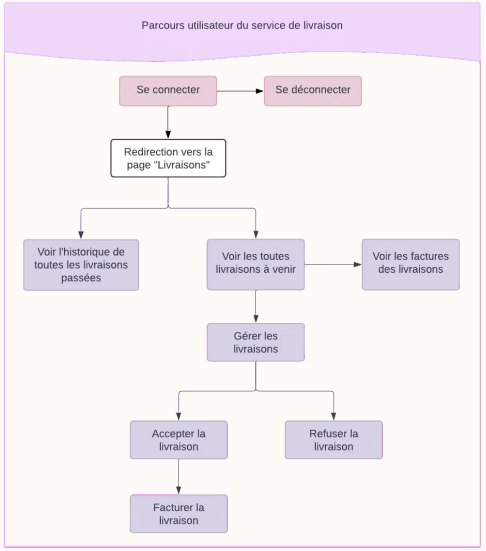
Made with Gamma

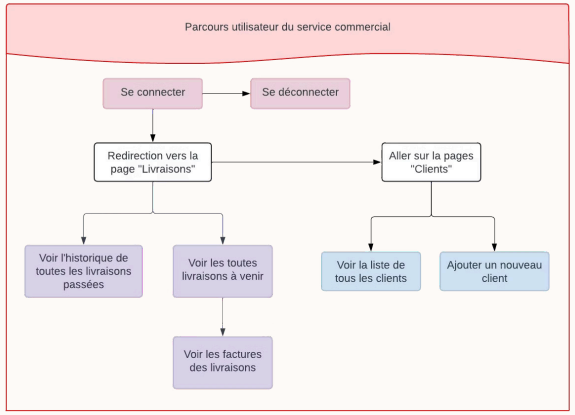
Description fonctionnelle

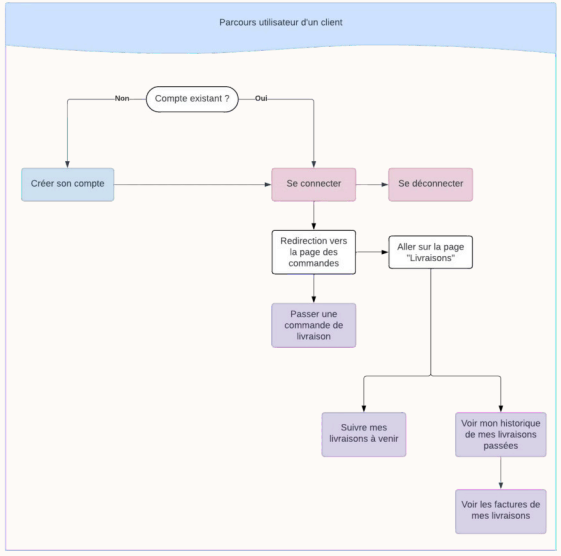
/	Type d’utilisateur	Fonctionnalités	Besoin client
Général	Tous	Se connecter	Accéder à l'application et aux fonctionnalités propres à chaque rôle
Général	Tous	Se déconnecter	Sécuriser l'accès en fermant la session
Gestion des comptes clients	Service commercial / Client	Création de compte client	Permettre l'accès au service en autonomie pour les clients ou via le service commercial
Gestion des comptes clients	Service commercial	Voir la liste de tous les clients	Permet au service commercial de gérer et de suivre l'ensemble des comptes clients dans l'application.
Gestion des comptes clients	Client	Modification du compte client	Permettre aux clients de mettre à jour leurs informations personnelles (RGPD)
Gestion des comptes clients	Client	Suppression du compte client	Offrir aux clients la possibilité de supprimer leur compte si besoin (RGPD)
Gestion des livraisons	Service commercial / Service de livraison	Suivre toutes les livraisons à venir	Facilite la gestion et le suivi des livraisons pour le service commercial et de livraison.
Gestion des livraisons	Service de livraison	Accepter ou refuser des livraisons	Donne au service de livraison le contrôle sur les commandes.
Gestion des livraisons	Service de livraison	Facturer les livraisons	Permet au service de livraison de facturer les clients après chaque livraison effectuée.
Gestion des livraisons	Service commercial	Voir tout l’historique des livraisons passées des clients	Offrir un historique pour le suivi client
Gestion des livraisons	Service commercial / Service de livraison	Voir les factures de toutes les livraisons passées	Gérer la facturation et les archives comptables
Gestion des livraisons	Client	Passer une commande de livraison	Offrir un service autonome pour commander des livraisons
Gestion des livraisons	Client	Suivre ses livraisons à venir	Offrir un suivi en temps réel des commandes en cours
Gestion des livraisons	Client	Voir son historique de ses livraisons passées	Fournir un accès à l’historique des livraisons
Gestion des livraisons	Client	Voir les factures de ses livraisons passées	Permettre aux clients de suivre leurs factures de leurs livraisons

Made with Gamma

Fonctionnement de l'application







Made with Gamma

Impact Social

Impact environnemental

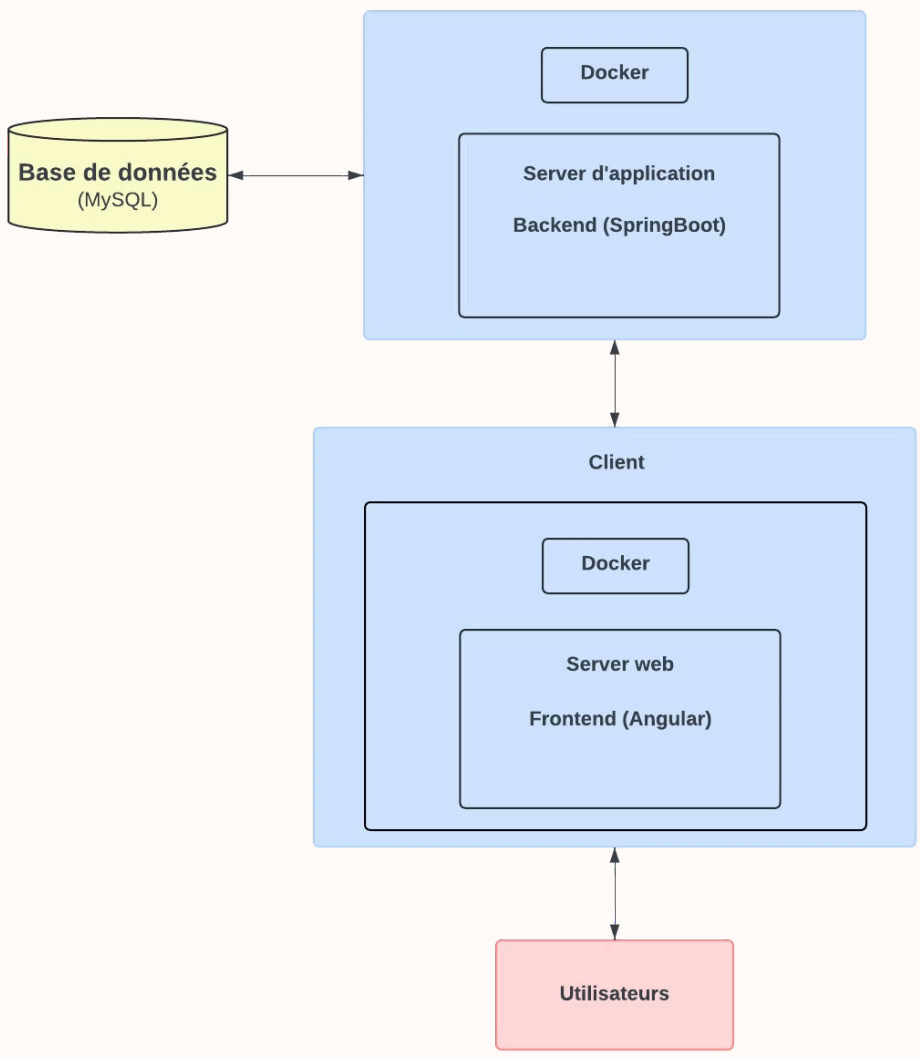
La refonte vise à améliorer l'efficacité de l'application, réduisant ainsi les besoins en ressources serveur et, par conséquent, l'empreinte carbone associée. Cette optimisation des ressources contribue à une utilisation plus durable des technologies de l'information.

Accessibilité

L'application doit être conçue en tenant compte des normes d'accessibilité web (WCAG) pour garantir que les personnes en situation de handicap puissent utiliser toutes les fonctionnalités de l'application. Cela inclut des considérations pour la navigation au clavier, le contraste des couleurs, et la compatibilité avec les lecteurs d'écran.

Made with Gamma

Description technique



Composant	Version
Architecture	Architecture Client-Server
Languages	
Java	17 (LTS)
Angular	16.1.0
Frameworks	
Spring Boot	3.1.2
Spring Security	6.1.0
Spring Data JPA	3.1.2
Hibernate ORM	6.2.7.Final
Lombok	1.18.30
Maven	3.8.7
MySQL	8.1.0
RxJS	7.8.0
Plugins Maven	
maven-clean-plugin	3.1.0
maven-resources-plugin	3.0.2
maven-compiler-plugin	3.11.0
maven-surefire-plugin	3.0.0-M7
maven-failsafe-plugin	3.0.0-M7
jacoco-maven-plugin (coverage)	0.8.10
Sécurité	
OAuth2 / JWT	6.1.0
Tests	
JUnit5	5.10.0
Spring Boot Test	3.1.2
Mockito	4.11.0
Cypress	13.3.0
Jest	29.6.4
Déploiement	
Docker	24.0.2

Made with Gamma

Volbi-Tech



Alisa's Closet

Application DeliveryFit

Proposition commerciale

---

1. **Contexte**
    a. Contexte global
    b. Fonctionnalités métier
2. **Réalisation du projet**
    a. Définition des tâches techniques
    b. Gestion des points de complexité
    c. Coûts
    d. Risques identifiés
    e. Définition des objectifs de performance
3. **Synthèse**

---

1. Contexte

1.1 Contexte global :

Alisa’s Closet est un détaillant de vêtements en ligne cherchant à optimiser ses délais de livraison en développant **DeliveryFit**. Cette application web a pour objectif de réduire le temps de livraison en expédiant les produits depuis le point de vente le plus proche du client, plutôt que depuis un centre de livraison centralisé. Cela vise à améliorer l'expérience client, réduire l'empreinte carbone, et optimiser les coûts logistiques.

Contraintes :

- **Budget :** 10.000 €
- **Deadline :** 5 mois
- **Méthodologie :** Agile (Scrum)

1.2 Fonctionnalités métier :

- **Algorithme de Comparaison des Distances :** Trouver le point de vente le plus proche.
- **Interface Utilisateur** : Validation du panier et saisie des informations d’expédition.
- **Notifications Automatiques :** Informer automatiquement le magasin expéditeur une fois la livraison confirmée.

---

1. Réalisation du projet

2.1 Définition des tâches techniques :

Relations avec les bases de données :

1. Créer la base de données relationnelle
2. Configurer des entités et les relations avec Spring Data JPA
3. Intégrer et connecter la base de données MySQL dans Spring Boot

Backend :

1. **Créer l'application Backend en utilisant Spring Boot :**
    a. Initialisation du projet Spring Boot avec Spring Initializr
    b. Configuration du projet avec Maven et ajout des dépendances
    c. Mise en place de la structure du projet Spring Boot
    d. Configuration des fichiers de propriétés de l'application
2. **Comparer les distances entre les magasins et le domicile :**
    a. Création de l'API REST et des endpoints pour récupérer et transmettre les données
    b. Développer et implémenter un algorithme de comparaison des distances pour l'optimisation des livraisons
    c. Création des services, de leur implémentation et des repositories pour la gestion des livraisons
3. **Trouver le point de livraison le plus proche :**
    a. Récupérer le meilleur résultat de l'algorithme de comparaison des distances et le retourner

4. **Envoyer une notification au magasin expéditeur :**
    a. Implémenter une solution pour notifier automatiquement le magasin une fois qu'une livraison est confirmée

Frontend :

1. **Créer l’application Frontend en utilisant Angular :**
    a. Initialisation du projet Angular
    b. Installation et configuration des dépendances (avec RxJS, Angular Material)
    c. Configuration des environnements de build et de déploiement
    d. Création de la structure des composants (modèles, services, routes)
    e. Configuration des tests unitaires et de la couverture de code (à l’aide d’outils tel que Jest ou Cypress)
2. **Transmettre les informations de livraison :**
    a. Création des modèles de livraison pour la gestion des données
    b. Création des services pour transmettre les données au backend
3. **Intercepter l’URL du site :**
    a. Implémentation d'un intercepteur pour capturer et rediriger les URLs.
4. **Créer la redirection vers le site de vente :**
    a. Rediriger l'utilisateur vers le site de vente en utilisant la logique implémentée dans l'intercepteur.
5. **Création des formulaires pour la validation du panier et les informations d'expédition :**

a. Développer les formulaires d'expédition avec validation des données

Authentification :

1. **Mise en place de l'authentification côté backend via un système de token JWT et OAuth2**
2. **Mise en place de l'authentification côté frontend avec JWT et des guards Angular**

Gestion de projet : L'architecte logiciel va jouer le rôle de SCRUM master pour la gestion du projet.

1. **Supervision des tâches et de l’équipe**
2. **Démonstration au client**

2.2 Gestion des points de complexité :

Les tâches sont estimées selon la Suite de Fibonacci et l'équation PERT.

Estimer la complexité des tâches avec la Suite de Fibonacci :

- **1 :** Pour les tâches mineures ou corrections de bugs rapides.
- **2 :** Pour les tâches de petite envergure ou fonctionnalités basiques.
- **3 :** Pour les tâches avec un peu plus de logique ou d'intégration.
- **5 :** Pour les fonctionnalités plus larges nécessitant une réflexion sur la conception.
- **8 :** Pour les fonctionnalités majeures, des intégrations multiples, ou des tâches nécessitant des analyses supplémentaires.

Calculer le temps pour chaque tâche avec l'équation PERT :

Complexité de la tâche	Temps optimiste (en jour)	Temps plus probable (en jour)	Temps pessimiste (en jour)	Équation PERT	Résultat (en jour)	Arrondir le résultat (en jour)
1	0,25	0,50	1	(0,25 + (4 * 0,5) + 1) / 6	0,54	**0,5**
2	0,5	1,00	2	(0,5 + (4 * 1) + 2) / 6	1,08	**1**
3	1	1,50	3	(1 + (4 * 1,5) + 3) / 6	1,67	**1,75**
5	1,5	2,50	5	(1,5 + (4 * 2,5) + 5) / 6	2,75	**2,75**
8	2,5	4	8	(2,5 + (4 * 4) + 8) / 6	4,42	**4,5**

Calculer la marge de sécurité :

Une marge de sécurité est ajoutée à chaque tâche pour gérer les imprévus et s'assurer de respecter les délais.

Complexité de la tâche	Durée estimée	Marge de sécurité	Durée finale (arrondi)
1	0,5	10 % (0,05 jour)	**0 jour**
2	1	20 % (0,20 jour)	**0,25 jour**
3	1,75	25 % (0,44 jours)	**0,5 jour**
5	2,75	30 % (0,83 jour)	**0,75 jour**
8	4,5	30 % (1,35 jour)	**1,5 jour**

Estimation du temps pour chaque tâche :

Tâche technique	Complexité	Estimation temps (en jour)	Marge (en jour)
Sans marge			
Créer l'application Backend en utilisant Spring Boot	**1**	0,5	0

Créer l'application Frontend en utilisant Angular	**1**	0,5	0
Créer la redirection de DeliveryFit vers le site de vente en ligne	**1**	0,5	0
Marge de 20 %			
Intercepter l'URL du site de vente en ligne et rediriger vers l'URL de DeliveryFit	**2**	1	0,25
Transmettre les informations de livraison	**2**	1	0,25
Trouver le point de vente le plus proche	**2**	1	0,25
IHM - Créer la page de validation du panier avec les informations d'expédition	**2**	1	0,25
Marge de 25 %			
Envoyer une notification au magasin expéditeur	**3**	1,75	0,5
Marge de 30 %			

Comparer les distances entre les magasins et/ou le centre de livraison et le domicile	**5**	2,75	0,75

2.3 Coûts :

Estimation des coûts basée sur le temps de travail prévu pour chaque tâche et le taux journalier moyen (TJM) des membres de l'équipe.

Tâche	**Temps (en jour)**	**Membre de l'équipe**	**TJM**	**Total**
Sans marge				
Gestion de projet	0,5	Architecte Logiciel	800 €	400 €
Créer l'application Backend en utilisant Spring Boot	0,5	Architecte Logiciel	800 €	400 €
Créer l'application Frontend en utilisant Angular	0,5	Architecte Logiciel	800 €	400 €
Correction Backend	0,5	Développeur Backend	700 €	350 €

Correction Frontend	0,5	Développeur Frontend	700 €	350 €
Créer la redirection de DeliveryFit vers le site de vente en ligne	0,5	Développeur Frontend	700 €	350 €
Marge de 20%				
Intercepter l'URL du site de vente en ligne et rediriger vers l'URL de DeliveryFit	1,25	Développeur Frontend	700 €	875 €
Transmettre les informations de livraison	1,25	Développeur Backend	700 €	875 €
Trouver le point de vente le plus proche	1,25	Développeur Backend	700 €	875 €
IHM - Créer la page de validation du panier avec les informations d'expédition	1,25	Développeur Frontend	700 €	875 €
Marge de 25 %				

Envoyer une notification au magasin expéditeur	2,25	Développeur Backend	700 €	1 575 €
Marge de 30 %				
Comparer les distances entre les magasins et/ou le centre de livraison et le domicile	3,5	Développeur Backend	700 €	2 450 €
Total	**13,75**	**/**	**/**	**9 775 €**

Total estimé : **9 775 €**

Marge disponible : **225 €**

2.4 Risques identifiés :

La gravité d'un risque est mesurée selon l'échelle suivante :

- **1 :** Négligeable
- **2 :** Mineure
- **3 :** Modérée
- **4 :** Majeure
- **5 :** Catastrophique

La probabilité qu'un risque se produise est évaluée comme suit :

- **1 :** Très probable
- **2 :** Probable
- **3 :** Possible
- **4 :** Peu probable
- **5 :** Très peu probable

L'impact d'un risque est calculé avec la formule :
Impact = Gravité × Probabilité

L'impact est classé selon les catégories suivantes :

- **Faible :** entre 1 et 6
- **Moyen :** entre 7 et 12
- **Élevé :** entre 13 et 25

Risque	Gravité	Probabilité	Impact	Stratégie de mitigation	Tâche liée

Dépassement du budget	4	3	**12**	Suivi hebdomadaire des coûts et ajustement des tâches si nécessaire.	Général
Retard dans le respect des délais	3	4	**12**	Revue des priorités et adaptation des sprints.	Général
Problème de performances	3	3	**9**	Optimisation régulière des algorithmes et tests de performance réguliers..	Général

Bugs et erreurs	4	2	**8**	Utilisation de TDD et d'outils de détection automatisés	• **Backend :** ○ Comparer les distances entre les magasins et le domicile ○ Envoyer une notification au magasin expéditeur ○ ... • **Frontend :** ○ Transmettre les informations de livraison ○ Intercepter l'URL du site ○ ...
Failles de la sécurité	3	3	**9**	Audits réguliers et mise à jour des dépendances.	• Authentification backend & frontend

---

2.5 Définition des objectifs de performance

Objectif de performance	**Description**	**Mesure**

Temps de chargement des pages	Les pages doivent se charger en moins de 2s sur une connexion standard.	Google PageSpeed ou Lighthouse.
Temps de réponse des API	L'API doit renvoyer les résultats en moins de 500ms pour assurer une expérience fluide.	JMeter ou Postman.
Disponibilité du système	L'application doit être disponible 99,9% du temps.	Outils de monitoring.
Taux de réussite des tests	Atteindre un taux de réussite d'au moins 95% lors des tests unitaires et d'intégration.	JUnit, Jest ou Cypress..
Couverture des Tests Unitaires	Au moins 80% du code doit être couvert par des tests unitaires.	Jacoco ou SonarQube.

2.6 Définition des sprints

Organisation des Sprints :

Sprint	Durée (en jours)	Tâches principales
Sprint 1	1	Initialisation du projet et des environnements, gestion de projet

Sprint 2	8,25	Développement des fonctionnalités backend, algorithmes
Sprint 3	3	Développement frontend, intégration des API, formulaires.
Sprint 4	0,5	Démonstration client
Sprint 5	0,5	Corrections suite à la démonstration (backend & frontend simultanément

2. Synthèse

Proposition 1 : Respect strict du budget de 10 000 €

La durée totale du projet est estimée à **13,75 jours** avec un coût de **9 775 €**, respectant ainsi le budget initial avec une petite marge de **225 €**. Cette version garantit la réalisation dans les délais et les coûts.

Répartition du Travail des Membres de l'Équipe :

- **Architecte Logiciel : 1,5 jours**
- **Développeur Backend : 8,75 jours**
- **Développeur Frontend : 3,5 jours**

Comparaison avec le budget Initial :

- **Budget initial alloué :** 10 000 €
- **Coût total de la proposition améliorée :** 9 775 €
- **Reste du budget :** 225 €

Cette proposition convient pour livrer une application fonctionnelle et stable, tout en respectant les contraintes budgétaires et de temps.

Proposition 2 : Version améliorée avec optimisation technique (+15 % du budget)

Cette version propose des améliorations techniques supplémentaires, tout en limitant l'augmentation du budget à **15 %**, soit un total de **11.500 €**. Les optimisations apportent de meilleures performances et une sécurité renforcée sans dépasser excessivement le budget initial.

Améliorations supplémentaires proposées :

- **Optimisation de la performance :**
  Optimisation des temps de réponse API avec des algorithmes plus rapides et l'utilisation de la mise en cache.

- **Sécurité renforcée :**
  En plus des protocoles JWT et OAuth2, un audit de sécurité sera réalisé pour minimiser les failles. Des mesures supplémentaires de protection des données seront également mises en place.

- **Tests de performance avancés :**
  Utilisation d'outils tels que JMeter ou Postman pour garantir un temps de réponse inférieur à 300 ms sous diverses conditions.

Durée totale de réalisation :

La durée estimée pour cette version améliorée est de 14,5 jours de travail.

Répartition du Travail des Membres de l'Équipe :

- **Architecte Logiciel : 2 jours**
  - Similaire à la proposition 1
  - + Supervision des optimisations techniques
- **Développeur Backend : 10 jours**
  - Similaire à la proposition 1
  - + Optimisation des algorithmes
  - + Sécurisation des API avec des pratiques avancées et audit de sécurité.
- **Développeur Frontend : 3,5 jours**
  - Similaire à la proposition 1

    - + Amélioration de l’expérience utilisateur

Comparaison avec le budget Initial :

- **Budget initial alloué :** 10 000 €
- **Coût total de la proposition améliorée :** 11 500 €
- **Dépassement du budget :** 1 500 € (+15 %)

Bien que cette version dépasse le budget initial de **1.500 €**, elle apporte des améliorations importantes en termes de sécurité et de performance, tout en optimisant l’expérience utilisateur. Cet investissement supplémentaire reste justifié par l'amélioration de la robustesse et de la rapidité de l'application.

BobApp : Workflow CI/CD

Sommaire

Étapes mises en œuvre dans les GitHub Actions
1. Déclencheurs du workflow
2. Jobs
    backend-testing-and-sonarcloud-analysis
    frontend-testing-and-sonarcloud-analysis
    docker-build-and-push

KPI Proposés
1. Couverture des tests (Code Coverage)
2. Duplication du code

Analyse des métriques et des avis utilisateurs
1. Métriques
    Backend
    Frontend
2. Analyse des avis utilisateurs et des métriques obtenues
    Avis utilisateurs pertinents et priorités
    Priorité des métriques à améliorer

Étapes mises en œuvre dans les GitHub Actions

1. Déclencheurs du workflow

    Backend

    - **Événements push :** Le workflow est déclenché automatiquement à chaque push de modifications sur la branche main, à condition que ces changements affectent les fichiers situés dans les dossiers back/** ou .github/workflows/**.
    - **Événements pull_request :** Le workflow se déclenche aussi pour les événements de pull_request vers la branche main, incluant l'ouverture, la synchronisation et la réouverture des pull requests. Cela garantit que tout nouveau code proposé pour intégration dans main est rigoureusement testé avant fusion.

Frontend

- **Événements push :** Le workflow est déclenché automatiquement à chaque push de modifications sur la branche main, à condition que ces changements affectent les fichiers situés dans les dossiers front/** ou .github/workflows/**.
- **Événements pull_request :** Le workflow se déclenche aussi pour les événements de pull_request vers la branche main, incluant l'ouverture, la synchronisation et la réouverture des pull requests. Cela garantit que tout nouveau code proposé pour intégration dans main est rigoureusement testé avant fusion.

2. Jobs

Backend

1. **backend-run-tests-for-protection-rule**
    - **Objectif** : Vérifier que les tests passent pour pouvoir merge les branches
    - **Étapes** :
        - **Checkout Repository** : Récupère le code source du dépôt. La profondeur de clonage est définie à 0 pour obtenir l'historique complet, nécessaire pour certains outils de vérification.
        - **Set up Java** : Configure l'environnement Java avec la version 11, utilisant la distribution Temurin.
        - **Install Dependencies and Run Tests** : Utilise les commandes Maven pour installer les dépendances nécessaires et exécuter les tests.
2. **backend-testing-and-sonarcloud-analysis**
    - **Objectif** : Valider les tests sur une pull request et analyser avec SonarCloud.
    - **Étapes** :

- **Checkout Repository** : Récupère le code source du dépôt. La profondeur de clonage est définie à 0 pour obtenir l'historique complet, nécessaire pour certains outils de vérification.
- **Set up Java** : Configure l'environnement Java avec la version 11, utilisant la distribution Temurin.
- **Install Dependencies and Run Coverage** : Utilise les commandes Maven pour installer les dépendances nécessaires et exécuter les tests, incluant la mesure de la couverture du code. Un rapport de couverture est généré automatiquement à l'aide de JaCoCo.
- **Upload Code Coverage Report** : Télécharge le rapport de couverture généré (jacoco.xml) en tant que artifact pour une consultation ultérieure.
- **SonarCloud Scan** : Effectue une analyse de qualité du code à l'aide de SonarCloud.

3. **docker-build-and-push**

- **Objectif** : Construire et déployer l'image Docker.
- **Dépendance** : Nécessite la réussite du job backend-testing-and-sonarcloud-analysis.
- **Condition** : Le job s'exécute uniquement pour les pull requests vers la branche main.
- **Étapes** :
  - **Checkout Repository** : Récupère une fois de plus le code source du dépôt.
  - **Log in to Docker Hub** : Se connecte à Docker Hub en utilisant les informations d'identification stockées en tant que secrets. Cela permet d'authentifier le déploiement de l'image.
  - **Build and Push Docker Image** : Construit l'image Docker à partir du répertoire ./back et lui attribue un tag. Ensuite, elle pousse l'image vers Docker Hub, ce qui rend

l'application prête à être déployée sur des serveurs ou des environnements de production.

Frontend

1. **frontend-run-tests-for-protection-rule**
    - **Objectif** : Vérifier que les tests passent pour pouvoir merge les branches
    - **Étapes** :
        - **Checkout Repository** : Récupère le code source du dépôt. La profondeur de clonage est définie à 0 pour obtenir l'historique complet, nécessaire pour certains outils de vérification.
        - **Set up Node.js** : Configure l'environnement Node.js avec la version 16.
        - **Install Dependencies and Run Test** : Utilise les commandes NPM pour installer les dépendances et lancer les tests.
2. **frontend-testing-and-sonarcloud-analysis**
    - **Objectif** : Valider les tests sur une pull request et analyser avec SonarCloud.
    - **Étapes** :
        - **Checkout Repository** : Récupère le code source du dépôt. La profondeur de clonage est définie à 0 pour obtenir l'historique complet, nécessaire pour certains outils de vérification.
        - **Set up Node.js** : Configure l'environnement Node.js avec la version 16.
        - **Install Dependencies and Run Coverage** : Utilise les commandes NPM pour installer les dépendances et lancer les tests, avec une analyse de la couverture du code. Un rapport de couverture est produit via Karma.

        - **Upload Code Coverage Report** : Télécharge le rapport de couverture généré (lcov.info) en tant que artifact pour une consultation ultérieure.
        - **SonarCloud Scan** : Effectue une analyse de qualité du code à l'aide de SonarCloud.

3. **docker-build-and-push**
    - **Objectif** : Construire et déployer l'image Docker.
    - **Dépendance** : Nécessite la réussite du job frontend-testing-and-sonarcloud-analysis.
    - **Condition** : Le job s'exécute uniquement pour les pull requests vers la branche main.
    - **Étapes** :
        - **Checkout Repository** : Récupère une fois de plus le code source du dépôt.
        - **Log in to Docker Hub** : Se connecte à Docker Hub en utilisant les informations d'identification stockées en tant que secrets. Cela permet d'authentifier le déploiement de l'image.
        - **Build and Push Docker Image** : Construit l'image Docker à partir du répertoire ./front et lui attribue un tag. Ensuite, elle pousse l'image vers Docker Hub, ce qui rend l'application prête à être déployée sur des serveurs ou des environnements de production.

KPI Proposés

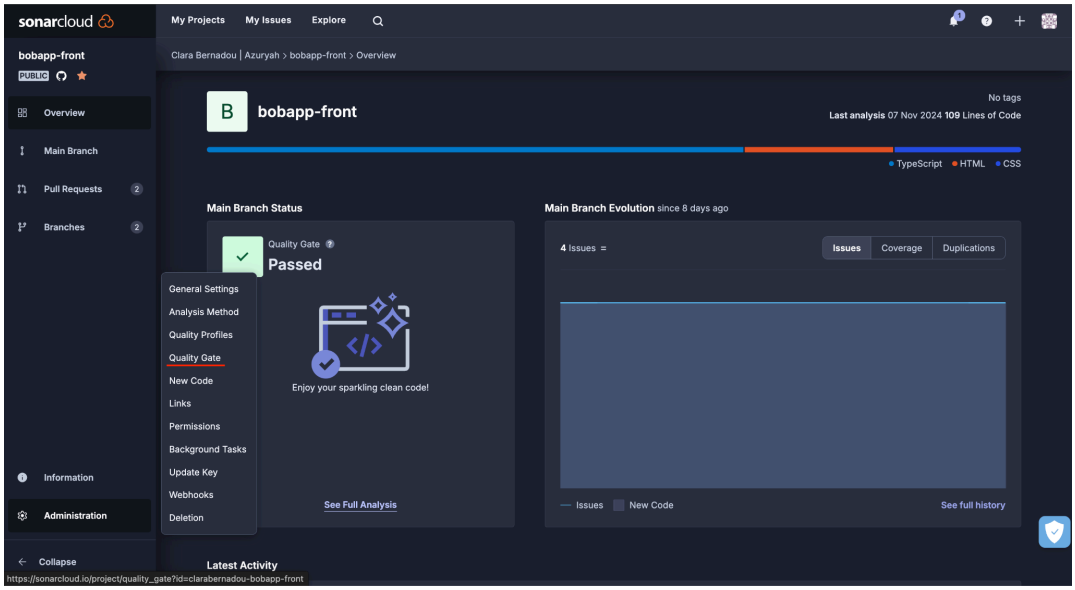
1. Couverture des tests (Code Coverage)
    - **Seuil proposé** : **80 %**
    - **Explication** : Un taux de couverture de 80% permet de garantir que l'essentiel du code est testé, ce qui réduit les risques de bugs en

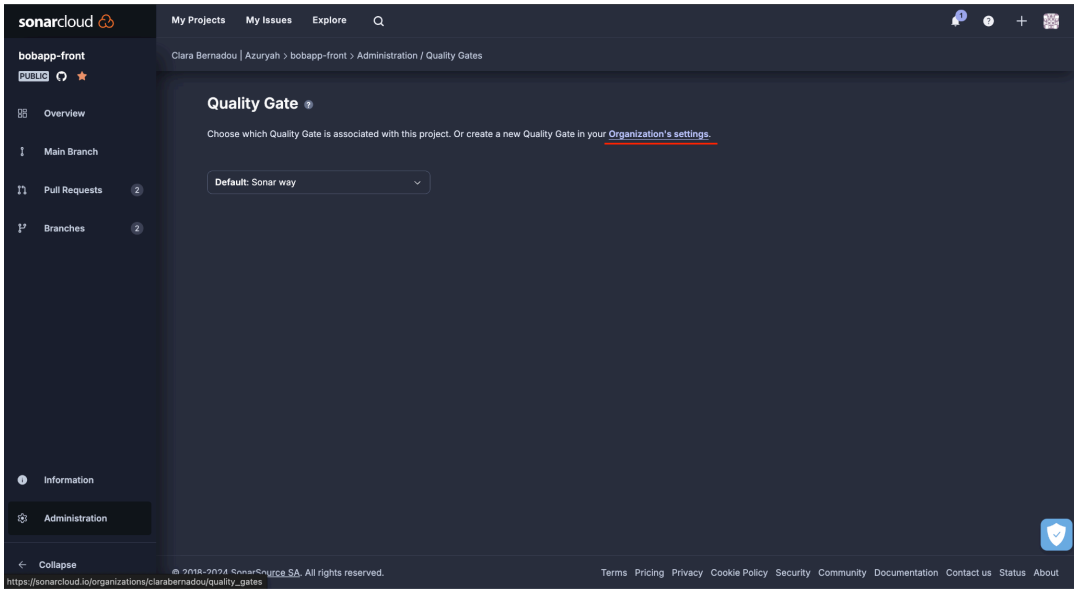
production. Ce seuil est un compromis reconnu entre une couverture élevée et une efficacité de développement raisonnable.

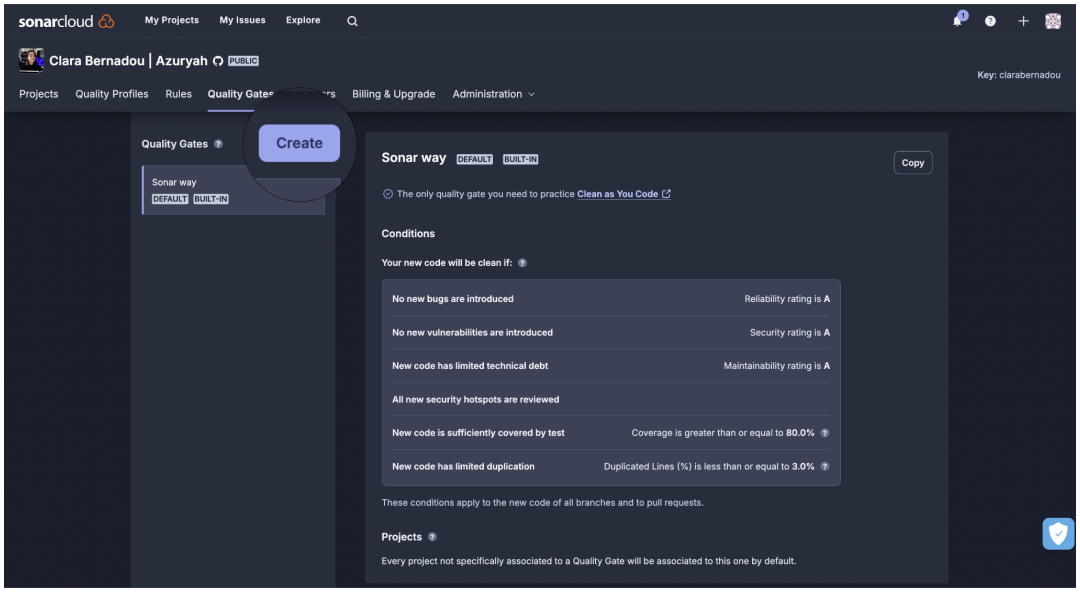
2. Duplication du code

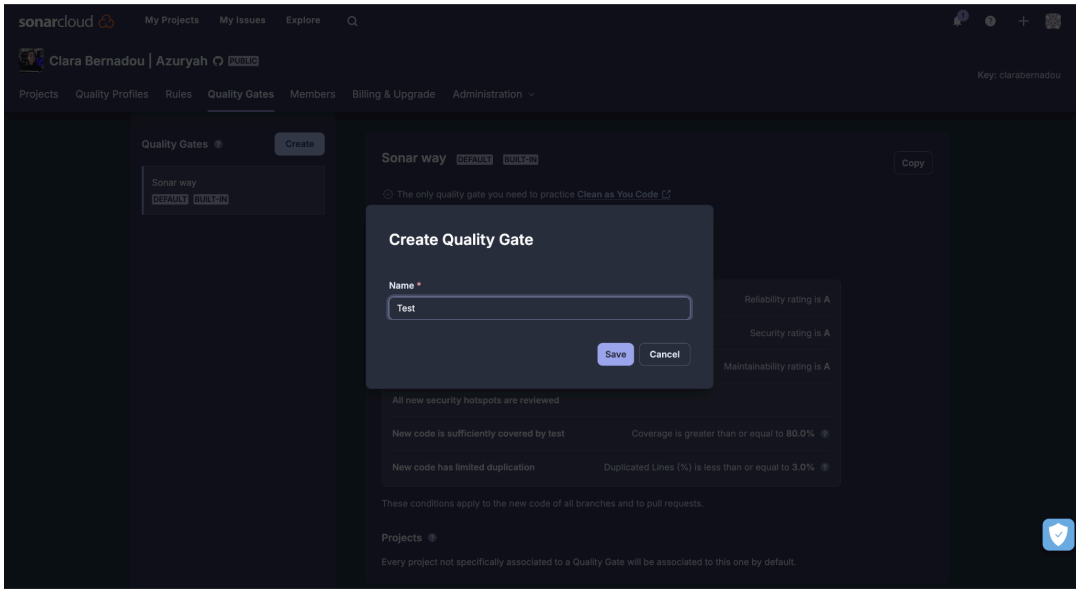
    - **Seuil proposé** : **3%**
    - **Explication** : Un taux de duplication de code de 3% ou moins aide à maintenir un code propre et facile à maintenir. Un code dupliqué peut entraîner une augmentation des efforts de maintenance et des risques de bugs.

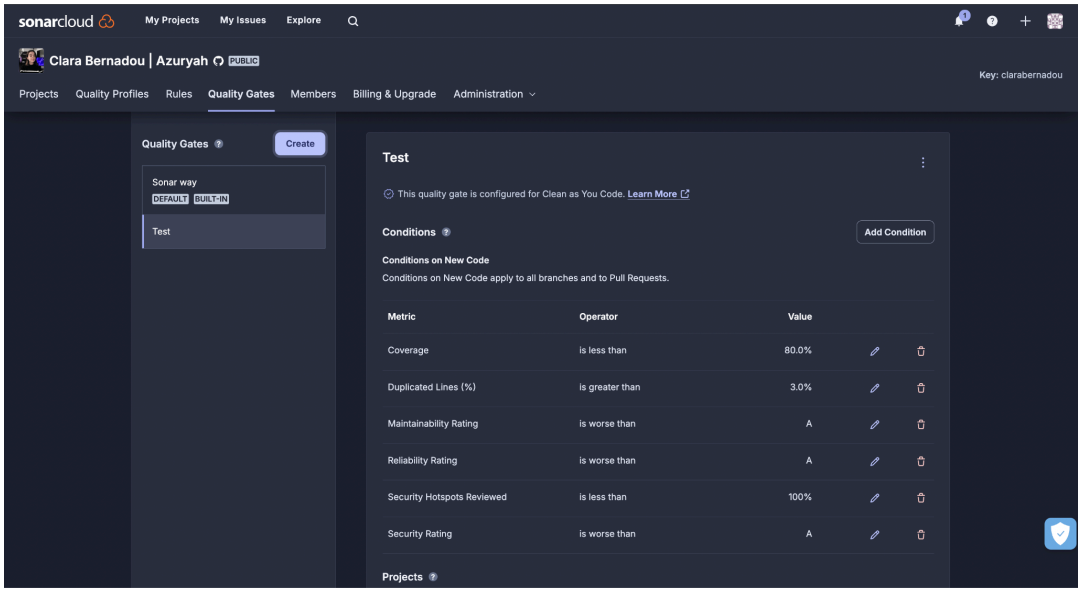
Il est possible de modifier les KPI mentionnés ci-dessus directement dans Sonar. Voici quelques images pour illustrer cela.

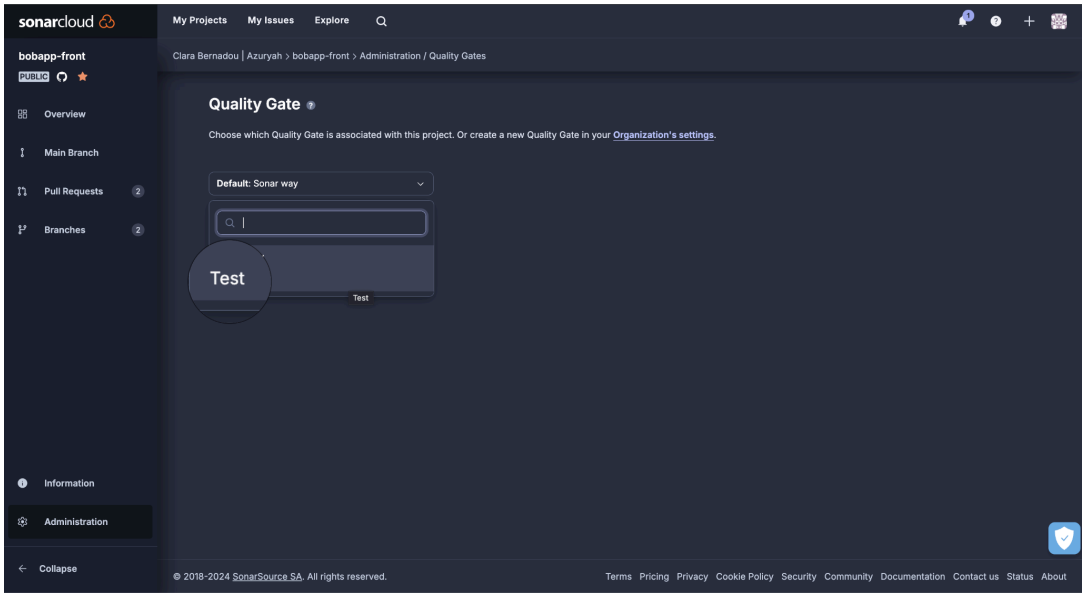

sonarcloud
My Projects
My Issues
Explore
bobapp-front
PUBLIC
Clara Bernadou | Azuryah > bobapp-front > Overview
Overview
Main Branch
Pull Requests
Branches
Information
Administration
Collapse
B
bobapp-front
No tags
Last analysis 07 Nov 2024 109 Lines of Code
TypeScript
HTML
CSS
Main Branch Status
Quality Gate
Passed
General Settings
Analysis Method
Quality Profiles
Quality Gate
New Code
Links
Permissions
Background Tasks
Update Key
Webhooks
Deletion
Enjoy your sparkling clean code!
See Full Analysis
Main Branch Evolution since 8 days ago
4 Issues
Issues
Coverage
Duplications
Issues
New Code
See full history
Latest Activity
https://sonarcloud.io/project/quality_gate?id=clarabernadou-bobapp-front


sonarcloud
My Projects
My Issues
Explore
bobapp-front
PUBLIC
Clara Bernadou | Azuryah > bobapp-front > Administration / Quality Gates
Overview
Main Branch
Pull Requests
Branches
Information
Administration
Collapse
Quality Gate
Choose which Quality Gate is associated with this project. Or create a new Quality Gate in your Organization's settings.
Default: Sonar way
© 2018-2024 SonarSource SA. All rights reserved.
Terms
Pricing
Privacy
Cookie Policy
Security
Community
Documentation
Contact us
Status
About
https://sonarcloud.io/organizations/clarabernadou/quality_gates


sonarcloud
My Projects
My Issues
Explore
Clara Bernadou | Azuryah
PUBLIC
Key: clarabernadou
Projects
Quality Profiles
Rules
Quality Gates
Billing & Upgrade
Administration
Quality Gates
Create
Sonar way
DEFAULT
BUILT-IN
Sonar way
DEFAULT
BUILT-IN
Copy
The only quality gate you need to practice Clean as You Code
Conditions
Your new code will be clean if:
No new bugs are introduced
Reliability rating is A
No new vulnerabilities are introduced
Security rating is A
New code has limited technical debt
Maintainability rating is A
All new security hotspots are reviewed
New code is sufficiently covered by test
Coverage is greater than or equal to 80.0%
New code has limited duplication
Duplicated Lines (%) is less than or equal to 3.0%
These conditions apply to the new code of all branches and to pull requests.
Projects
Every project not specifically associated to a Quality Gate will be associated to this one by default.


sonarcloud
My Projects
My Issues
Explore
Clara Bernadou | Azuryah
PUBLIC
Key: clarabernadou
Projects
Quality Profiles
Rules
Quality Gates
Members
Billing & Upgrade
Administration
Quality Gates
Create
Sonar way
DEFAULT
BUILT-IN
Create Quality Gate
Name *
Test
Save
Cancel
Sonar way
DEFAULT
BUILT-IN
Copy
The only quality gate you need to practice Clean as You Code
Reliability rating is A
Security rating is A
Maintainability rating is A
All new security hotspots are reviewed
New code is sufficiently covered by test
Coverage is greater than or equal to 80.0%
New code has limited duplication
Duplicated Lines (%) is less than or equal to 3.0%
These conditions apply to the new code of all branches and to pull requests.
Projects
Every project not specifically associated to a Quality Gate will be associated to this one by default.



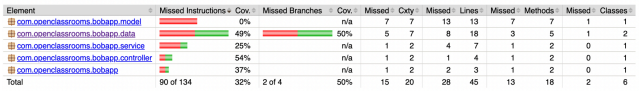


Analyse des métriques et des avis utilisateurs

1. Métriques

Backend

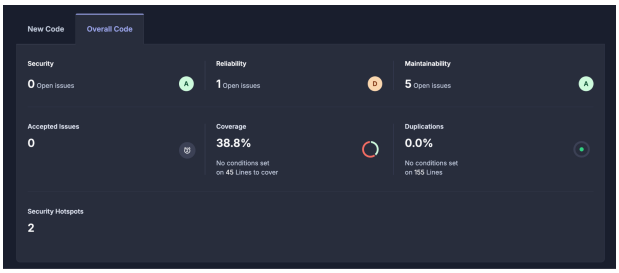
a. Rapport de couverture



Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.openclassrooms.bobapp.model		0%		n/a	7	7	13	13	7	7	1	1
com.openclassrooms.bobapp.data		49%		50%	5	7	8	18	3	5	1	2
com.openclassrooms.bobapp.service		25%		n/a	1	2	4	7	1	2	0	1
com.openclassrooms.bobapp.controller		54%		n/a	1	2	1	4	1	2	0	1
com.openclassrooms.bobapp		37%		n/a	1	2	2	3	1	2	0	1
Total	90 of 134	32%	2 of 4	50%	15	20	28	45	13	18	2	6

/	Model	Data	Service	Controller	Global	Total
Pourcentage	0%	49%	25%	54%	37%	32%
Commentaire	La couverture est inexistante, ce qui signifie que ce package n'est pas testé. Il est important d'ajouter des tests pour améliorer la fiabilité du modèle.	La couverture est faible, presque la moitié du code n'est pas testée. Des tests supplémentaires sont nécessaires pour garantir la stabilité des opérations sur les données.	Ce niveau de couverture est très bas pour les services, où des tests sont essentiels pour s'assurer que la logique métier fonctionne correctement.	La couverture est modérée, mais il serait bénéfique d'augmenter ce score pour garantir que les contrôleurs répondent comme prévu.	Cette couverture est faible. Des tests globaux sur ce package principal aideraient à assurer une meilleure fiabilité.	La couverture de code totale est faible. Il est essentiel d'augmenter la couverture des tests, en particulier pour les packages model, service, et data, afin de garantir que le code est bien testé et que les fonctionnalités principales sont fiables.

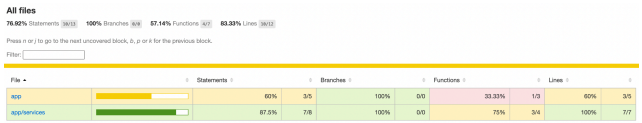
b. SonarCloud



/	Sécurité	Fiabilité	Maintenabilité	Problèmes acceptés	Couverture de code	Duplications de code
Note	A	D	A	0	38.8%	0.0%
Commentaire	Le code ne présente aucune faille de sécurité détectée, assurant une bonne protection des données.	La fiabilité nécessite des améliorations, avec potentiellement des bugs ou du code sujet aux erreurs.	Le code est bien structuré et facile à maintenir, facilitant les évolutions futures.	Aucun hotspot de sécurité n'a été examiné, ce qui pourrait laisser des vulnérabilités non identifiées.	Le taux de couverture est relativement bas. Cela laisse une grande partie du code non testée, augmentant les risques d'erreurs en production.	Absence de duplication dans le code, ce qui améliore la maintenabilité et réduit les risques de bugs.

Frontend

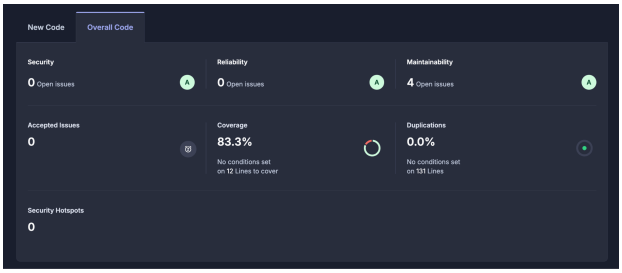
a. Coverage



/	Global	Controller	Total
Pourcentage	60%	87.5%	76.92%
Commentaire	La couverture pour les branches est complète, mais les fonctions et les lignes ont encore des parties	Ce répertoire a une excellente couverture, ce qui indique que les services sont bien testés. Peu	La couverture globale est assez bonne, avec un total de 76.92% pour les statements et une couverture

	non testées. Il est recommandé d'ajouter des tests pour couvrir davantage de fonctions et lignes dans ce répertoire.	d'améliorations sont nécessaires ici, sauf pour atteindre une couverture de 100% des statements.	complète pour les branches. Cependant, les fonctions restent partiellement couvertes (57.14%) et il reste des lignes non testées (83.33%). Il est recommandé de concentrer les efforts sur le répertoire app pour améliorer la couverture des statements et des fonctions.

b. SonarCloud

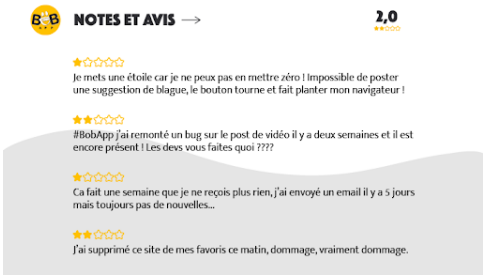


/	Sécurité	Fiabilité	Maintenabilité	Problèmes acceptés	Couverture de code	Duplications de code
Note	A	A	A	0	83.3%	0.0%

Commentaire	Aucun problème de sécurité est détecté, garantissant la sûreté de l'application côté client.	Le code est fiable avec peu de risques de bugs, assurant une bonne robustesse pour les utilisateurs.	Le code est bien structuré et facilement modifiable, facilitant les évolutions futures.	Tous les hotspots de sécurité ont été examinés, garantissant une protection renforcée contre les vulnérabilités potentielles.	Une bonne couverture de tests qui assure la qualité et réduit les risques de régression. Toutefois, viser 100% pourrait encore renforcer la robustesse.	Aucun code dupliqué, ce qui réduit la complexité et les risques d'incohérence.

---

2. Analyse des avis utilisateurs et des métriques obtenues



Avis utilisateurs pertinents et priorités

Problèmes de performance lors de l'ajout de suggestions :

- **Explication** : Un utilisateur signale qu'il est impossible de poster une suggestion de blague, une fonctionnalité pourtant attendue. Après vérification, il apparaît que ni l'application web ni le code ne contiennent cette fonctionnalité. Il peut s'agir d'une régression, d'un retrait volontaire, ou d'une fonctionnalité prévue mais jamais implémentée.
- **Priorité** : Élevée

- **Solution** : Vérifier l'historique des versions afin de déterminer si la fonctionnalité a déjà été développée puis retirée, ou si elle n’a jamais été implémentée. Si cette fonctionnalité est attendue et jugée nécessaire, procéder à son implémentation ou réimplémentation. Une fois développée, il est essentiel de mettre en place des tests unitaires et d’intégration pour garantir son bon fonctionnement et éviter toute régression future.

Bug persistant concernant le lecteur vidéo :

- **Explication** : Un utilisateur indique avoir signalé un bug sur le post vidéo il y a deux semaines, mais il est toujours présent. Après vérification, il apparaît que ni l'application web ni le code ne contiennent cette fonctionnalité. Il peut s'agir également d'une régression, d'un retrait volontaire, ou d'une fonctionnalité prévue mais jamais implémentée.
- **Priorité** : Élevée
- **Solution** : Vérifier l'historique des versions afin de déterminer si la fonctionnalité a déjà été développée puis retirée, ou si elle n’a jamais été implémentée. Si cette fonctionnalité est attendue et jugée nécessaire, procéder à son implémentation ou réimplémentation. Une fois développée, il est essentiel de mettre en place des tests unitaires et d’intégration pour garantir son bon fonctionnement et éviter toute régression future.

Support client lent ou inactif :

- **Explication** : Un utilisateur mentionne un problème de non-réception d'e-mails et déplore l'absence de réponse malgré plusieurs relances (une semaine sans nouvelles).
- **Priorité** : Moyenne
- **Solution** : Améliorer les processus internes du support client, par exemple en mettant en place un système de réponse

automatique pour informer les utilisateurs de la prise en charge de leur demande. Augmenter la fréquence des vérifications de tickets non résolus pour garantir une réactivité accrue. Former l'équipe pour mieux gérer les délais de réponse, surtout pour les cas techniques urgents.

Perte de confiance et désinstallation de l'application :

- **Explication** : Un utilisateur a décidé de supprimer BobApp de ses favoris, exprimant une grande déception face à l'application.
- **Priorité** : Moyenne
- **Solution** : Envoyer un sondage ou un e-mail de suivi aux utilisateurs pour comprendre leurs frustrations. Mettre en place un plan d'amélioration basé sur les retours d'expérience, incluant des mises à jour pour corriger les principales plaintes. Communiquer les améliorations apportées pour reconquérir la confiance des utilisateurs et favoriser la réinstallation de l'application.

Priorité des métriques à améliorer

Couverture de code insuffisante en backend (38,8%) :

- **Explication** : Le faible niveau de couverture de code augmente les risques de bugs non détectés en production. Il est essentiel d'augmenter cette couverture pour garantir une meilleure stabilité et fiabilité du système.
- **Priorité** : Élevée
- **Solution** : Augmenter le nombre de tests unitaires et d'intégration, en ciblant prioritairement les parties critiques du code. Utiliser le rapport de couverture JaCoCo pour identifier les zones non testées et développer des tests supplémentaires.

Fiabilité (Note D) :

- **Explication** : La note de fiabilité du backend est alarmante, suggérant des problèmes de performance et de stabilité qui doivent être traités en urgence.
- **Priorité** : Élevée
- **Solution** : Réaliser une analyse approfondie des zones à risque identifiées, en particulier celles qui causent des instabilités. Mettre en place une revue de code plus stricte, corriger les points de vulnérabilité et améliorer la gestion des erreurs. Tester le système avec des charges de travail élevées pour évaluer et ajuster la performance.

Points d'accès sécurisés (2) :

- **Explication** : Il est crucial d'augmenter le nombre de points d'accès sécurisés pour protéger l'application contre les menaces potentielles.
- **Priorité** : Moyenne
- **Solution** :
    - **Valider les entrées** : Assurez-vous que toutes les requêtes envoyées au backend, même pour des actions simples comme changer de blague, sont validées. Cela inclut la vérification des formats de données et la gestion des erreurs d'entrée pour éviter les injections de code ou les attaques par script.
    - **Configurer les en-têtes de sécurité** : Utiliser des en-têtes HTTP comme Content-Security-Policy, X-Content-Type-Options, et X-Frame-Options pour limiter les risques d'attaques comme le clickjacking ou les injections de scripts.
    - **Limiter les requêtes** : Ajouter une restriction sur la fréquence de changement de blague (par exemple, un

limiteur de requêtes) pour éviter les abus, comme un spam de requêtes au serveur.

Lien de la Backlog sur Notion :
https://paint-dime-6da.notion.site/Formez-et-encadrez-une-quipe-de-d-veloppement-Full-Stack-Product-Backlog-12f045bb5f92803fb934fde752e21cef

Document des normes RGPD et des bonnes pratiques ITIL

Document des normes RGPD et des bonnes pratiques ITIL 1
1. Définitions RGPD et ITIL 2
   a. RGPD (Règlement Général sur la Protection des Données) 2
   b. ITIL (Information Technology Infrastructure Library) 2
2. Conformité aux exigences RGPD 4
   a. Cartographie des données 4
   b. Consentement des utilisateurs 4
   c. Droits des utilisateurs 4
   d. Sécurité des données 4
   e. Analyse d'impact relative à la protection des données (AIPD) 5
3. Intégration des pratiques ITIL 6
   a. Gestion des services 6
   b. Gestion des changements 6
   c. Gestion des incidents 6
   d. Gestion des problèmes 6
   e. Gestion de la sécurité de l'information 6
   f. Gestion de la capacité et de la disponibilité 6
4. Pourquoi respecter RGPD et ITIL ? 8
   a. Bénéfices liés à la conformité 8
   b. Risques liés à la non-conformité 9
   c. Conseils pour garantir le respect de ces normes 9

1. Définitions RGPD et ITIL

a. RGPD (Règlement Général sur la Protection des Données)

Le RGPD est une réglementation européenne entrée en vigueur le 25 mai 2018, visant à protéger les données personnelles des individus. Elle impose des règles strictes sur la collecte, le traitement, le stockage et la suppression des données, et accorde des droits aux utilisateurs comme le droit d'accès, de rectification, de suppression et de portabilité de leurs données. L'objectif est de garantir la transparence et la sécurité des données tout en responsabilisant les entreprises.

Exemple pratique : Lorsqu'un utilisateur souhaite supprimer ses données personnelles, il doit pouvoir le faire via une interface dédiée ou en envoyant une demande à l'équipe support.

Pour plus de détails sur le RGPD, référez-vous au texte officiel disponible sur CNIL.

Possibilité de se former au RGPD gratuitement avec l'Atelier RGPD, une attestation de suivi sera délivrée à la fin de chaque module validé.

b. ITIL (Information Technology Infrastructure Library)

ITIL est un cadre de référence regroupant les meilleures pratiques pour la gestion des services informatiques. Il aide les organisations à aligner leurs services IT avec les besoins de l'entreprise tout en garantissant efficacité, fiabilité et qualité. Les processus ITIL incluent la gestion des incidents, des problèmes, des changements, des services, et de la sécurité de l'information.

Exemple de pratique : Définir un SLA qui garantit une disponibilité du service à 99,9 % sur une période mensuelle. Un autre SLA pourrait spécifier un délai de réponse initial de 30 minutes maximum pour tout incident critique signalé.

Les meilleures pratiques ITIL sont documentées dans les guides officiels disponibles sur ITIL Official Site.

*SLA (Service-Level Agreement ou Accord de niveau de service) = Contrat passé entre un fournisseur de service et ses clients internes ou externes. Ce contrat documente les services que le fournisseur met à disposition et des paramètres comme leurs disponibilités ou les temps de réponse.

2. Conformité aux exigences RGPD

Pour vérifier la conformité RGPD, l'équipe pourra effectuer des audits trimestriels sur la gestion des consentements et le respect des droits des utilisateurs. Et un tableau de bord de suivi des demandes RGPD pourra être mis en place pour mesurer les délais de réponse et la satisfaction des utilisateurs.

a. Cartographie des données

- **Responsable** : Rachida (Développeur back-end)
- **Action** : Identifier les données personnelles collectées et leur cycle de vie (collecte, traitement, stockage, suppression) avec MySQL. Protéger les données sensibles grâce à des mesures de sécurité adaptées.

b. Consentement des utilisateurs

- **Responsable** : Dimitry (Développeur front-end)
- **Action** : Intégrer des formulaires de consentement dans Angular (pop-ups, cases à cocher) et s'assurer que ces consentements sont sauvegardés de manière sécurisée.

c. Droits des utilisateurs

- **Responsable** : Dimitry (Développeur front-end) et Rachida (Développeur back-end)
- **Action** : Permettre aux utilisateurs de consulter, modifier ou supprimer leurs données via des interfaces (front-end) et des API sécurisées (back-end).

d. Sécurité des données

- **Responsable** : Rachida (Développeur back-end)
- **Action** :
    - Sécuriser le back-end avec Spring Security et JWT
    - Utiliser un chiffrement AES-256 pour les données sensibles
    - Appliquer une politique stricte de suppression des données obsolètes
    - Mettre en place une réponse rapide aux incidents en cas de violation des données :
        - Une analyse immédiate de l'incident.
        - Une notification aux autorités compétentes dans un délai de 72 heures, conformément à l'article 33 du RGPD.

        - Une communication transparente avec les utilisateurs affectés.

e. Analyse d'impact relative à la protection des données (AIPD)

- **Responsable** : Jorge (UX Designer) et Rachida (Développeur back-end)
- **Action** : Évaluer les risques pour la vie privée grâce à une matrice des risques et documenter les mesures pour atténuer ces impacts.

3. Intégration des pratiques ITIL

L'équipe utilisera Notion pour gérer les incidents et les SLA, permettant une traçabilité complète et un suivi des objectifs définis. Des processus seront mis en place pour approuver et documenter les changements, gérer les incidents et problèmes, et garantir la sécurité de l'information.

a. Gestion des services

- **Responsable** : Jorge (UX Designer)
- **Action** : Créer un catalogue des fonctionnalités et des SLA clairs (exemple : disponibilité à 99,9 %, résolution des incidents en 24 h).

b. Gestion des changements

- **Responsable** : Rachida (Développeur back-end)
- **Action** : Établir un processus pour approuver et documenter les changements en analysant leur impact sur la sécurité et la conformité RGPD.

c. Gestion des incidents

- **Responsable** : Dimitry (Développeur front-end) et Rachida (Développeur back-end)
- **Action** : Concevoir des pages d'erreur pour les utilisateurs (front-end) et configurer un suivi des incidents côté serveur pour une résolution rapide.

d. Gestion des problèmes

- **Responsable** : Rachida (Développeur back-end)
- **Action** : Identifier les causes récurrentes des incidents, documenter des solutions de contournement, et créer une base de connaissances pour faciliter les diagnostics.

e. Gestion de la sécurité de l'information

- **Responsable** : Rachida (Développeur back-end)
- **Action** : Renforcer la sécurité avec Spring Security et JWT. Organiser des audits réguliers pour vérifier la conformité RGPD et ITIL, et garantir une réponse rapide aux fuites de données (notification sous 72 heures).

f. Gestion de la capacité et de la disponibilité

- **Responsable** : Jorge (UX Designer) et Rachida (Développeur back-end)
- **Action** : Optimiser les interfaces pour réduire la charge sur le front-end et planifier la gestion des montées en charge du serveur pour assurer une continuité de service.

4. Pourquoi respecter RGPD et ITIL ?

a. Bénéfices liés à la conformité

RGPD :

- **Renforcement de la confiance des utilisateurs** : Le respect du RGPD assure aux utilisateurs que leurs données sont protégées et traitées de manière transparente, renforçant ainsi leur fidélité et confiance envers l'entreprise.
- **Protection contre les sanctions financières** : Le non-respect du RGPD expose à des amendes pouvant atteindre 20 millions d'euros ou 4 % du chiffre d'affaires annuel mondial, selon le montant le plus élevé, ce qui peut avoir des conséquences financières graves pour l'entreprise.
- **Réduction des risques juridiques** : En respectant le RGPD, l'entreprise minimise les risques juridiques, notamment les actions en justice pour traitement illégal des données ou manquements à la protection de la vie privée des utilisateurs.

ITIL :

- **Réduction des temps d'arrêt et amélioration de l'expérience utilisateur** : L'application des bonnes pratiques ITIL permet de réduire les temps d'arrêt des systèmes jusqu'à 30 %, ce qui a un impact direct sur l'expérience utilisateur et la satisfaction des clients.
- **Amélioration de la communication avec les utilisateurs** : En garantissant des processus de gestion des incidents efficaces, ITIL permet une communication rapide avec les utilisateurs, renforçant ainsi leur confiance dans la disponibilité et la fiabilité du service.
- **Optimisation de l'agilité et de l'efficacité opérationnelle** : En intégrant des processus ITIL, l'entreprise améliore sa capacité à répondre rapidement aux incidents et aux demandes des utilisateurs, augmentant ainsi l'efficacité globale et l'agilité de ses opérations.

b. Risques liés à la non-conformité

RGPD :

- Le non-respect du RGPD peut entraîner des sanctions financières importantes, ainsi qu'une perte de réputation et de confiance auprès des utilisateurs, impactant ainsi l'image et la pérennité de l’entreprise.

ITIL :

- En l'absence de bonnes pratiques ITIL, les coûts de résolution des incidents peuvent augmenter, et des pannes prolongées peuvent entraîner une insatisfaction croissante des utilisateurs, dégradant leur expérience.

c. Conseils pour garantir le respect de ces normes

Les KPI suivants peuvent être mis en place pour assurer la conformité avec le RGPD et ITIL :

- **Taux de conformité RGPD** : Assurer que 100 % des consentements sont documentés et vérifiables.
- **Temps de réponse moyen pour les incidents critiques** : Répondre en moins de 30 minutes pour les incidents critiques.
- **Satisfaction utilisateur** : Maintenir une satisfaction utilisateur supérieure à 90 % via un sondage trimestriel.



CATASTERRE

Plan d’action

Sommaire

Plan d'action 1

Sommaire 2

1. Amélioration de la coopération dans l'équipe 3
   a. Session de Pair Programming 3
   b. Mise en place d'une documentation accessible et de standards de tests simplifiés 3
   c. Ateliers pratiques sur les tests et la qualité de code 3
2. Analyse de compétences de l'équipe 4
3. Outil pour l'analyse continue des compétences 6
4. Formations 8
   a. OpenClassrooms – Testez votre code Java pour réaliser des applications de qualité 8
   b. OpenClassrooms - Testez l'interface de votre site (Cours bientôt archivé) 8
   c. Jasmine – Documentation 9
   d. Selenium - WebDriver 10

1. Amélioration de la coopération dans l’équipe

a. Session de Pair Programming

Le pair programming consiste en une collaboration directe entre deux développeurs sur une même tâche. Un développeur écrira le code (*driver*) tandis que l'autre supervisera et guidera le processus (*navigator*). Cette méthode permettra de résoudre rapidement des problèmes techniques complexes, notamment l’écriture de tests unitaires et d'intégration, tout en partageant des connaissances pratiques. Le pair programming améliora la qualité du code grâce à une révision instantanée et favorisera le transfert de compétences en temps réel. Il est particulièrement utile pour renforcer les compétences de l'équipe dans des domaines clés, tels que les tests, tout en optimisant la vitesse de développement.

b. Mise en place d'une documentation accessible et de standards de tests simplifiés

En créant une documentation claire et facilement accessible, l'équipe pourra se référer aux processus et outils de testing quand nécessaire, facilitant ainsi l'apprentissage autonome. Bien que sa mise en place demande un investissement de temps initial, elle est essentielle pour l'autonomisation des membres de l'équipe à long terme. Une fois cette documentation en place, elle servira de référence pour tous, renforçant ainsi la culture du partage des connaissances et améliorant l'intégration des nouveaux arrivants.

c. Ateliers pratiques sur les tests et la qualité de code

Les ateliers pratiques sont des sessions de formation interactives où l’équipe apprendra les principes fondamentaux des tests unitaires, d'intégration et d'automatisation. Ces ateliers combinent théorie et pratique, offrira à chaque membre de l’équipe une compréhension approfondie des outils et bonnes pratiques de testing. L'objectif est de renforcer la compétence collective en matière de tests, de partager des techniques éprouvées et d’améliorer la qualité globale du code. En favorisant l’apprentissage à travers des exercices pratiques, ces ateliers permettent à l’équipe de mieux répartir la charge des tests, garantissant ainsi une couverture de tests efficace et une validation rapide des fonctionnalités.

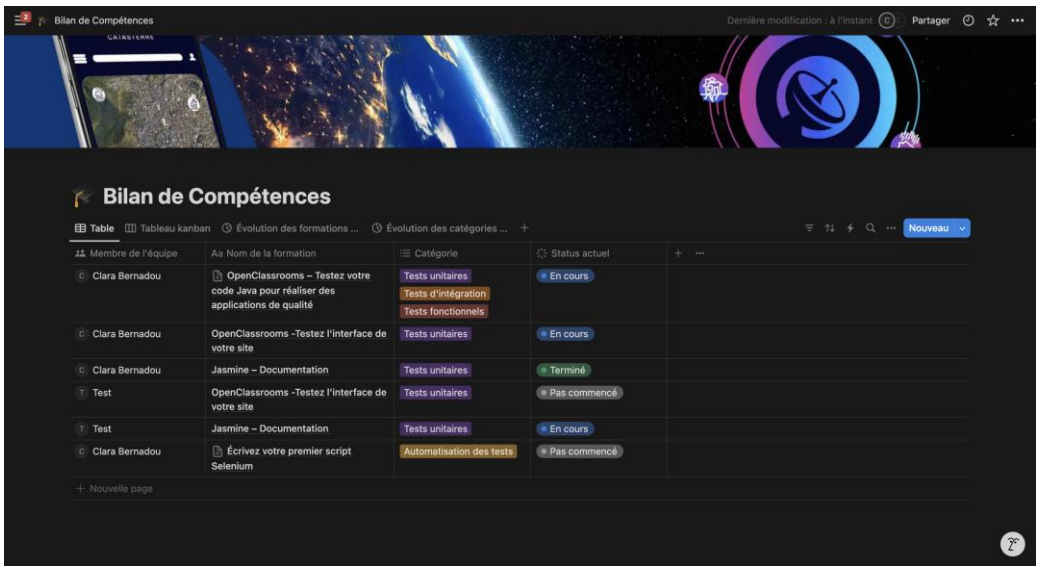
2. Analyse de compétences de l’équipe

Membre de l’équipe	Compétences actuelles	Compétences manquantes
Dimitry	**Langages maîtrisés** : HTML, CSS, JavaScript **Frameworks maîtrisés** : Angular, RxJS **Outils et environnement** : Windows, Linux (Debian)	**Tests** : Les compétences en tests (unitaires, d’intégration, etc.) ne sont pas mentionnées et Rachida nous a signalé un manque de compétences de l’équipe dans ce domaine. Il est probable que Dimitry manque de compétences dans ce domaine, ce qui pourrait être un frein pour l’écriture et l’exécution de tests sur le front-end.
Rachida	**Langages maîtrisés** : Java **Frameworks maîtrisés** : Spring, Hibernate **Outils maîtrisés** : MySQL, Postman **Environnement** : Linux (Debian), Docker	**Tests** : Bien que Rachida soit spécialisée dans le back-end et maîtrise des outils comme Postman pour tester des API, les compétences en tests unitaires et d’intégration pour le back-end (avec Spring, Hibernate, etc.) ne sont pas mentionnées et Rachida nous a signalé un manque de compétences de l’équipe dans ce domaine, ce qui suggère un besoin de développement dans cette compétence.
Jorges	**Spécialisation** : Wireframes/Maquettes avec Figma **Outils maîtrisés** : Figma et Miro	**Aucun écart technique** : Le profil de Jorge est adéquat pour le travail UX/UI demandé dans le projet. Bien qu'il ne soit pas impliqué dans le développement technique, sa contribution sera essentielle pour assurer

	Environnement : Windows	une bonne expérience utilisateur.

3. Outil pour l’analyse continue des compétences

Création avec Notion :



- Créer une page Notion nommée “Bilan de Compétences”.
- Ajouter une vue “Table” pour structurer les données.
    - Ajouter les colonnes suivantes :
        - Membre de l’équipe (Personne) : Nom des membres.
        - Nom de la formation (texte) : Intitulé de la formation.
        - Catégories (sélection) : Exemple : "Tests unitaires", "Tests d’intégration", etc.
        - Status actuel (état) : Options comme "En cours", "Terminé", ou "À faire".
- Configurer les propriétés des colonnes :
    - Définir les options pour les sélections et les statuts.
- Ajouter une vue “Tableau Kanban” :
    - Regrouper par status pour une visualisation plus claire des formations.
- Ajouter des vues “Graphiques” :
    - Configurer les types de graphiques (barres, camemberts, etc.).
    - Définir les axes X (ex. : membres) et Y (ex. : catégories ou status).
    - Personnaliser le style pour un affichage visuel intuitif.

Avantages :

- **Méthode agile et flexible** : Adaptation rapide des priorités et suivi dynamique des compétences en temps réel.

- **Simplicité d’utilisation** : Interface intuitive qui permet à l’équipe d’ajouter et modifier facilement les informations.
- **Visualisation claire des progrès** : Suivi facile des formations et compétences via des vues personnalisées (tableaux, Kanban, graphiques).
- **Suivi centralisé** : Tous les progrès des membres sont centralisés, assurant transparence et visibilité.
- **Personnalisation** : Vues et filtres adaptables pour répondre aux besoins spécifiques de l’équipe.
- **Suivi des écarts** : Identification rapide des lacunes de compétences pour un développement ciblé.
- **Amélioration continue** : Encourage une culture de progression et de collaboration au sein de l’équipe.

4. Formations

a. OpenClassrooms – Testez votre code Java pour réaliser des applications de qualité

Technologies : JUnit, Mockito

Types de tests : Tests unitaires, tests d'intégration, tests fonctionnels

Durée : 10h

Coût : Gratuit

Approche : Formation combinant théorie et pratique

Objectifs :

- Assimiler les concepts fondamentaux des tests unitaires et d'intégration
- Maîtriser l'utilisation de JUnit et Mockito pour rédiger des tests robustes et efficaces

Justification : Cette formation permet de combler les lacunes sur les tests unitaires et d'intégration, essentiels pour la qualité du code. Elle est adaptée à Rachida qui manque d'expérience en tests avec JUnit et Mockito. Elle permet également un gain de compétence rapide, car la durée est courte (10h), ce qui permet de l'intégrer facilement dans un sprint.

Lien : Testez votre code Java

b. OpenClassrooms - Testez l'interface de votre site **(Cours bientôt archivé)**

Technologie : Jasmine

Type de tests : Tests unitaires

Durée : 4h

Coût : Gratuit

Approche : Formation combinant théorie et pratique

Objectifs :

- Comprendre les principes de base du Test-Driven Development (TDD)

- Apprendre à utiliser Jasmine pour écrire des tests unitaires simples et efficaces

Justification : Cette formation permet d'introduire le TDD et les tests unitaires dans le front-end. Elle permet également un gain de compétence rapide, car la durée est très courte (4h), ce qui permet de l'intégrer facilement dans un sprint, idéal pour une montée en compétence rapide de Dimitri sur les tests en front-end.

Lien : Testez l'interface de votre site

c. Jasmine – Documentation

Technologie : Jasmine

Type de tests : Tests unitaires

Durée : 15h

Coût : Gratuit

Approche : Guide qui permet d'écrire étape par étape ses premiers scripts et comprendre les bases des tests unitaires avec Jasmine

Sections à faire :

- Basics
- Your First Suite
- Testing Async Code
- Spies/Mocking
- Matchers and Equality Customization

Objectifs :

- Comprendre les principes des tests unitaires et leur utilité dans le développement.
- Apprendre à écrire des tests simples pour valider le comportement du code.
- Gérer les tests asynchrones avec Jasmine.
- Utiliser les spies pour mocker les appels de fonctions.
- Maîtriser les matchers pour affiner les assertions.

Justification : La documentation officielle de Jasmine est une excellente ressource pour maîtriser les tests unitaires. La formation couvre des sujets essentiels comme les tests asynchrones, les spies et la personnalisation des assertions, qui sont des compétences cruciales pour améliorer la couverture de tests dans l'application.

Lien : Documentation Jasmine

d. Selenium - WebDriver

Technologie : Selenium

Type de tests : Automatisation des tests

Durée : 22h

Coût : Gratuit

Approche : Guide qui permet d'écrire ses premiers scripts et comprendre les bases de l'automatisation des tests

Sections à faire :

- Getting Started
- Driver Sessions
- Supported Browsers
- Waiting Strategies
- Web Elements
- Browser Interactions

Objectifs :

- Comprendre les bases de Selenium WebDriver et de son architecture
- Apprendre à interagir avec les éléments d'une page Web pour automatiser les tests fonctionnels
- Développer des compétences pour écrire des scripts de test automatisés efficaces pour valider les fonctionnalités d'une application

Justification : L'automatisation des tests avec Selenium est un atout majeur pour tester des applications web de manière plus rapide et fiable. Si Rachida et Dimitri doivent améliorer la couverture des tests fonctionnels, cette formation sera particulièrement utile.

Lien : WebDriver



Fiche de poste – Responsable Qualité

Notre équipe professionnelle et très motivée de développement de produits à CATASTERRE (France), qui travaille en SCRUM, recherche un responsable de la qualité engagé à temps plein pour nous aider à améliorer le système de visualisation de catastrophes naturelles via satellite, afin qu'il devienne encore plus efficace.

Type de contrat	CDI
Rémunération	45 k€ / année
Lieu	À distance / Paris

Responsabilités :

- Élaborer des stratégies de tests manuels et automatisés en adéquation avec les exigences techniques du projet, tout en veillant à leur alignement avec les objectifs de qualité de l'entreprise.
- Assurer que toutes les fonctionnalités livrées respectent les normes de qualité à travers des processus rigoureux de validation et d'approbation des versions de production.
- Maintenir et améliorer continuellement les outils et processus de test pour garantir une efficacité maximale.
- Produire des rapports réguliers sur les résultats des tests et les indicateurs clés (KPI) pour évaluer l'efficacité et l'impact des processus de test.

Missions :

- Développer des environnements de tests isolés, incluant des mocks, pour assurer une validation indépendante des modules applicatifs.
- Concevoir, développer et maintenir des scripts d'automatisation (exemple : Selenium/Jasmine pour les tests, JMeter pour les tests de performance).
- Surveiller et améliorer la couverture de code avec des outils comme SonarQube, en s'assurant que les zones critiques sont suffisamment couvertes.
- Rédiger et maintenir une documentation claire des plans et résultats de tests, et transmettre les retours aux équipes concernées pour favoriser l'amélioration continue.

Vous :

Idéalement, vous avez déjà acquis au moins 3 ans d'expérience professionnelle dans le domaine des tests et de l'assurance qualité de systèmes logiciels complexes, après avoir obtenu un diplôme en informatique ou une formation équivalente. Vous avez un esprit d'équipe de premier ordre avec une forte motivation pour résoudre les problèmes des clients à fort impact, en coordination avec votre équipe.

Qualifications :

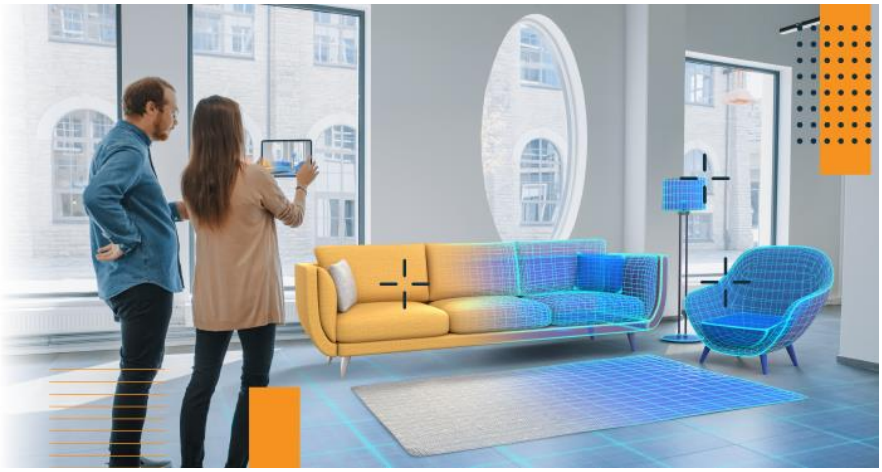
- Capacité à intégrer les pratiques de qualité dans un cadre SCRUM.
- Connaissance des langages : Java 17 avec Spring Boot 3 et Angular 14.
- Maîtrise des principes SOLID et de Test-Driven Development (TDD) et ou de BDD.
- Expérience avec des frameworks de tests tels que JUnit, Jasmine et Selenium.
- Familiarité avec des outils de couverture de code, notamment SonarQube.
- Connaissance d'outils de tests d'API comme Postman.

Attitude et culture idéales :

- Vous avez évolué au sein d'environnements complexes demandant un attrait pour la résolution de problèmes et un esprit positif à toute épreuve !
- Vous possédez le goût de l'engagement et du respect des jalons, et votre quotidien est notamment rythmé par le partage de vos connaissances (Dojo !), les challenges, l'amélioration continue ! La maîtrise de techniques d'animation d'atelier innovantes est un plus.
- Nous recherchons avant tout des personnalités humbles, curieuses et créatives, altruistes et ayant le sens du service, afin de distribuer la bonne parole agile au sein d'une équipe.

Êtes-vous prêt à relever ces défis en intégrant une structure différente des grands cabinets, en pleine expansion et restant fidèle à ses valeurs ?

MAISON
EN PLACE



Plan de test

1. Contexte 2
2. Scénarios 3
   2.1 - Fonctionnalité : Authentication 3
   2.2 - Fonctionnalité : Catalogue produit 4
   2.3 - Fonctionnalité : Simulateur de réalité augmentée 5
   2.4 Fonctionnalité : Tunnel d'achat 6

1. Contexte

La plateforme de MaisonEnPlace permet aux utilisateurs de visualiser et d'acheter des meubles via un simulateur de réalité augmentée. Les objectifs de la V2 incluent :

- Améliorer la fiabilité des fonctionnalités critiques.
- Renforcer la sécurité (remplacement des pratiques obsolètes, ex. HTTP, MD5).
- Introduire une méthodologie rigoureuse de tests (BDD).

Nous utiliserons la méthodologie Behavior Driven Development (BDD) pour structurer nos tests, en formulant les scénarios selon le modèle :

- Étant donné que [contexte]
- Quand [action]
- Alors [résultat attendu]

Cette approche garantit que les tests sont centrés sur les comportements attendus, facilitant leur compréhension par toutes les parties prenantes.

2. Scénarios

2.1 - Fonctionnalité : Authentication

Étape	Résultat attendu	Résultat obtenu	Conclusion
Créer un compte avec des informations valides	Compte créé, redirection au catalogue	*(Laisser vide - devra être rempli par le testeur)*	*(Laisser vide - devra être rempli par le testeur)*
Créer un compte avec des informations invalides	Message d'erreur affiché	*(Laisser vide - devra être rempli par le testeur)*	*(Laisser vide - devra être rempli par le testeur)*
Connexion avec des identifiants valides	Connexion réussie, accès au catalogue	*(Laisser vide - devra être rempli par le testeur)*	*(Laisser vide - devra être rempli par le testeur)*
Connexion avec des identifiants invalides	Message d'erreur est affiché	*(Laisser vide - devra être rempli par le testeur)*	*(Laisser vide - devra être rempli par le testeur)*

Fonctionnalité exclue :

- **Création de compte via Google ou Apple** : Ces fonctionnalités reposent sur des services tiers (API Google et Apple) qui ne relèvent pas directement de la responsabilité de la plateforme. Les éventuels problèmes ou comportements anormaux liés à ces intégrations doivent être gérés par les fournisseurs des services.

2.2 - Fonctionnalité : Catalogue produit

Étape	Résultat attendu	Résultat obtenu	Conclusion
Afficher la liste complète des produits	Liste complète affichée	*(Laisser vide - devra être rempli par le testeur)*	*(Laisser vide - devra être rempli par le testeur)*
Filtrer les produits par type	Produits filtrés correctement	*(Laisser vide - devra être rempli par le testeur)*	*(Laisser vide - devra être rempli par le testeur)*
Rechercher un produit existant	Produit correspondant affiché	*(Laisser vide - devra être rempli par le testeur)*	*(Laisser vide - devra être rempli par le testeur)*
Rechercher un produit inexistant	Message "Aucun produit trouvé" affiché	*(Laisser vide - devra être rempli par le testeur)*	*(Laisser vide - devra être rempli par le testeur)*

2.3 - Fonctionnalité : Simulateur de réalité augmentée

Étape	Résultat attendu	Résultat obtenu	Conclusion
Charger un modèle 3D	Modèle affiché avec succès	*(Laisser vide - devra être rempli par le testeur)*	*(Laisser vide - devra être rempli par le testeur)*
Manipuler le modèle (rotation, zoom)	Manipulations fluides	*(Laisser vide - devra être rempli par le testeur)*	*(Laisser vide - devra être rempli par le testeur)*
Quitter le simulateur	Retour au catalogue	*(Laisser vide - devra être rempli par le testeur)*	*(Laisser vide - devra être rempli par le testeur)*

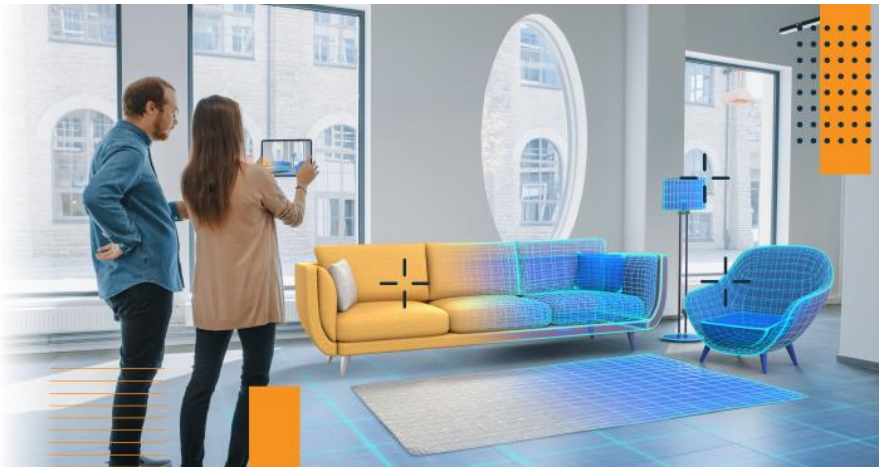
2.4 Fonctionnalité : Tunnel d’achat

Étape	Résultat attendu	Résultat obtenu	Conclusion
Ajouter un produit au panier	Le panier contient le produit	*(Laisser vide - devra être rempli par le testeur)*	*(Laisser vide - devra être rempli par le testeur)*
Valider une commande valide	Commande validée et confirmation affichée	*(Laisser vide - devra être rempli par le testeur)*	*(Laisser vide - devra être rempli par le testeur)*

Fonctionnalité exclue :

- **Tests des méthodes de paiement spécifiques (ex. carte bancaire, PayPal)** : Les transactions sont traitées par des prestataires de paiement sécurisés (exemple : Stripe, PayPal). Ces systèmes sont certifiés selon des standards de sécurité élevés (PCI DSS) et ne nécessitent pas de tests internes approfondis sur leur fonctionnement. Nous nous limiterons à vérifier l’intégration générale, comme la redirection vers le fournisseur et le retour avec un statut valide.

MAISON
EN PLACE



Plan de Sécurité

1. Risques et solutions identifiés 2
2. Annexe : Technologies mentionnées..... 4

1. Risques et solutions identifiés

Risque	Contexte	Gravité (de 1 à 5)	Solutions / outils potentiels
Problèmes de compatibilité	L'application peut être incompatible avec certains navigateurs (versions anciennes d'Internet Explorer), appareils (mobiles ou tablettes spécifiques), ou systèmes d'exploitation (versions obsolètes).	**3/5** - Affecte l'expérience utilisateur mais ne bloque pas toutes les fonctionnalités.	**Tester l'application sur plusieurs configurations** : Utiliser des outils comme BrowserStack* pour détecter les incompatibilités. **Utiliser des polyfills** : Ajouter des bibliothèques comme core-js pour garantir la compatibilité avec les versions plus anciennes de JavaScript.
Défaillances des API tierces	Les services externes, comme l'authentification (Google et Apple) ou les paiements, peuvent devenir inaccessibles à cause de pannes ou de changements non anticipés.	**4/5** - Peut bloquer des fonctionnalités clés comme la création de compte ou la finalisation d'achat.	**Logs et alertes :** Intégrer des journaux détaillés et des alertes pour surveiller l'état des API et intervenir rapidement. **Gestion des erreurs robuste :** Notifier les utilisateurs en cas de problème et permettre de réessayer l'action après un délai.
Performances insuffisantes	L'application peut souffrir de lenteurs ou de faible réactivité, particulièrement lors des pics de trafic.	**5** - Dégradation de l'expérience utilisateur, pouvant entraîner des abandons massifs.	**Utilisation de systèmes de cache :** Implémenter Redis* ou un CDN* pour accélérer les temps de réponse. **Optimisation du chargement des pages :** Implémenter le chargement différé (lazy

			loading) et compresser les fichiers.
Vol de données	Une mauvaise sécurisation des données pourrait entraîner leur interception (vol de données personnelles ou bancaires).	**5/5 -** Conséquences graves sur la confiance des utilisateurs et la conformité réglementaire.	**Cryptage des données sensibles :** Assurer des connexions sécurisées avec HTTPS (TLS*). **Stockage sécurisé :** Crypter les données sensibles côté server avec des algorithmes robustes AES*. **Authentification renforcée :** Implémenter OAuth2* et l'authentification multi-facteurs (MFA*).
Pertes de données critiques	Les données critiques (comme les informations utilisateur) pourraient être perdues à cause d'une panne serveur ou d'un mauvais backup.	**4/5 -** Interruption du service et perte de données sensibles.	**Backups automatiques :** Planifier des sauvegardes automatiques sur des serveurs sécurisés. **Plan de reprise :** Prévoir des procédures pour restaurer rapidement le service. **Redondance :** Configurer des bases de données secondaires pour éviter les pertes.
Accès non autorisé aux serveurs	Des attaquants peuvent exploiter des failles pour accéder aux serveurs de l'application.	**5/5 -** Accès aux informations sensibles, compromission de l'ensemble du système.	**Pare-feu et WAF :** Installer un pare-feu applicatif (WAF*) pour bloquer les tentatives malveillantes. **Audits réguliers :** Effectuer des tests de pénétration pour identifier les failles.

			Restrictions d’accès : Configurer des règles strictes (par exemple : accès basé sur les IP ou les rôles).

2. Annexe : Technologies mentionnées

BrowserStack : Outil pour tester les applications sur divers environnements.

Redis : Système de cache rapide pour améliorer les performances.

CDN : Réseau de serveurs pour accélérer la diffusion des contenus.

TLS : Protocole de sécurité pour protéger les communications.

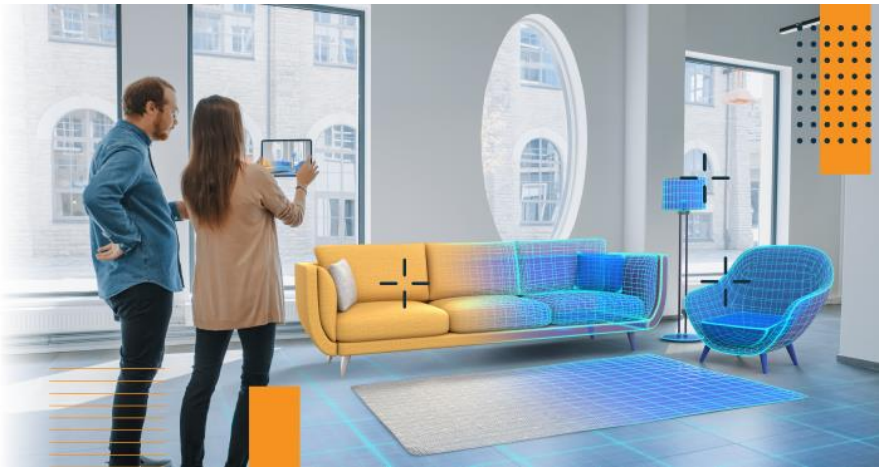
OAuth2 : Standard pour gérer les autorisations d’accès.

AES : Algorithme de cryptage pour sécuriser les données.

MFA : Méthode d’authentification renforcée combinant plusieurs facteurs.

WAF : Pare-feu spécialisé pour protéger les applications web.

MAISON
EN PLACE



Plan de Sauvegarde des Données

1. Définition des besoins 2
   1.1 - Disponibilité et continuité de service 2
   1.2 - Automatisation des sauvegardes 2
   1.3 - Sécurisation des sauvegardes 2
   1.4 - Redondance géographique 2
   1.5 - Tests réguliers de restauration 3
   1.6 - Conformité et traçabilité 3
2. Solution sélectionnée : Solution hybride 4
   2.1 - Avantages 4
   2.2 - Planification des sauvegardes 4
   2.3 - Plan de test de restauration 4
   2.4 - Coûts estimés 5
   2.5 - Rétention des données 5
3. Plan de restauration des données 6

1. Définition des besoins

1.1 - Disponibilité et continuité de service

- **Objectif** :
  Minimiser les interruptions en cas d'incident pour garantir que les utilisateurs puissent acheter des produits à tout moment.

- **Besoins** :
  - Sauvegardes permettant une restauration rapide (RTO* court)
  - Limiter la perte de données à 1 heure ou moins (RPO**)

RTO **(Recovery Time Objective) : Temps maximal toléré pour restaurer un système ou une application après une panne afin de reprendre une activité normale.*

***RPO **(Recovery Point Objective)** : Quantité maximale de données pouvant être perdue (en temps) lors d'un incident. Cela détermine la fréquence des sauvegardes.*

1.2 - Automatisation des sauvegardes

- **Objectif** :
  Réduire les erreurs humaines et garantir la régularité des sauvegardes.

- **Besoins** :
  - Mettre en place des sauvegardes automatiques
  - Programmer des sauvegardes fréquentes (quotidiennes ou horaires selon les données critiques)

1.3 - Sécurisation des sauvegardes

- **Objectif** :
  Protéger les données contre la perte ou la corruption.

- **Besoins** :
  - Stocker les sauvegardes sur un support distinct (Serveur NAS*, serveur dédié)
  - Utiliser le chiffrement AES-256 pour protéger les sauvegardes

NAS **(Network Attached Storage) : Serveur de stockage dédié, connecté au réseau, permettant de centraliser et partager des données ou sauvegardes entre plusieurs utilisateurs ou systèmes.*

1.4 - Redondance géographique

- **Objectif** :
  Assurer la résilience face aux catastrophes locales.

- **Besoins** :

    - Répliquer les sauvegardes dans des emplacements éloignés (cloud ou autre datacenter)

1.5 - Tests réguliers de restauration

- **Objectif** :
  Vérifier que les sauvegardes sont fiables et restaurables.

- **Besoins** :
    - Tester mensuellement la restauration des données
    - Garantir que les restaurations respectent les objectifs de délai (RTO)

1.6 - Conformité et traçabilité

- **Objectif** :
  Répondre aux exigences légales et assurer un suivi des opérations.

- **Besoins** :
    - Documenter les processus de sauvegarde/restauration
    - Conserver des journaux d'activité pour tracer les sauvegardes et restaurations

2. Solution sélectionnée : Solution hybride

Sauvegardes locales :
Utilisation d'un NAS pour des sauvegardes rapides et fréquentes, facilitant la restauration immédiate en cas de problème.

Sauvegardes distantes :
Réplication automatique des sauvegardes locales vers un service cloud (AWS S3, Google Cloud, Azure, etc.) pour protéger les données contre les catastrophes locales.

2.1 - Avantages

- **Disponibilité** :
  Restauration immédiate avec les sauvegardes locales.

- **Sécurité** :
  Résilience accrue grâce à la redondance géographique.

- **Automatisation** :
  Planification des sauvegardes pour éviter les erreurs humaines.

2.2 - Planification des sauvegardes

- **Sauvegardes locales** :
  Quotidiennes à 2h via un NAS.

- **Sauvegardes distantes** :
  Quotidiennes à 3h avec le service cloud AWS S3 pour réduire au minimum les risques de perte de données.

  La fréquence quotidienne répond au besoin de continuité de service, notamment dans un contexte où les utilisateurs accèdent en permanence à l'application.

2.3 - Plan de test de restauration

Fréquence des tests :
- **Tests locaux** : Mensuels.
- **Tests distants** : Trimestriels.

Procédure :
1. Restaurer un jeu de données complet sur un environnement de test.
2. Vérifier l'intégrité des données restaurées.

3. Tester les fonctionnalités critiques (authentification, catalogue, tunnel d'achat).
4. Documenter les résultats des tests.

Les tests réguliers garantissent la fiabilité et la validité des sauvegardes, minimisant ainsi les risques en cas d'incident.

2.4 - Coûts estimés

- **Infrastructure locale (NAS)** :
  - **Synology DS923+** (exemple) : ~800 €
  - **4 disques de 4 To en RAID5** (stockage) : ~500 €

- **Stockage cloud** :
  - **AWS S3 Standard :** 0,023 $/Go/mois.
  - 1 To sur un an (1000 Go x 12 x 0,023 $) : ~265,49 Euro

2.5 - Rétention des données

- **Sauvegardes locales :**
  - **Incrémentales** : Conserver les sauvegardes des 7 derniers jours.
  - **Complètes** : Conserver 30 jours de sauvegardes.
- **Sauvegardes distantes :**
  - Rétention minimale d'un mois avec possibilité d'archivage longue durée.

3. Plan de restauration des données

1. **Identifier la défaillance** :
   Repérer les signes du problème (perte de données, indisponibilité du système, corruption).

2. **Notifier les équipes** :
   Informer l'équipe chargée des sauvegardes et des restaurations pour qu'elle prenne en main la situation.

3. **Localiser la sauvegarde fonctionnelle** :
   Identifier la dernière sauvegarde saine (complète ou incrémentale).

4. **Isoler les services défaillants** :
   Mettre hors ligne les services affectés pour éviter d'aggraver la situation.

5. **Restaurer les données** :
   - Charger la sauvegarde complète depuis le stockage (local ou cloud).
   - Appliquer les sauvegardes incrémentales, si nécessaire.
   - Vérifier l'intégrité des données : Contrôler la cohérence et la complétude des données restaurées.

6. **Tester les fonctionnalités critiques** :
   Vérifier que les fonctions essentielles (authentification, catalogue, achats, etc.) fonctionnent correctement.

7. **Redémarrer les services** :
   Relancer les services une fois la restauration validée.

8. **Informer les parties prenantes** :
   Communiquer avec les utilisateurs et/ou équipes internes pour expliquer la situation et rassurer sur le rétablissement.

9. **Analyser et améliorer** :
   Documenter l'incident, ses causes et les actions entreprises. Mettre à jour le plan de sauvegarde/restauration pour prévenir des problèmes similaires à l'avenir.



YOUR CAR
YOUR WAY

Business Requirements

1. Objet du document 2
2. Contexte 2
3. Périmètre 2
4. Liste des fonctionnalités 3
5. Exigences particulières 5

1. Objet du document

Le document “Business Requirements” liste les fonctionnalités à implémenter pour le projet Your Car Your Way. Ces fonctionnalités sont exprimées du point de vue métier sous la forme d’actions que l’utilisateur peut effectuer sur l’application.

2. Contexte

Your Car Your Way est une entreprise de location de voitures. Les clients utilisent actuellement des applications web qui ne correspondent plus aux besoins fonctionnels ni aux contraintes techniques. Une nouvelle application centralisée pour tous les clients doit être créée.

3. Périmètre

Les fonctionnalités décrites dans ce document concernent la première version de la nouvelle application Your Car Your Way. Cette application sera déployée à l'international, et utilisée par tous les clients de l’entreprise. Cette application est à destination des clients, et ne concerne pas les actions que les employés de Your Car Your Way doivent faire en agence.

4. Liste des fonctionnalités

Gestion du profil

- Connexion via la page de connexion.
- Inscription via la page d'inscription.
- Modifier ses informations personnelles (nom, prénom, date de naissance, adresse) via la page de profil.
- Supprimer son compte.

Gestion des agences

- Consulter la liste des agences de location.
- Affichage des détails d'une agence (adresse, horaires d'ouverture, services disponibles, véhicules disponibles).
- Rechercher / filtrer les agences avec une barre de recherche

Gestion d’une location de voitures

- Afficher les offres de location après avoir rempli un formulaire de recherche avec les critères suivants :
    - Ville de départ ;
    - Ville de retour ;
    - Date et heure de début ;
    - Date et heure de retour ;
    - Catégorie du véhicule.
- Consulter le détail d’une offre de location.
- Réserver une location correspondant à une offre, ce qui inclut :
    - Fournir ses informations personnelles (récupérables depuis le profil si présentes) ;
    - Effectuer le paiement.
- Consulter l’historique de ses réservations (passées et en cours).

- Consulter une réservation.
- Modifier et annuler une réservation.

Autres

- Envoyer un message au support (communication synchrone)
- Visio avec le support (communication synchrone)
- Envoyer un message à un chatbot si vous avez une question (communication asynchrone)

5. Exigences particulières

- La modification d’une réservation est possible jusqu’à 48 h avant le début de cette dernière.

- À moins d'une semaine du début de la réservation, YourCarYourWay ne rembourse que 25 % du montant total de la réservation.

- Les catégories du véhicule reprennent la norme ACRISS : https://www.acriss.org/car-codes/.

- La suppression du compte implique de saisir le mot de passe du compte.

- Les applications utilisées en agence doivent avoir accès à une API pour consulter et modifier les données traitées par l’application à destination des clients. Les opérations CRUD standard sont requises pour chaque domaine (exemple : utilisateur, réservation, etc.).

- La gestion du paiement doit être externalisée auprès d’un fournisseur de service de paiement en ligne (par exemple : Stripe).



YOUR CAR
YOUR WAY

Architecture Definition Document

1. Objet du document 2
2. Objectifs du projet 3
3. Principes d'architecture 4
4. Architecture existante 5
5. Architectures 6
6. Justification de l'approche architecturale 12
7. Architectures de transition 16

1. Objet du document

Ce document permet de modéliser l’architecture qui sera réalisée par les équipes de développement. Au-delà de la modélisation graphique, il s’agit également d’énoncer les principes sur lesquels l’architecture s'appuie, de justifier cette approche architecturale, mais également d’indiquer des transitions si nécessaire.

2. Objectifs du projet

Améliorer l’expérience client :

- Créer une application centralisée, intuitive et accessible à l’international.
- Simplifier la gestion des profils, recherches et réservations.

Faciliter la location de voitures :

- Centraliser les informations des agences et des offres des véhicules.
- Optimiser la recherche et la modification des réservations.

Optimiser les revenus :

- Appliquer une politique claire d’annulation et limiter les remboursements tardifs.
- Intégrer un système de paiement sécurisé via Stripe.

Renforcer la collaboration agences/clients :

- Fournir une API permettant aux agences de mettre à jour leurs données en temps réel.

Améliorer le support client :

- Proposer un chatbot pour les requêtes simples.
- Offrir un support synchrone via messagerie et visioconférence.

Garantir la sécurité et la conformité :

- Respecter les normes de sécurité (PCI DSS et RGPD) pour les paiements et les données personnelles.

3. Principes d’architecture

Modularité :

L’architecture sera divisée en modules distincts (gestion des profils, réservations, paiements, etc.) pour une meilleure maintenabilité.

Scalabilité :

Les composants doivent être conçus pour supporter une montée en charge et des pics de trafic.

Interopérabilité :

L’application doit utiliser une d’API RESTful pour les interactions entre applications client et agence.

Sécurité :

Intégration de JWT et OAuth2 pour l’authentification, chiffrement des données sensibles.

Performance :
Garantir un temps de réponse moyen inférieur à 200ms pour les principales opérations.

Portabilité :
Utiliser Docker pour garantir des déploiements consistants dans différents environnements.

4. Architecture existante

Le projet ne s'appuie pas sur un produit existant qui serait repris puis mis à jour. Il n’y a donc pas d’architecture existante à définir.

5. Architectures

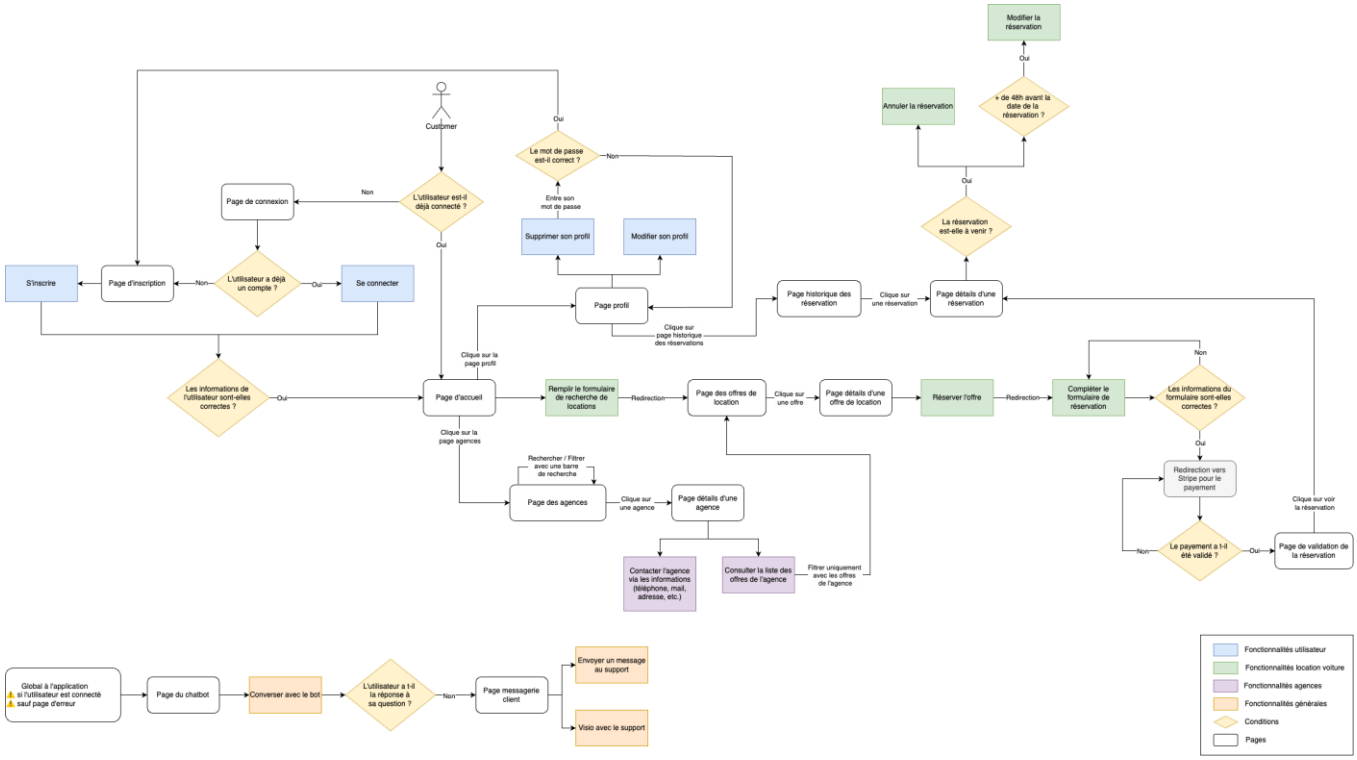
a. Métier

L’application permet les actions suivantes :

- **Gestion des profils utilisateurs :**
    - Inscription et connexion sécurisée (email et mot de passe, ou authentification sociale).
    - Modification et consultation des informations personnelles (nom, adresse, numéro de téléphone, etc.).
- **Recherche et réservation de véhicules :**
    - Recherche avancée avec des critères comme la ville de départ, la date et l’heure de prise en charge/retour, et la catégorie du véhicule.
    - Consultation des offres détaillées, incluant les tarifs, les caractéristiques des véhicules, et la disponibilité.
    - Réservation rapide avec un processus fluide, incluant la récupération des données du profil utilisateur et le paiement en ligne.
- **Gestion des réservations :**
    - Visualisation des réservations en cours et passées dans un historique clair.
    - Modification ou annulation des réservations, avec des règles d’annulation adaptées (par exemple, remboursement partiel en cas d’annulation tardive).
- **Support client :**
    - Chatbot intégré pour des réponses immédiates aux questions fréquentes.
    - Messagerie directe et visioconférence pour des interactions personnalisées avec le support.
- **Paiements sécurisés en ligne :**

- Intégration de Stripe pour gérer les transactions financières de manière fiable et conforme aux normes de sécurité.

Schéma fonctionnel :



---

b. De données

Les entités principales :

- **Utilisateurs :**
    - Informations personnelles (nom, prénom, email, adresse, numéro de téléphone).
- **Agences :**
    - Informations sur les agences (nom, adresse, horaires d'ouverture, coordonnées).
    - Relation avec les véhicules disponibles.
- **Véhicules :**

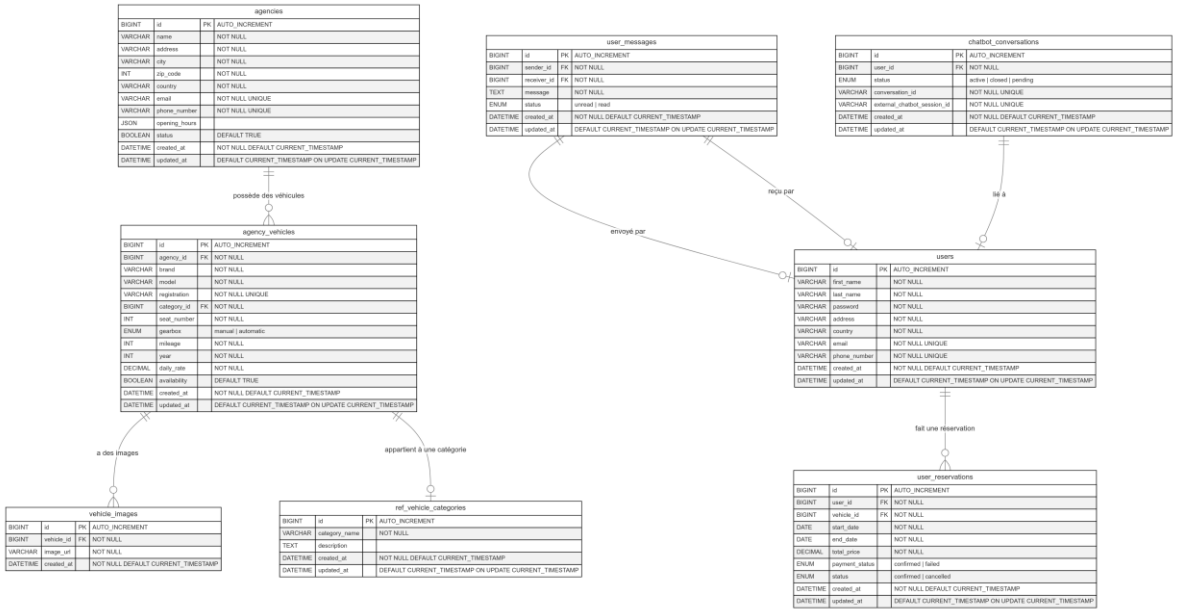
    - Caractéristiques des véhicules (marque, modèle, année, kilométrage, catégorie, tarif journalier).
    - Disponibilité et images associées.
- **Réservations :**
    - Détails des réservations (utilisateur, véhicule, agence, dates de début et de fin, état de la réservation).
    - Historique des modifications ou annulations.
- **Paiements :**
    - Informations sur les transactions (montant, méthode de paiement, statut).
- **Messages :**
    - Gestion des communications entre utilisateurs et support (asynchrones ou synchrones).

Relations clés :

- Une agence peut posséder plusieurs véhicules (relation 1-N*).
- Un utilisateur peut effectuer plusieurs réservations (relation 1-N*).
- Chaque réservation relie un utilisateur, un véhicule et une agence.
- Les messages peuvent relier plusieurs utilisateurs au support (relation N-N**).

*relation 1-N : One-to-Many
**relation N-N : Many-to-Many
***Respect la normalisation 3NF (Troisième Forme Normale) : Utilisée en conception de bases de données relationnelles pour structurer les données afin de minimiser la redondance et garantir l'intégrité des données.

Schéma des tables de données et de leurs relations :




```
CREATE TABLE users ( id BIGINT PRIMARY KEY AUTO_INCREMENT, first_name VARCHAR(255)
NOT NULL, last_name VARCHAR(255) NOT NULL, password VARCHAR(255) NOT NULL, address
VARCHAR(255) NOT NULL, country VARCHAR(255) NOT NULL, email VARCHAR(255) NOT NULL
UNIQUE, phone_number VARCHAR(255) NOT NULL UNIQUE, created_at DATETIME NOT NULL
DEFAULT CURRENT_TIMESTAMP, updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP );

CREATE TABLE agencies ( id BIGINT PRIMARY KEY AUTO_INCREMENT, name VARCHAR(255) NOT
NULL, address VARCHAR(255) NOT NULL, city VARCHAR(255) NOT NULL, zip_code INT NOT
NULL, country VARCHAR(255) NOT NULL, email VARCHAR(255) NOT NULL UNIQUE, phone_number
VARCHAR(255) NOT NULL UNIQUE, opening_hours JSON, status BOOLEAN DEFAULT TRUE,
created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP, updated_at DATETIME DEFAULT
CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP );

CREATE TABLE ref_vehicle_categories ( id BIGINT PRIMARY KEY AUTO_INCREMENT,
category_name VARCHAR(255) NOT NULL, description TEXT, created_at DATETIME NOT NULL
DEFAULT CURRENT_TIMESTAMP, updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP );

CREATE TABLE agency_vehicles ( id BIGINT PRIMARY KEY AUTO_INCREMENT, agency_id BIGINT
NOT NULL, brand VARCHAR(255) NOT NULL, model VARCHAR(255) NOT NULL, registration
VARCHAR(255) NOT NULL UNIQUE, category_id BIGINT NOT NULL, seat_number INT NOT NULL,
gearbox ENUM('manual', 'automatic'), mileage INT NOT NULL, year INT NOT NULL,
daily_rate DECIMAL NOT NULL, availability BOOLEAN DEFAULT TRUE, created_at DATETIME
NOT NULL DEFAULT CURRENT_TIMESTAMP, updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON
UPDATE CURRENT_TIMESTAMP, FOREIGN KEY (agency_id) REFERENCES agencies(id), FOREIGN
KEY (category_id) REFERENCES ref_vehicle_categories(id) );

CREATE TABLE vehicle_images ( id BIGINT PRIMARY KEY AUTO_INCREMENT, vehicle_id BIGINT
NOT NULL, image_url VARCHAR(255) NOT NULL, created_at DATETIME NOT NULL DEFAULT
CURRENT_TIMESTAMP, FOREIGN KEY (vehicle_id) REFERENCES agency_vehicles(id) );

CREATE TABLE user_reservations ( id BIGINT PRIMARY KEY AUTO_INCREMENT, user_id BIGINT
NOT NULL, vehicle_id BIGINT NOT NULL, start_date DATE NOT NULL, end_date DATE NOT
NULL, total_price DECIMAL NOT NULL, payment_status ENUM('confirmed', 'failed'),
status ENUM('confirmed', 'cancelled'), created_at DATETIME NOT NULL DEFAULT
CURRENT_TIMESTAMP, updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE
```

```
CURRENT_TIMESTAMP, FOREIGN KEY (user_id) REFERENCES users(id), FOREIGN KEY
(vehicle_id) REFERENCES agency_vehicles(id) );

CREATE TABLE user_messages ( id BIGINT PRIMARY KEY AUTO_INCREMENT, sender_id BIGINT
NOT NULL, receiver_id BIGINT NOT NULL, message TEXT NOT NULL, status ENUM('unread',
'read'), created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP, updated_at DATETIME
DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP, FOREIGN KEY (sender_id)
REFERENCES users(id), FOREIGN KEY (receiver_id) REFERENCES users(id) );

CREATE TABLE chatbot_conversations ( id BIGINT PRIMARY KEY AUTO_INCREMENT, user_id
BIGINT NOT NULL, status ENUM('active', 'closed', 'pending'), conversation_id
VARCHAR(255) NOT NULL UNIQUE, external_chatbot_session_id VARCHAR(255) NOT NULL
UNIQUE, created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP, updated_at DATETIME
DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP, FOREIGN KEY (user_id)
REFERENCES users(id) );
```

---

c. Technologique

Frontend :
Angular 16 pour une interface utilisateur réactive et moderne.

Backend :
Spring Boot 2.7.3 pour les API REST de manière performante et sécurisée.

Base de données :
MySQL 8.1 pour le stockage relationnel des données.

Communication :

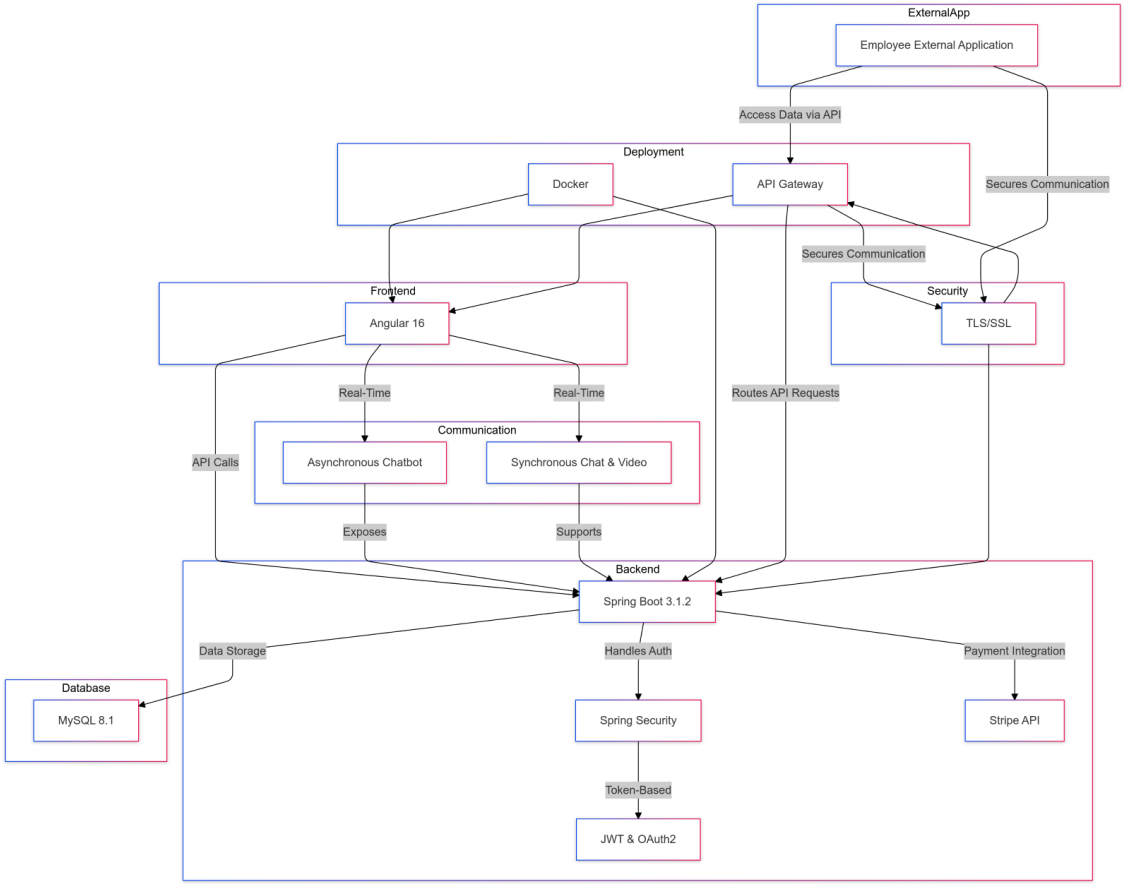
- **Asynchrone** : Chatbot (API REST).
- **Synchrone** : Messagerie (API REST, possibilité d'évolution avec les websockets) et visioconférence (WebRTC).

Authentification et sécurité :
Spring Security avec JWT et OAuth2 pour gérer l'authentification et l'autorisation de manière sécurisée.

Paiements :
Intégration de Stripe pour la gestion des paiements en ligne.

Déploiement :
Docker pour la conteneurisation et la portabilité des services.

Schéma de l’architecture logicielle (modulaire) :



6. Justification de l’approche architecturale

L’approche architecturale retenue répond aux objectifs business et techniques grâce aux éléments suivants :

Cohérence avec les objectifs business :

- Une architecture modulaire permet de centraliser efficacement la gestion des profils, des réservations et des paiements, répondant ainsi aux besoins de simplicité et d’efficacité pour les clients.

- L’interopérabilité offerte par les API RESTful garantit une intégration fluide entre les clients, agences, et systèmes tiers.

- Le support client est amélioré grâce à une gestion claire des communications asynchrones (chatbot) et synchrones (messagerie, visioconférence).

Respect des principes d’architecture :

- **Modularité**
  Chaque composant est conçu indépendamment, facilitant ainsi le développement, les tests, et la maintenance.

- **Scalabilité**
  Les technologies choisies (Spring Boot, Angular, Docker) permettent une montée en charge adaptée à une audience internationale.

- **Interopérabilité**
  Utilisation d’APIs standardisées pour garantir une intégration fluide avec les systèmes tiers et les outils des agences.

- **Expérience utilisateur**
  La rapidité de réponse et une interface intuitive sont des priorités pour garantir la satisfaction des clients.

Sécurité renforcée :

- L’utilisation de Spring Security avec JWT et OAuth2 assure une gestion robuste des droits d’accès, tout en protégeant les données sensibles des utilisateurs.

- L’intégration de Stripe pour les paiements respecte les normes PCI DSS, garantissant ainsi la sécurité des transactions.

Suivi des normes et bonnes pratiques reconnues :

- Respect des réglementations RGPD pour la gestion des données personnelles.
- Conformité avec les standards internationaux de sécurité et de paiement (ex. PCI DSS).
- Adoption des frameworks et technologies modernes largement supportés et documentés (Angular, Spring Boot, Docker).

Robustesse technique et performance :

- Angular 16 et Spring Boot 3.1.2 offrent une base moderne et performante, adaptée aux montées en charge et aux exigences de performance élevées.
- Docker permet une portabilité accrue et facilite le déploiement dans des environnements variés.

Évolutivité et maintenabilité :

- L’architecture modulaire facilite l’ajout de nouvelles fonctionnalités sans impact sur les services existants.
- Les technologies choisies, comme Angular et Spring Boot, sont bien supportées et documentées, ce qui réduit les coûts de maintenance et garantit une évolution continue.

Expérience utilisateur améliorée :

- Les temps de réponse rapides et les interfaces intuitives améliorent la satisfaction des clients.

- La prise en charge des communications temps réel (chat et visioconférence) renforce l'engagement des utilisateurs.

Comparaison entre différentes architectures :

- **Architecture monolithique classique :**
  Bien que simple à déployer et à maintenir, elle manque de flexibilité pour s'adapter aux évolutions futures. Tout changement nécessite un redéploiement complet.

- **Architecture microservices :**
  Idéale pour des systèmes complexes et distribués, elle introduit cependant une complexité importante, avec la gestion de multiples bases de données, la latence des appels entre services et l'orchestration (ex. Kubernetes).

- **Architecture modulaire (choisie) :**
  Combine les avantages du monolithique et des microservices. Les modules sont indépendants dans le code mais partagent une base de données et un déploiement commun, simplifiant ainsi la gestion tout en offrant une bonne évolutivité.

Pourquoi une architecture modulaire est adaptée à ce projet ?

- **Simplicité initiale :**
  Une architecture modulaire centralisée permet de gérer l'ensemble des fonctionnalités (authentification, réservations, paiements, etc.) dans une seule application. Cela réduit la complexité technique par rapport à une architecture microservices.

- **Coût réduit :**
  Avec une architecture monolithique modulaire, les coûts de développement, de déploiement et de maintenance sont plus faibles. Elle évite la surcharge liée à la gestion et à l'orchestration de multiples services.

- **Adaptée à la taille du projet :**
  Pour un projet en phase de développement initial ou avec des fonctionnalités centralisées, une architecture monolithique est suffisante. Une migration vers des microservices peut être envisagée si les besoins évoluent (ex. : augmentation du trafic ou nouvelles fonctionnalités majeures).

- **Rapidité de mise en œuvre :**

Spring Boot permet de structurer facilement une architecture modulaire tout en gardant un déploiement unique. Cela accélère les cycles de développement et réduit les délais de livraison.

7. Architectures de transition

Phase 1 : Fonctionnalités essentielles

- Implémentation des fonctionnalités de base : inscription, connexion, gestion des profils, recherche de véhicules, réservation, et paiement sécurisé.

- Mise en place des API REST pour la communication entre le frontend et le backend.

Phase 2 : Amélioration des interactions client

- Intégration du chatbot pour les communications asynchrones avec les utilisateurs.

- Ajout de l'historique des réservations pour améliorer la transparence.

Phase 3 : Support avancé et scalabilité

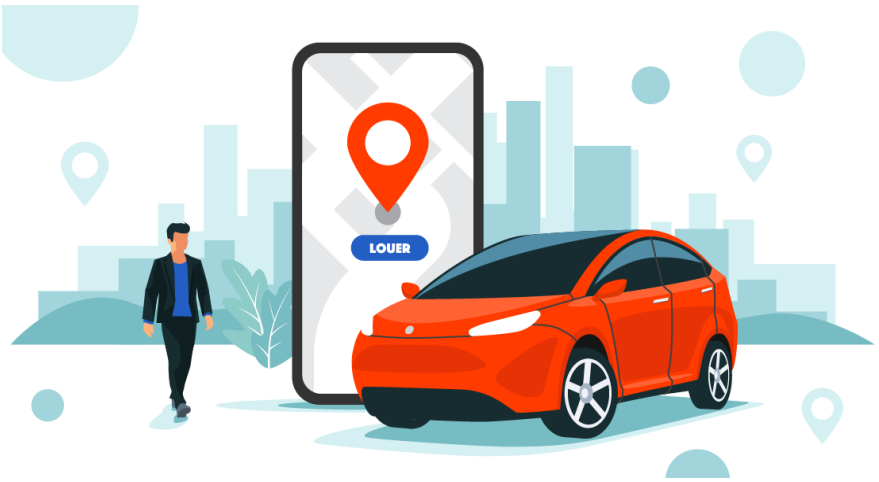
- Déploiement de la messagerie directe et des appels vidéo (visioconférence) pour les interactions synchrones avec le support client.

- Optimisation de la gestion des données avec des stratégies de cache pour améliorer les performances.

Phase 4 : Évolutivité et modularisation

- Préparation à une éventuelle migration vers des microservices si les besoins du projet évoluent (augmentation du trafic ou ajout de nouvelles fonctionnalités).

- Ajout de tests automatisés (unitaires, d'intégration, et E2E) pour renforcer la qualité et la fiabilité du système.



YOUR CAR
YOUR WAY

Compliance Assessment

1. Description générale de l’architecture 2
2. Liste de contrôle de l’architecture terminée 3
3. Services ou composants tiers logiciels 4
4. Gestion des données 5
5. Infrastructure 6
6. Sécurité 7

1. Description générale de l’architecture

L’architecture se repose sur une architecture modulaire centralisée, s’appuyant sur Angular 16 pour le front-end, Spring Boot 3.1.2 pour le back-end, et MySQL 8.1 pour la base de données relationnelle. Les interactions entre les modules sont basées sur des APIs RESTful. L’authentification est assurée par Spring Security avec JWT et OAuth2. Les paiements en ligne utilisent l’intégration de Stripe, et le déploiement est effectué via Docker.

2. Liste de contrôle de l’architecture terminée

Gestion des utilisateurs :

- **Contrôle :**
  Vérifier que les utilisateurs peuvent s’inscrire, se connecter, et modifier leur profil facilement.

- **Critère :**
  Les utilisateurs doivent pouvoir s’inscrire et se connecter sans erreur, avec un taux de succès supérieur à 99%.

Réservation de services :

- **Contrôle** :
  Vérifier que la recherche et la réservation de services (comme la location de véhicules) fonctionne correctement.

- **Critère :**
  La recherche doit afficher les bonnes options (ville, date, heure, catégorie) et se mettre à jour rapidement (en moins de 1 seconde).

Support client :

- **Contrôle :**
  Vérifier que le chatbot et le support en ligne répondent correctement aux demandes des utilisateurs.

- **Critère :**
  Le chatbot doit résoudre au moins 80% des demandes sans intervention humaine.

Performances :

- **Contrôle :**
  Vérifier que l’application répond rapidement, même en cas de forte utilisation.

- **Critère :**
  Le temps de réponse des services API ne doit pas dépasser 200 millisecondes.

3. Services ou composants tiers logiciels

Paiements avec Stripe :

- **Contrôle :**
  Vérifier l’intégration du système de paiement Stripe et qu'il respecte les normes de sécurité.

- **Critère :**
  Les paiements doivent être traités sans erreur et en toute sécurité.

APIs RESTful :

- **Contrôle :**
  Vérifier que les communications entre le frontend et le backend (via les APIs) sont fiables.

- **Critère :**
  Les appels API doivent être réussis dans 100% des cas.

4. Gestion des données

Base de données MySQL :

- **Contrôle :**
  Vérifier que la base de données est bien structurée et que les données sont stockées correctement.

- **Critère :**
  Les opérations (ajout, modification, suppression) doivent fonctionner sans erreur.

Respect de la confidentialité (RGPD) :

- **Contrôle :**
  Vérifier que les données personnelles des utilisateurs sont protégées et peuvent être supprimées sur demande.

- **Critère :**
  Les données sensibles doivent être chiffrées et supprimées en 24 heures si demandé par l'utilisateur.

5. Infrastructure

Docker :

- **Contrôle :**
  Vérifier que l'application peut être facilement déployée dans différents environnements (test et production).

- **Critère :**
  Les déploiements doivent réussir sans ajustement de configuration.

Scalabilité :

- **Contrôle :**
  Vérifier que l'application peut supporter une augmentation du nombre d'utilisateurs sans ralentir.

- **Critère :**
  L'application doit supporter deux fois plus d'utilisateurs sans perte de performance.

6. Sécurité

Authentification et sécurité des données :

- **Contrôle :**
  Vérifier que l'authentification avec JWT fonctionne correctement et que les données sensibles sont protégées.

- **Critère :**
  Tous les utilisateurs doivent être correctement authentifiés avec JWT, et les communications doivent être sécurisées avec HTTPS.

Respect des normes de sécurité (PCI DSS pour les paiements) :

- **Contrôle :**
  Vérifier que les paiements sont traités selon les normes de sécurité PCI DSS.

- **Critère :**
  Tous les paiements doivent respecter les règles de sécurité sans incident.

Your Car Your Way

Made with Gamma

Introduction

Made with Gamma

Présentation de Your Car Your Way

- Entreprise internationale de location de voitures, active depuis **20 ans**.
- Applications actuelles **obsolètes**, ne répondant plus aux exigences des clients et aux nouvelles contraintes techniques.

Objectif de la présentation

- Expliquer et **justifier les choix architecturaux** en fonction des besoins métiers et techniques.
- Présenter les fonctionnalités clés et la **démonstration du POC**.

Made with Gamma

Objectifs du projet

Made with Gamma

Pourquoi une refonte ?

1

Améliorer l’expérience utilisateur

Interface intuitive, fluide et accessible à l’international.

2

Faciliter la gestion des locations

Centralisation des informations des agences et véhicules.

3

Optimiser la rentabilité

Stratégie d’annulation claire et paiements sécurisés via Stripe.

4

Favoriser l’interaction avec les agences

API pour mise à jour des offres en temps réel.

5

Renforcer le support client

Chatbot et service de messagerie/visioconférence.

6

Garantir la sécurité et la conformité

Respect du **RGPD** et des normes **PCI DSS**.

Made with Gamma

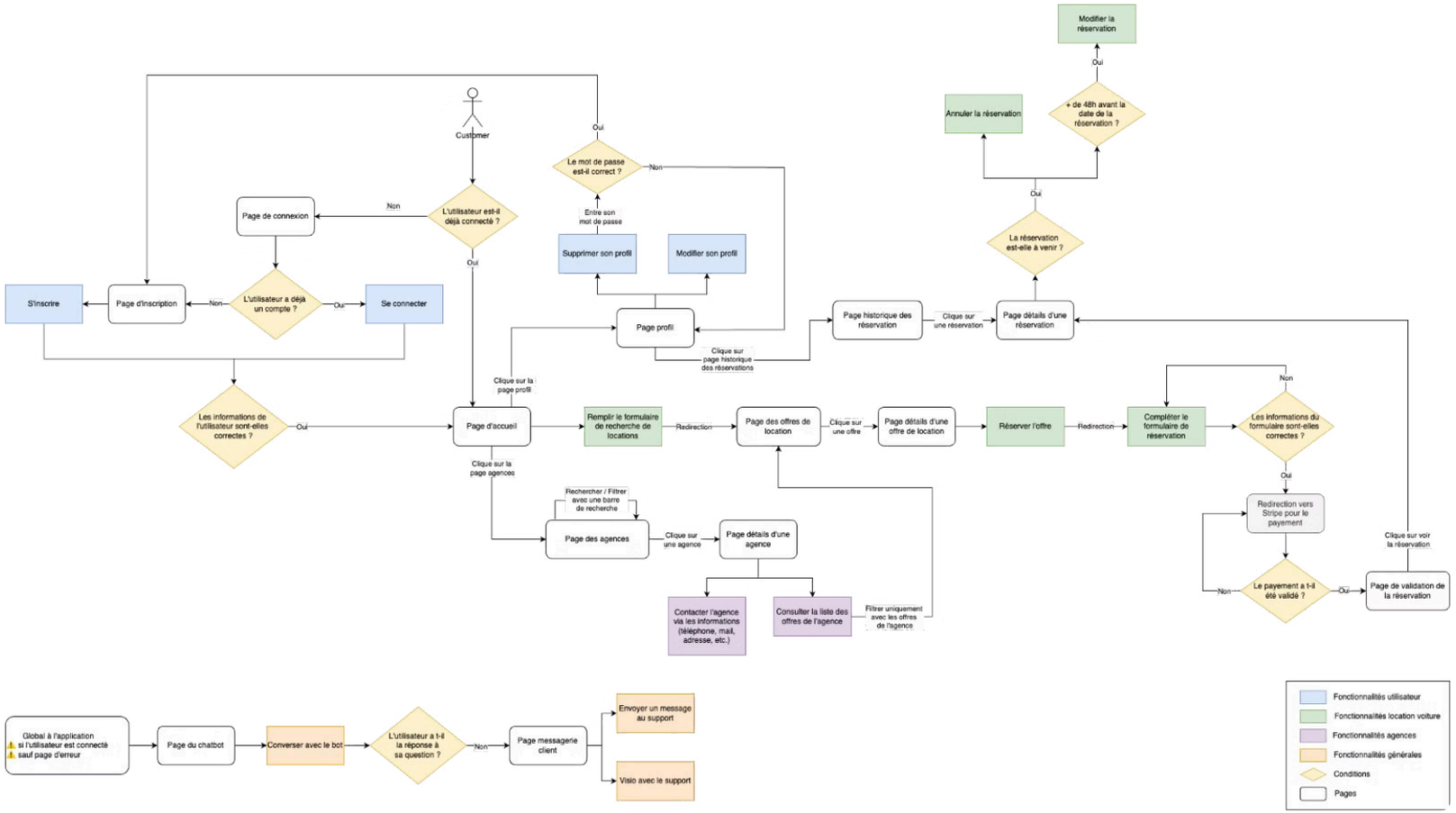
Exigences

- **Modification des réservations jusqu’à 48h avant le début**.
- **Remboursement limité à 25%** si annulation à moins d’une semaine.
- **Norme ACRISS** pour la classification des véhicules.
- **API en agence** pour gérer les réservations et utilisateurs via opérations CRUD.

Les **normes ACRISS** (**Association of Car Rental Industry Systems Standards**) sont un ensemble de critères utilisés dans l'industrie de la location de voitures pour **standardiser la classification des véhicules**. Elles attribuent des codes à différents types de voitures en fonction de leur **taille**, de leur **capacité**, de leur **type de transmission**, de leur **carburant** et d'autres caractéristiques spécifiques.

Made with Gamma

Fonctionnalités Clés



Gestion des profils

Connexion, inscription, modification, suppression.

Gestion des agences

Recherche, consultation des détails.

Location de véhicules

Recherche avancée, réservation et paiement.

Support client

Chatbot et assistance par messagerie/visioconférence.

Made with Gamma

Architecture Technique et Technologies

Made with Gamma

Technologies

Frontend 🎨

Angular 16 → Interface fluide et interactive.

Backend 👩‍💻

Spring Boot 2.7.3 → Performances élevées et scalabilité.

Base de données 💽

MySQL 8.1 → Stockage relationnel optimisé.

Sécurité 🔐

Spring Security, JWT, OAuth2 → Gestion sécurisée des sessions.

Déploiement 💻

Docker → Portabilité et homogénéité des environnements.

Made with Gamma

Principes d'architecture appliqués

Les **principes d’architecture** définissent les règles essentielles pour concevoir un système **efficace, évolutif et cohérent**. Ils guident les choix techniques afin d’assurer **performance, sécurité et facilité de maintenance**.

1

Modularité

Composants indépendants (profils, réservations, paiements).

2

Scalabilité

Support de montée en charge et pics de trafic.

3

Interopérabilité

APIs RESTful pour connecter agences et partenaires.

4

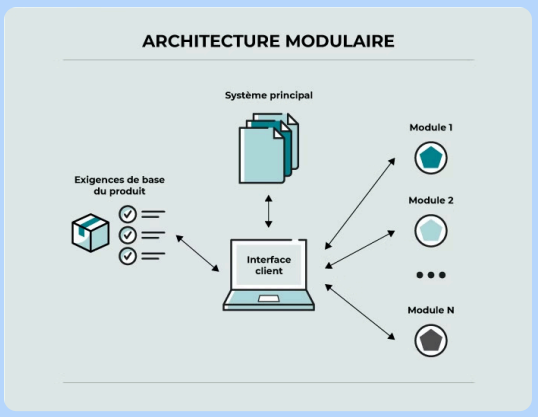
Sécurité

Cryptage des données et conformité PCI DSS/RGPD.

Le **PCI DSS** (**Payment Card Industry Data Security Standard**) est un ensemble de normes de sécurité pour **protéger les informations** des cartes de paiement.

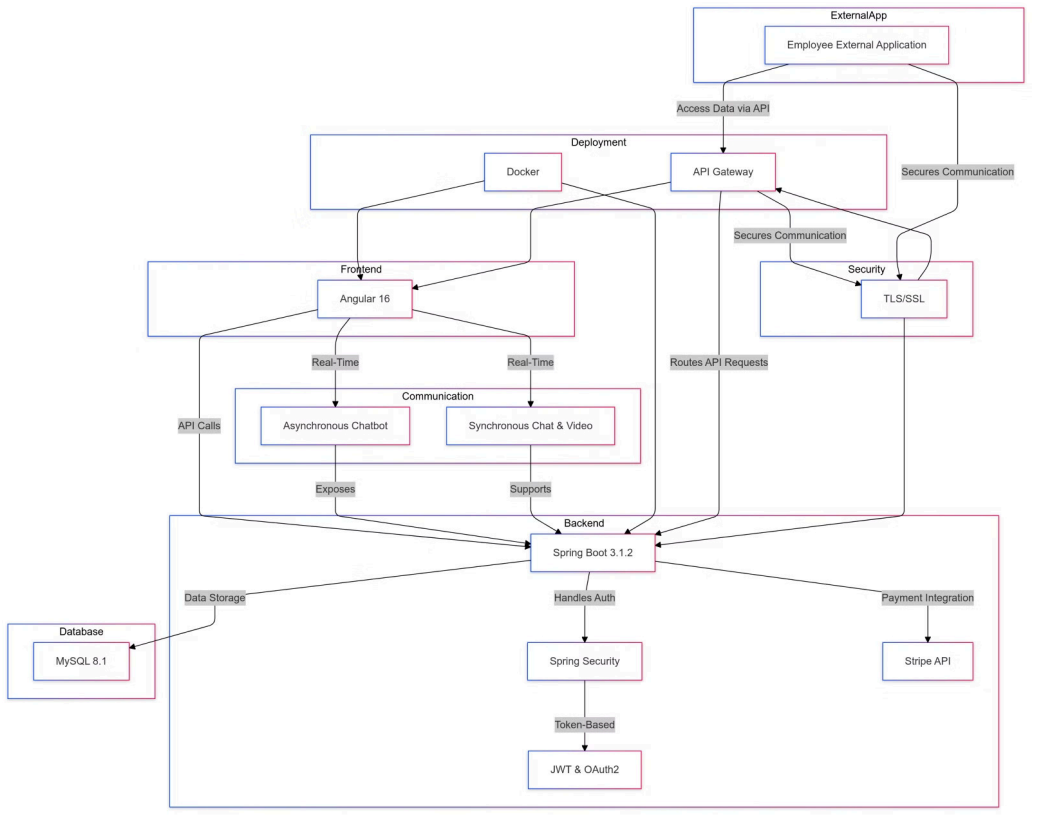
Made with Gamma

Justification du Choix de l’Architecture Modulaire



Une **architecture modulaire** divise l’application en plusieurs **blocs indépendants**, facilitant **le développement, la maintenance et l’évolution** du système. Chaque module remplit une fonction précise tout en restant connecté au système principal.

Notre architecture



Notre architecture se repose bien sur les **quatre parties essentielles** :

1. **Exigences de base** : Fonctions minimales essentielles (authentification, gestion des réservations, paiements sécurisés).
2. **Système principal** : Backend **Spring Boot** pour orchestrer les données, l’authentification et la communication entre les modules.
3. **Interface client** : Frontend **Angular**, garantissant une interaction fluide et intuitive pour l’utilisateur.
4. **Modules complémentaires** :
   - **Chatbot asynchrone** pour le support client.
   - **Chat & visioconférence** pour une assistance en temps réel.
   - **Paiement sécurisé** via Stripe API.

Pourquoi une architecture modulaire ?

Alignement avec les objectifs business

- Centralisation et accessibilité des profils, réservations et paiements.
- APIs RESTful garantissant l’interopérabilité entre agences et clients.
- Expérience client améliorée avec des services en temps réel (chat, vidéo).

Respect des principes d’architecture

- **Modularité** → Développement et maintenance facilités.
- **Scalabilité** → Technologies adaptées à la croissance.
- **Interopérabilité** → APIs standardisées pour une communication fluide.
- **Performance** → Temps de réponse inférieur à **200 ms**.

Sécurité et conformité

- Gestion des accès via **JWT & OAuth2**.
- Transactions Stripe sécurisées et conformes **PCI DSS**.
- Respect du **RGPD** pour la gestion des données personnelles.

Évolutivité et maintenabilité

- Possibilité **d’ajouter de nouvelles fonctionnalités** sans impacter l’existant.
- Technologies modernes et largement documentées.

Made with Gamma

Comparaison avec d’autres Architectures



Architecture Monolithique Classique

✅ Facile à **developper** et à **maintenir** au départ.

❌ Manque de **flexibilité** → tout changement nécessite un **redéploiement complet**.



Architecture Microservices

Excellente **évolutivité** et **indépendance** des services.

❌ Trop **complexe** pour ce projet → multiples bases de données, orchestration difficile.



Architecture Modulaire (choisie)

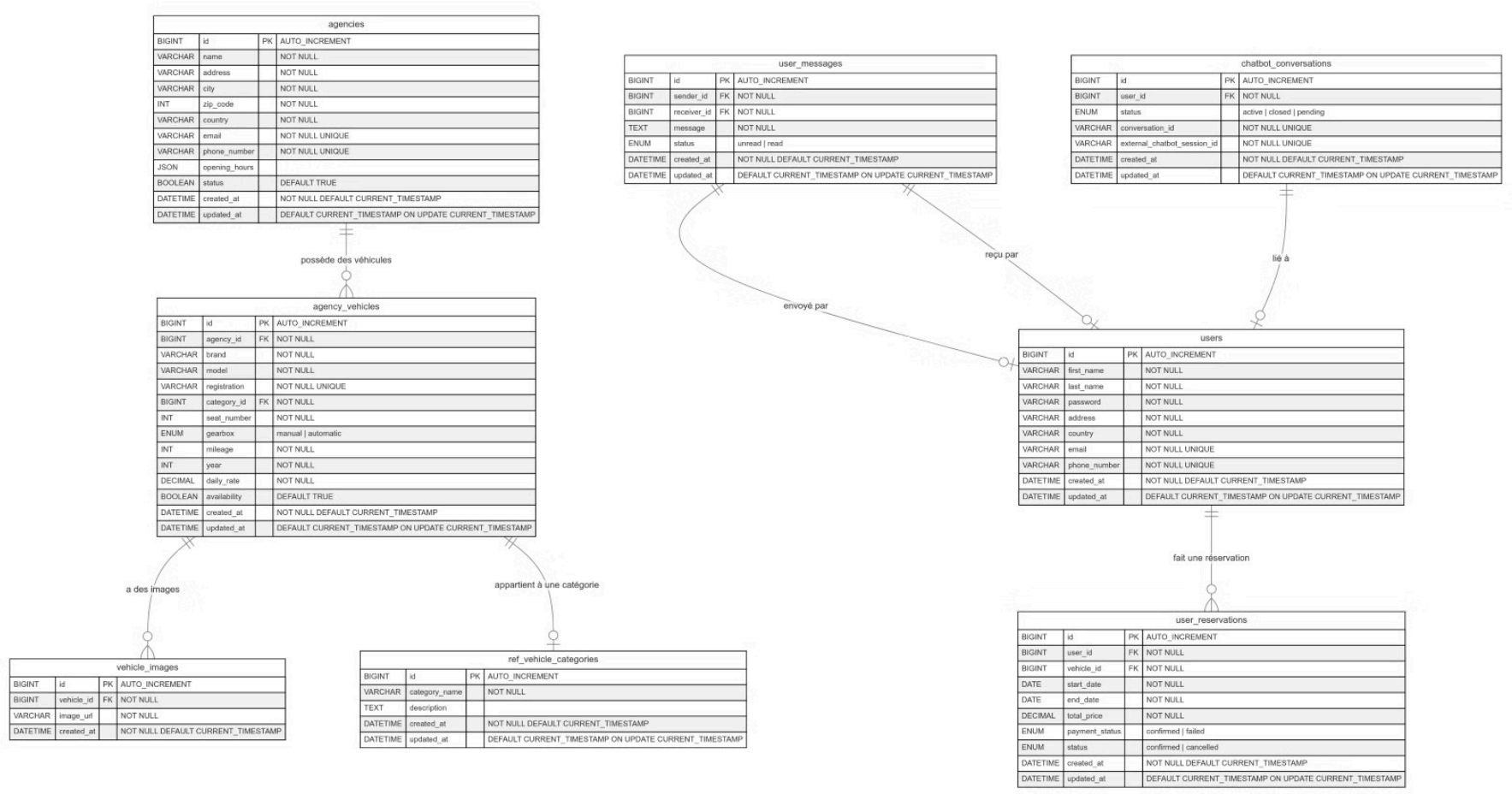
Simplicité du monolithique + **flexibilité** des microservices.

Modules **indépendants** partageant une base de données et un **déploiement unique.**

✅ **Évolutivité** garantie sans complexité excessive.

Made with Gamma

Structuration de la Base de Données



Pourquoi une base bien structurée ?

- Assure la **cohérence et l'intégrité** des données.
- Optimise **les requêtes et la scalabilité**.
- Permet une **intégration fluide** avec les agences via API.

Schéma de la base de données

- Tables principales : **Utilisateurs, Agences, Véhicules, Réservations, Paiements, Messages**.
- **Respect de la normalisation 3NF** → Évite la redondance et garantit des performances optimales.
- API RESTful permettant aux agences d'accéder aux données via **CRUD**.

La **normalisation 3NF (Troisième Forme Normale)** structure les bases de données pour **éviter la redondance** et garantir **l'intégrité des données**. Elle améliore la **cohérence**, **optimise** les requêtes et facilite la **maintenance**.

Made with Gamma

Présentation du POC

Made with Gamma

Conclusion

Pourquoi cette architecture est le meilleur choix ?

- **Répond aux besoins business et techniques** (centralisation, interopérabilité, sécurité).
- **Évolutive** → possibilité de transition vers une architecture microservices si nécessaire.
- **Optimisation des coûts et des délais de développement**.

Perspectives d'évolution

- Ajout de **nouvelles fonctionnalités** et optimisation des performances.
- Renforcement de la sécurité avec **tests unitaires et d'intégrations**.
- Migration potentielle vers **microservices** en fonction de la croissance.

Made with Gamma

Avez-vous des questions ?



Made with Gamma